



Empowering Intelligence

Hailo Dataflow Compiler User Guide



Release 3.32.0
9 July 2025

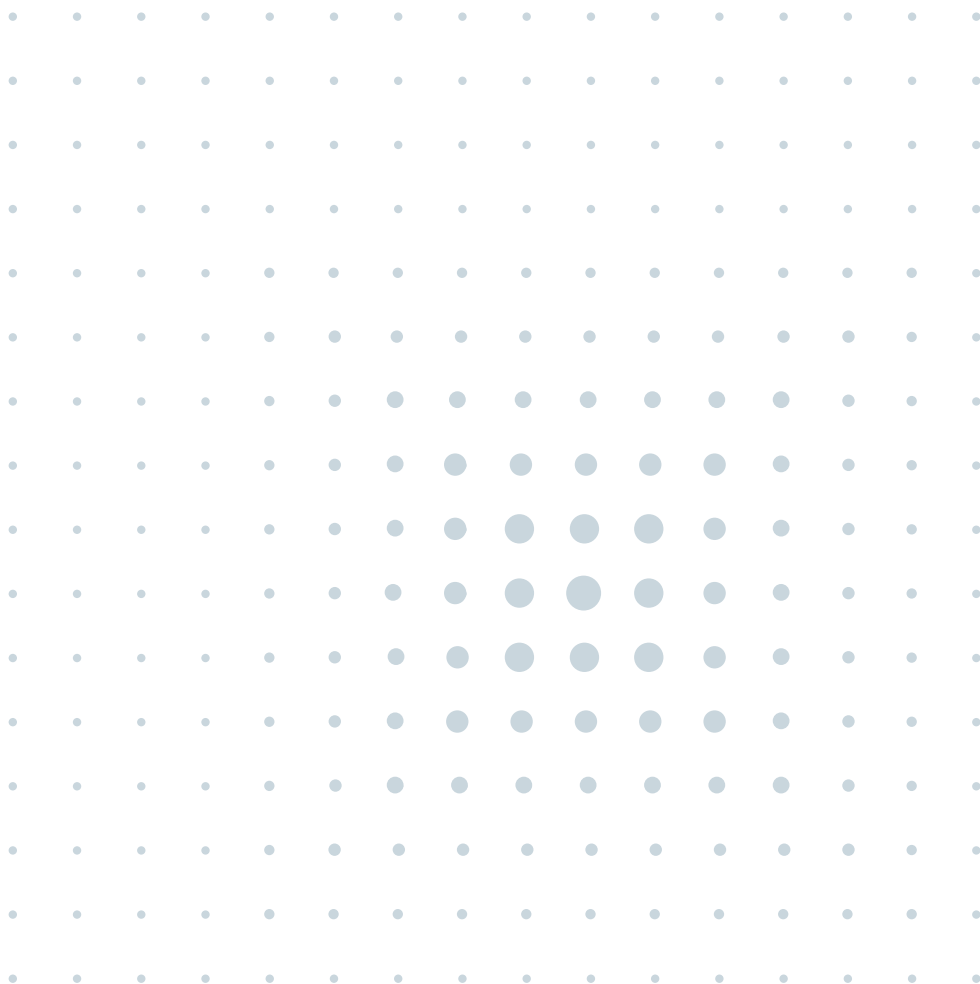


Table of Contents

I	User Guide	2
1	Hailo Dataflow Compiler Overview	3
1.1	Introduction	3
1.2	Model Build Process	3
1.3	Deployment Process	6
1.4	Supported Hardware Architectures	6
2	Changelog	8
3	Dataflow Compiler Installation	21
3.1	System Requirements	21
3.2	Installing / Upgrading Hailo Dataflow Compiler	21
4	Tutorials	23
4.1	Dataflow Compiler Tutorials Introduction	23
4.2	Parsing Tutorial	24
4.3	Model Optimization Tutorial	27
4.4	Compilation Tutorial	41
4.5	Inference Tutorial	42
4.6	Accuracy Analysis Tool Tutorial	46
4.7	Quantization Aware Training Tutorial	51
5	Building Models	58
5.1	Translating Tensorflow and ONNX Models	60
5.2	Model Scripts	73
5.3	Model Optimization	74
5.4	Model Compilation	121
5.5	Supported Layers	132
6	Profiler and Other Command Line Tools	144
6.1	Using Hailo Command Line Tools	144
6.2	Running the Profiler	145
6.3	Using The Dataflow Compiler Studio	153
7	Additional Topics	157
7.1	Environment Variables	157
II	API Reference	158
8	Model Build API Reference	159
8.1	hailo_sdk_client.runner.client_runner	159
8.2	hailo_sdk_client.exposed_definitions	169
8.3	hailo_sdk_client.hailo_archive.hailo_archive	171
8.4	hailo_sdk_client.tools.hn_modifications	171
9	Common API Reference	172
9.1	hailo_sdk_common.model_params.model_params	172
9.2	hailo_sdk_common.hailo_nn.hailo_nn	172
9.3	hailo_sdk_common.hailo_nn.hn_definitions	173
	Bibliography	174
	Python Module Index	175

Disclaimer and Proprietary Information Notice

Copyright

© 2024 Hailo Technologies Ltd ("Hailo"). All Rights Reserved.

No part of this document may be reproduced or transmitted in any form without the expressed, written permission of Hailo. Nothing contained in this document should be construed as granting any license or right to use proprietary information for that matter, without the written permission of Hailo.

This version of the document supersedes all previous versions.

General Notice

Hailo, to the fullest extent permitted by law, provides this document "as-is" and disclaims all warranties, either express or implied, statutory or otherwise, including but not limited to the implied warranties of merchantability, non-infringement of third parties' rights, and fitness for particular purpose.

Although Hailo used reasonable efforts to ensure the accuracy of the content of this document, it is possible that this document may contain technical inaccuracies or other errors. Hailo assumes no liability for any error in this document, and for damages, whether direct, indirect, incidental, consequential or otherwise, that may result from such error, including, but not limited to loss of data or profits.

The content in this document is subject to change without prior notice and Hailo reserves the right to make changes to content of this document without providing a notification to its users.

Part I

User Guide

1. Hailo Dataflow Compiler Overview

1.1. Introduction

The Dataflow Compiler API is used for compiling users' models to Hailo binaries. The input of the Dataflow Compiler is a trained Deep Learning model, the output is a binary file which is loaded to the Hailo device.

The HailoRT API is used for deploying the built model on the target device. This library is used by the runtime applications.

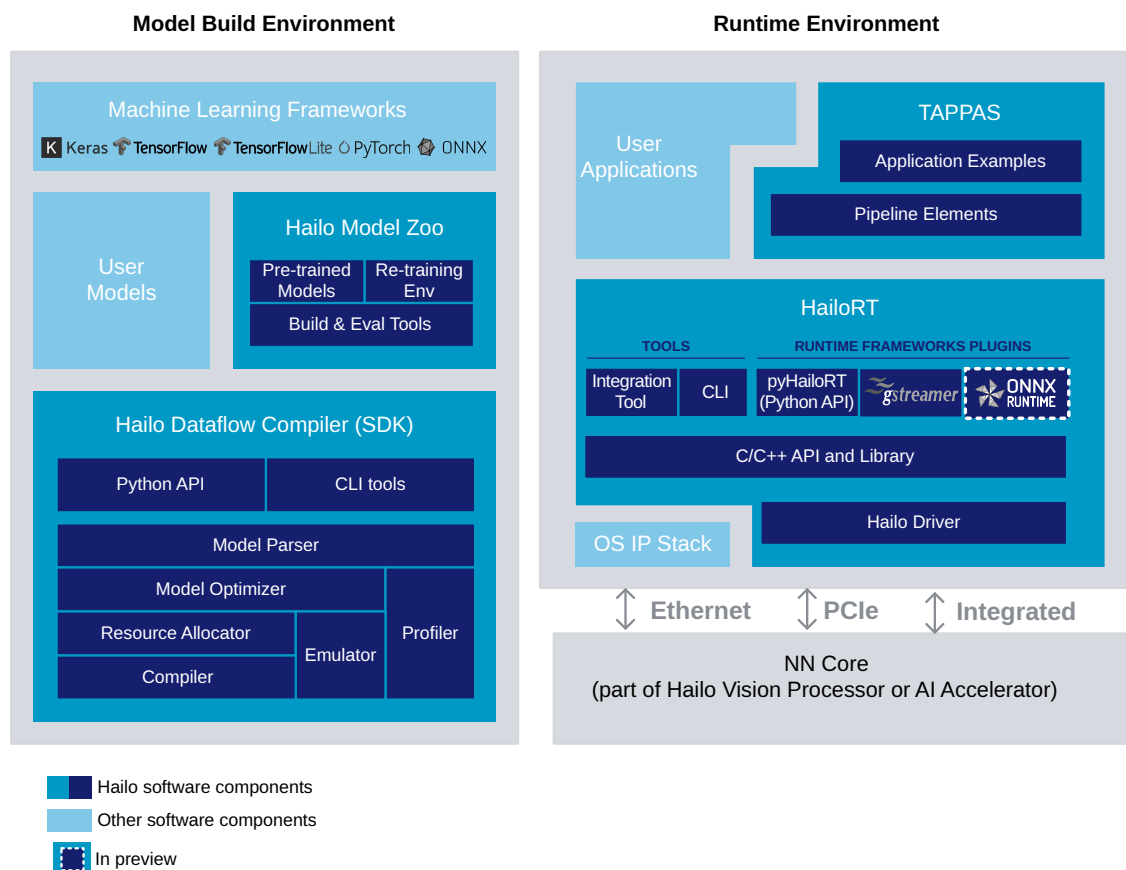


Figure 1. Detailed block diagram of Hailo software packages

1.2. Model Build Process

The Hailo Dataflow Compiler toolchain enables users to generate a Hailo executable binary file (HEF) based on input from a [Tensorflow checkpoint](#), a Tensorflow® frozen graph file, a TFLite file, or an ONNX file. The build process consists of several steps including translation of the original model to a Hailo model, model parameters optimization, and compilation.

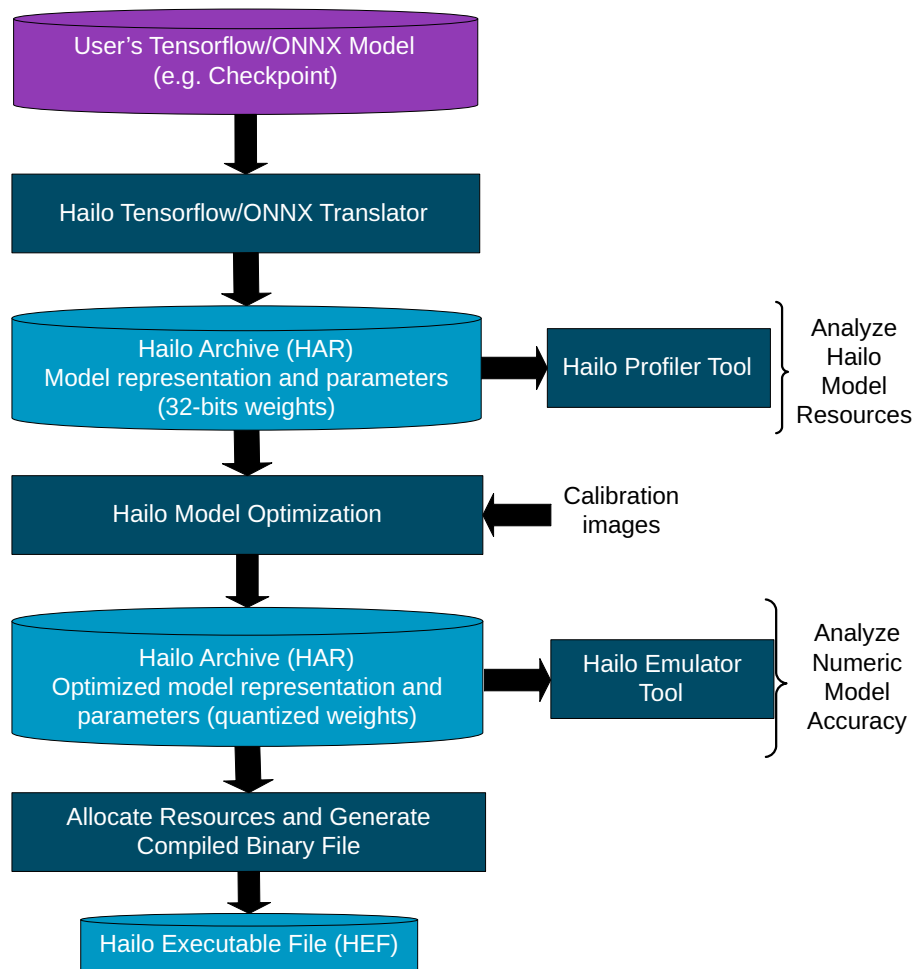


Figure 2. Model build process, starting in a Tensorflow or ONNX model and ending with a Hailo binary (HEF)

1.2.1. Tensorflow and ONNX Translation

After the user has prepared the model in its original format, it can be converted into Hailo-compatible representation files. The translation API receives the user's model and generates an internal Hailo representation format (HAR compressed file, which includes HN and NPZ files). The HN model is a textual JSON output file. The weights are also returned as a NumPy NPZ file.

1.2.2. Profiler

The Profiler tool uses the HAR file and profiles the expected performance of the model on hardware. This includes the number of required devices, hardware resources utilization, and throughput (in frames per second). Breakdown of the profiling figures for each of the model's layers is also provided.

1.2.3. Emulator

The Dataflow Compiler Emulator allows users to run inference on their model without actual hardware. The Emulator supports three main modes: *native* mode, *fp_optimize* mode and *quantized* mode. The native mode runs the original model with float32 parameters, the fp_optimize mode runs with float32 parameters and all model modifications, and the quantized mode provides results that mimics the hardware implementation. Please note that the quantized emulator is not bit-exact to the Hailo hardware, but offers good and fast approximation. The native mode can be used to validate the Tensorflow/ONNX translation process, the fp_optimize mode can be used to validate the model modifications, while the quantized mode can be used to analyze the optimized model's accuracy.

1.2.4. Model Optimization

After the user generates the HAR representation, the next step is to convert the parameters from float32 to integer representation. To convert the parameters, the user should run the model emulation in native mode on a small set of images and collect activation statistics. Based on these statistics, the calibration module will generate a new network configuration for the integer representation. This includes integer weights and biases, scaling configuration, and HW configuration.

1.2.5. Compiling the Model into a Binary Image

Now the model can be compiled into a HW compatible binary format with the extension HEF. The Dataflow Compiler Tool allocates hardware resources to reach the highest possible fps within reasonable allocation difficulty. Then the microcode is compiled and the HEF is generated. This whole step is performed internally, so from the user's perspective the compilation is done by calling a single API.

1.2.6. Dataflow Compiler Studio (Preview - Parsing stage only)

The Dataflow Compiler Studio allows users to parse and visualize neural network graphs efficiently. Users can upload ONNX or TFLite files. As default, the tool provides recommended start and end nodes for the parsing process. In the next screen, the GUI displays a side-by-side comparison of Hailo's parsed graph and the original graph. Users can review these recommended start and end nodes, make adjustments as needed, and re-parse the graph to see the updated results. This interactive process ensures that users can perform parsing to meet their specific requirements. See [Using Dataflow Studio](#) for more details.

1.3. Deployment Process

After the model is compiled, it can be used to run inference on the target device. The HailoRT library provides access to the device in order to load and run the model. This library is accessible from both C/C++ and Python® APIs. It also includes command line tools.

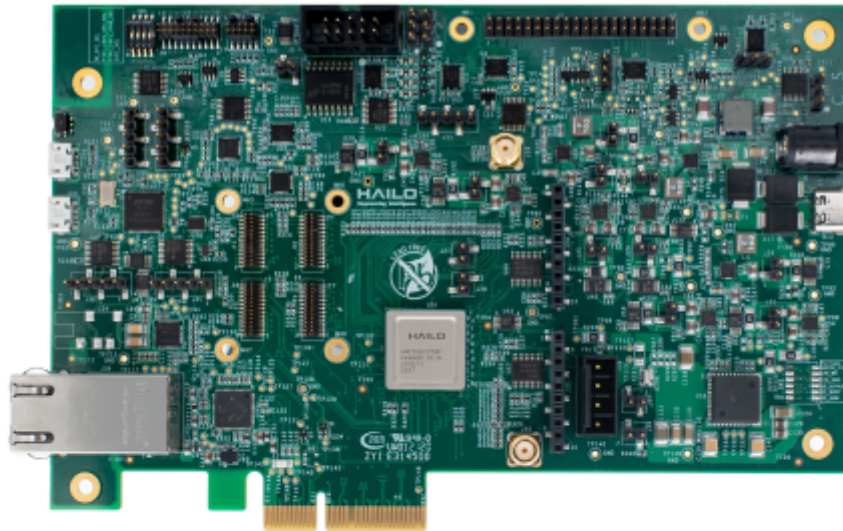
On Hailo-8, if the device is connected to the host through PCIe®, the HailoRT library uses Hailo's PCIe driver to communicate with the device. On Hailo-8, if Ethernet is used, the library uses the Linux® IP stack to communicate.

The HailoRT library can be installed on the same machine as the Dataflow Compiler (on accelerator modules, such as Hailo-8) or on a separate machine. A Yocto™ layer is provided to allow easy integration of HailoRT to embedded environments.

1.4. Supported Hardware Architectures

1.4.1. Hailo-8 family

Hailo-8™ is a series of AI accelerator modules, that allows edge devices to run deep learning applications at full scale more efficiently, effectively, and sustainably than other AI chips and solutions, while significantly lowering costs.



Hailo-8 Evaluation board



Hailo-8 mPCIe board



Hailo-8 M.2 board

Figure 3. Hailo-8 modules

The relevant hardware architecture types that should be used in the compilation process:

hailo8

Use `hw_arch=hailo8` to compile for Hailo-8 based devices, such as: [Hailo-8](#), [Century](#), or custom Chip-on-Board solutions.

This is the default compilation target (unless requested otherwise).

hailo8l

Use `hw_arch=hailo8l` to compile for Hailo-8L device, such as: [Hailo-8L](#), or custom Chip-on-Board solutions.

hailo8r

Use `hw_arch=hailo8r` to compile for the [Hailo-8 Mini PCIe](#) device.

2. Changelog

Dataflow Compiler v3.32.0 (July 2025)

General

- Starting from July 2025, Hailo-10 and Hailo-15 devices will only be supported from Dataflow Compiler versions v5.x and onwards. 3.x versions will continue to support Hailo-8 (only).
- Added support for Ubuntu 24.04, and Python 3.12.3.
- Ubuntu 20.04, and Python 3.8 are no longer supported.
- Tensorflow version was updated to 2.18.0 (CUDA 12.5.1, Cudnn 9.10).

Compiler

- Supports HailoRT version v4.22.0.
- Bug fixes.

Parser

- Deprecated support for parsing TensorFlow 1.x/2.x models (.ckpt/.pb) using all parsing APIs. See user guide for more information and guidelines for moving to Tensorflow Lite.

Package Updates

- Updated CUDA requirement to version 12.5.1, CuDNN to version 9.10.
- Updated Nvidia driver requirement to version 555+.

Dataflow Compiler v3.31.0 (March 2025)

Compiler

- Using XXHASH3_64 for error-detection in HEF files while previously were using CRC32. Older HailoRT versions may not work with HEF files compiled in this version.
- Support new HailoRT version v4.21.0.
- Bug fixes.

Dataflow Compiler v3.30.0 (January 2025)

Package Updates

- Updated ONNX requirement to version 1.16.0.
- Updated ONNXRuntime requirement to version 1.18.0.

Model Optimization

- Accuracy improvements for Transformer Models Quantization.
- Improved quantization support for LayerNorm operator.

Parser

- Parser improvements for Transformer models.
- Added support for new operation: Window Attention (preview).

Compiler

- Added support for new operation: Window Attention (preview).
- Optimization improvements for Transformer models.

- HEF compatibility warning: next DFC version is expected to use different mechanism for error-detecting in HEF files by default (CRC32 to XXHASH3_64), therefore, older HRT versions will not be able to run HEFs compiled by the new DFC version. Current version uses CRC32 by default, but can also support the new format with the following command: `hef_param(hef_signature=xxhash3_64)`.

Deprecated APIs

- Deprecation warning for parsing of TensorFlow 1.x models (.ckpt or .pb files) using all parsing APIs. TensorFlow users are recommended to move to Tensorflow-Lite.

Dataflow Compiler v3.29.0 (September 2024)

Parser

- Added the `input-format` flag to the parser CLI API. This flag gets mapped to the `net_input_format` flag in `translate_onnx_model()`. Command example:

```
hailo parser onnx model.onnx --input-format BWC
```

for an input tensor with shape [B, W, C] (batch, width, channels) (preview).

- Einsum operator support in ONNX was expanded with another equation, `bmchw,bnmc` → `bmhwn`, which represents a group convolution.

Model Optimization

- **Added format conversion support for YCbCr to RGB conversions full details in [input conversion documentation](#).**
 - YCbCr to RGB: allows full range YUV.
 - YUV601 to RGB: complies with BT.601 standard, equivalent to current default YUV to RGB conversion.
 - YUV709 to RGB: complies with BT.709 standard.
- Added a [model script command](#) to allow splitting of a fused activation from its layer, making it a standalone activation.

Tools

- The redesigned [dfc-studio](#) now supports viewing multiple models within the same project.
- New feature in [dfc-studio](#), mapping corresponding layers between the original and Hailo's graph in Parsing stage

Deprecated APIs

- The `tb` and `params-csv` Hailo CLI commands have been deprecated.

Dataflow Compiler v3.28.0 (July 2024)

General

- Hailo Dataflow Compiler now supports Hailo-10H.

Parser

- Added `net_input_format` flag to `translate_onnx_model()`, allowing to explicitly describe the tensor format in ambiguity cases such as [B, N, C] (batch, sequence, channels) or [B, C, N] (batch, channels, sequence) (preview).

Compiler

- Added a [new Compiler flag](#) for allocating very big models, which returns the first feasible partition it finds, allowing to reduce compile time at expense of the solution's quality.

Model Optimization

- Added a new flag to optimization CLI, *-compilation-only-har*, allowing to save a reduced size har, containing only compilation related data.
- Deprecate Auto MO model script.

Tools

- Added preview version of “Dataflow Compile Studio”, new graphical tool for Hailo’s toolchain.
- The current release includes only the parsing step.
- New CLI tool *dfc-studio* that launches the Studio.

Dataflow Compiler v3.27.0 (April 2024)

Parser

- **Added information to logger after model translation is completed:**
 - Mapping input layers to original input node names, to ease creation of feed dict for native inference.
 - Listing output layers by their original names, in the same order specified by the user (or as the original model, if not specified).

Post Processing

- Added support for *NanoDet* <<https://github.com/RangILyu/nanodet>> meta-arch based on YOLOv8 post-processing.
- Added support with post-processing of bbox decoding only in YOLOv5 by using *bbox_decoding_only=True*.
- YOLOv5 SEG NMS for instance segmentation task is supported in all stages of emulation and compilation with *engine=cpu* (preview).

Emulator

- Added emulation support for NV21 and i420 input conversions.

Dataflow Compiler v3.26.0 (January 2024)

General

- Hailo Dataflow Compiler now supports the Hailo-15M device.

Model Optimization

- *Resolution reduction* support for multiple input models.
- **Full Precision Optimization:**
 - Full precision models are serialized to Hailo archive in additional states: QUANTIZED_MODEL, COMPILED_MODEL.
- *output_encoding_vector* added to include a different multiplicative scale for each feature (preview).
- Improve large models optimization time and memory consumption.

Compiler

- Improve compilation time for big models in all hardware architectures for multi-context and single-context networks.

Kernels

- *reduce_sum* is now available also on width and height axis (together).

Post Processing

- YOLOv8 NMS is supported in all stages of emulation and compilation with *engine=cpu*.

Parser

- Added support for *LSTM bidirectional layers* (PyTorch™ and ONNX only) please notice this operator is unrolled by the sequence length which may add large number of layers to the model for large sequence lengths.

Deprecated APIs

- Profiler mode deprecation, Profiler will run it's inherit mode automatically.
- `resize_input` model script command deprecation, use *resize* instead.
- `Har_path` cli flag is deprecated.
- NMS arguments: `clip_boxes` and `normalized_output` are deprecated.

Dataflow Compiler v3.25.0 (October 2023)

General

- Tensorflow version was updated to 2.12.0 (CUDA 11.8, Cudnn 8.9).
- Hailo Dataflow Compiler now supports the newly-released Hailo-15M device

High-level and Documentation

- **Interactive Mode on parser CLI:** allows to retry failed parsing with suggested start/end nodes, or adding auto-detected NMS post-process to model script
- `analyze_noise()` results will be accessed from `get_params_statistics()` only.

Profiler

- A new HTML report template is used by default.
- Supports *optimization-only mode*, to only display optimization-related data (saves compilation time).

Compiler

- Allocation algorithm improvements that result in higher FPS for most models.

Model Modifications

- Added *resize* model script command for applying resize layer on input or output tensor(s).
- *input_conversion* command for NV conversion (nv12, nv21, i420) expects only one returned layer when converting to YUV and two conversion layers when converting to RGB.
- All layers that are added to the model using *input_conversion*, now show up on Netron and Visualizer.
- NMS post-process:
 - Changed default value of *engine* in *nms_postprocess command* for YOLOv5.
 - The value of *nms_scores_th* in the default NMS post-process config json was change from 0.01 to 0.3.
 - When using nms post-process on CPU with default configuration the *nms_iou_th* is changed to 0.6.

Model Optimization

- Ability to *run MO algorithms in reduced resolution*, to decrease running time and RAM consumption.
- Reducing spatial dimensions of Global Average Pool Layers.
 - Automatically performed on large tensors (preview).
 - Can be configured manually using a model script command.
- Full Precision Optimization:

- Added full precision only argument to hailo optimize CLI command, allowing running just the full precision optimizations on a model. Command example: `hailo optimize model.har --full-precision-only --model-script script.all`
- Defuse (split) Multi-head attention blocks to groups for easier compilation, using a [model script command](#).
- Convolution layers are defused (split) automatically by input features if they are large enough, also possible using a [model script command](#).

Parser

- Added support for Softsign activation (PyTorch, Tensorflow, not supported in TFLite).
- Added support with ceil_mode=True in pooling layers (PyTorch and ONNX only).
- Added support for [RNN and LSTM layers](#) (PyTorch and ONNX only), please notice this operator is unrolled by the sequence length which may add large number of layers to the model for large sequence lengths.
- Added support for height-width transpose (PyTorch and ONNX only).
- Added support for OneHot operator (preview level, PyTorch and ONNX only), limited to the last axis.
- Added support for Greater activation (PyTorch and ONNX only), limited to constant value only.
- Added support for [Conv3D](#) and Concat3D (PyTorch and ONNX only) - Preview, limited support - models are assumed to be rank4 input and output.

Deprecated APIs

- Deprecation warning for resize_input model script command, please use [resize](#) instead.
- Profiler:
 - `--use-new-report` flag was deprecated (since the new report is used by default)
 - `profile()` return type will change to a single Python dict type in the near future
 - Deprecation message for `-mode` CLI argument
 - Deprecation message for `profiling_mode` argument of `profile()`
 - `hailo profiler` accepts only HAR path as `model_path` (not an hn path)

Dataflow Compiler v3.24.0 (July 2023)

General

- Hailo Dataflow Compiler now supports the newly-released Hailo-15H device

Model Optimization

- The automatic 16-bit output layer feature is disabled
- System & GPU memory usage optimizations

Kernels and Activations

- 16-bit precision mode can be applied to specific Conv layers inside the model to increase their accuracy

Profiler

- Activation clipping values are showed in the activation histogram plot
- You can use the new profiler HTML design, by appending the `--use-new-report` flag to the CLI command (preview; will be default starting 2023-10)

Parser

- Apply padding correction on Average Pooling layer without external padding
- Start/End Node Name suggestion for models with unsupported ops

- Output layer names order is determined by their order on the parser API

Full Precision Optimization

- Dense layers (fully-connected) input features defuse
 - Automatically performed on large tensors
 - Can be configured manually using a model script command

High-level and Documentation

- NMS auto detection:
 - Detected NMS config saved to native HAR
 - NMS post-process command takes config from auto detection
 - Get auto-detected NMS config using the `get_detected_nms_config()` API
 - If a post-process json configuration files is used (on SSD, for example), the `reg` and `cls` layer names can remain empty, and the auto-detect algorithm will locate them
- Added *set_seed command* for reproducing of quantization results, affects the seed of tensorflow, numpy, and python.random libraries (preview)
- New API - `get_params_statistics()`
- Apply sigmoid automatically whenever is needed:
 - YOLOX - after the objectness and classes layers before the NMS
 - YOLOv5 - between output convolution layers and the NMS
 - SSD - between classes layers and the NMS

Compiler

- Improved *Performance Mode* algorithm
- Improved FPS on models that are compiled to Hailo-15H

Command Line Tools

- *hailo optimize* using RGB images instead of random data when using `-use-random-calib-set`
- *hailo analyze-noise* now saves its results inside the model's HAR

Deprecated APIs

- Deprecation warning for `performance_param(optimization_level=max)`, please use `performance_param(compiler_optimization_level)` instead
- Deprecation warning on the `-analysis-data` argument on *hailo profiler*
- Deprecated `get_tf_graph()` API was removed, please use `infer()`

Known issues

- Refer to [Hailo AI SW Suite: Known Issues](#) page for an updated list of issues

Dataflow Compiler v3.23.0 (April 2023)

Compiler

- Introducing *Performance Mode*, that gradually increases the utilization to achieve the best FPS (preview)
- The compiler has been optimized for better stability and performance

Model Optimization

- Supporting Quantization-Aware-Training using the `set_keras_model()` API. See the *Quantization-Aware-Training Tutorial* for more details
- *Added support* for 16-bit precision on full networks, in case all layers are supported (preview)
- *Optimization levels* are changed to be between 0 (no optimization) and 4 (best plausible optimization), as opposed to 0-3. Their current description is found in the *model_optimization_flavor* API guide
- The default optimization level is now 2 for GPU and 1024 images, 1 for GPU and less than 1024 images, and 0 for CPU only
- Bias Correction algorithm is used as default (optimization_level=1)
- When importing Hailo python libraries, TF memory allocation mechanism is set to “memory growth” by default, to decrease memory consumption. One can override this with an *environment variable*
- Improved the FineTune algorithm for models with multiple output nodes
- 16-bit output layer is enabled automatically when supported, for small output tensors
- When optimization fails, a better error message is displayed, referring to the failing algorithm

Kernels and Activations

- Transformer building block *Multi head Attention* is now supported (preview)
- Increased support for Conv&Add layers

Profiler

- The HTML profiler now displays a quick version of the layer analysis tool (*Accuracy* tab) automatically
- Added `-stream-fps` flag to *hailo profiler*, to be used with single-context models, to evaluate the performance using an FPS which is lower than the network's FPS
- Added `-collect-runtime-data` flag to *hailo profiler*, to automatically infer using *hailortcli* and display runtime data in the report

Emulator

- Added support for emulating YOLOv5 NMS with *engine=cpu*, as well as for SSD
- Added emulation support for RGBX, NV12, NV21 and i420

Parser

- *nms_postprocess command* supports SSD post-processing also on CPU using the ‘engine’ flag (preview)
- Automatic anchors extraction for YOLOv5-type NMS models, using a message is displayed during parsing
- Added support for on-chip i420->YUV conversion, using an *input_conversion* command
- Added support for Biased Delta activation on TFLite, that is implemented using ABS->SIGN->MUL
- Added support for SpaceToDepth kernel that is used on YOLOP
- Added support for Spatial Squeeze operator on TFLite
- Added support for new HardSwish structure in ONNX parser
- Added Global MaxPool operator in ONNX parser
- Fixed a bug in the HardSigmoid implementation

- Added *Hailo-ONNX* support for models with *Shape* connections around the HailoOp
- Added *Hailo-ONNX* support for external inputs to the post-processing section
- Added an option to disable hailo-onnx runtime model build, when it hinders model parsing
- Softmax and Argmax can be added to the model using the *logits_layer* model script command
- Whenever NMS is being added (using a *nms_postprocess* command), Sigmoid is now added automatically
- Added *hybrid conversion* commands on the *input_conversion* section: *yuy2_to_rgb*, *nv12_to_rgb*, *nv21_to_rgb*, *i420_to_rgb*

High-level and Documentation

- Log level can be set using the *LOGLEVEL* environment variable (0 [default] to 3)
- *hailo visualizer* shows layers added using model script commands that were folded
- *hailo visualizer* shows input layers conversion type
- Tutorials are now using *runner.infer* API instead of *runner.get_tf_graph*
- Layer Analysis Tool Tutorial has been updated to demonstrate how to increase accuracy
- Model Optimization Tutorial now uses YOLOv5 NMS with *engine=cpu*, and also a bbox visualization code
- Added description of which optimization algorithms are activated with each *optimization level*
- Removed the Multiple Models Tutorial. The *Join API* is still supported

Command Line Tools

- *hailo analyze* command removed, please use *hailo analyze-noise* instead
- New argument *-analyze-mode* added to *hailo analyze-noise*
- New argument *-disable-rt-metadata-extraction* added to *hailo parser onnx*
- New argument *-version* is added to *hailo*

Deprecated APIs

- Deprecation warning for *get_tf_graph()*, please use *infer()*
- *optimize()* is not allowed under *QUANTIZED_MODEL*
- Added *analyze_mode* to argument *analyze_noise()*
- Added *disable_rt_metadata_extraction* argument to *translate_onnx_model()*
- Deprecation warning for *quantization_params* and *compilation_params* arguments from *translate_onnx_model()* and *translate_tf_model()*, please use model script commands *quantization_param* and *compilation_param* instead
- The following ClientRunner APIs are now deprecated: *get_results_by_layer*, *update_params_layer_bias*, *profile_hn_model*, *get_mapped_graph*, *get_params_after_bn*, *set_original_model*, *apply_model_modification_commands*
- Removed deprecated argument *ew_add_policy* from *translate_onnx_model()* and *translate_tf_model()*
- Removed *dead_channels_removal_from_runner* API
- Deprecated *scores_scale_factor* argument to SSD post-process JSON file, use *bbox_dimensions_scale_factor* instead
- Deprecation warning for *context_switch_param* command parameters of type: *goal_network_X*

Known issues

- Some Transformer models are at risk for having a runtime bug when inferring with *batch_size > 1*, when multi-context allocation is used a workaround is to use the *max_utilization* parameter of *context_switch_param* command to change the failing context partition

- In some cases, using the Fine Tune algorithm when the whole network is quantized to 16-bit might cause a degradation

Dataflow Compiler v3.22.1 (February 2023)

Parser

- Fixed an issue where a model script had to be provided explicitly to *hailo compiler* when an NMS command was used
- Added support for Global Maxpool operator in ONNX parser
- Fixed a parsing issue in Hardswish activation
- Fixed an issue that has prevented YOLOv8 from parsing

Compiler

- Fixed prints to screen during compilation, regarding single/multi context flow and resources utilization
- Removed the warning message of using on-chip NMS with multi context allocation, since the new version of HailoRT fixes the issue

Dataflow Compiler v3.22.0 (January 2023)

Package Updates

- Added support for Ubuntu 22.04, Python 3.9, and Python 3.10
- Ubuntu 18.04, Python 3.6 and Python 3.7 are no longer supported
- Updated Tensorflow requirement to version 2.92
- Updated ONNX requirement to version 1.12.0
- Updated ONNXRuntime requirement to version 1.12.0

Profiler

- Introducing Accuracy Tab on the [HTML Profiler](#), to be used as a tool to analyze and improve accuracy
- Profiler in post-placement mode doesn't require .hef file, when working on a compiled .har file
- Profiler will apply model modifications on pre_placement mode, if a model script was supplied
- `profile()` API will not update the runner state, even if it compiles for the profiling process
- Bug fixes

Model Optimization

- ClientRunner now has a new `SdkFP Optimized` state (see [runner states diagram](#)), for assessing model accuracy before quantization
- Updated the [Model Optimization workflow](#) section with simple and advanced optimization flows
- Updated the [Model Optimization Tutorial](#) with step-by-step instructions for validating accuracy through the optimization and compilation phases
- Updated the [Layer Analysis Tool tutorial](#) to utilize the new HTML profiler Accuracy tab

Emulator

- Added Emulator support for YUY2 color conversions, using 'emulator_support=True' flag on the [input_conversion](#) command

Kernels and Activations

- Added support for on-chip NV12->YUV, NV21->YUV and YUV->BGR format conversions, using an [input_conversion](#)

- Further increased support for *Resize Bilinear layers*
- *Nearest Neighbor Resize* now supports downsampling
- Added support for ReduceSumSquare operator
- Add support for EfficientGCN pooling block

Parser

- *nms_postprocess command* now supports 'engine' flag, that instructs HailoRT to complete YOLOv5 NMS post-processing on the host platform (preview)
- Enhanced the suggestion for end-node names
- Added support for Less operator in both ONNX and Tensorflow parsers
- Add support for dual broadcast in element-wise mult ($H \times 1 \times C * 1 \times W \times C \rightarrow H \times W \times C$)
- Added support for multiplication by 0 in all frameworks ($x * 0$, $x * 0 + b$, $(x + b) * 0$)
- Added support for depthwise with depth multiplier as group convolution in TFLite
- Add support for ADD_N from TFLite models

Compiler

- Optimized the compiler for better stability and performance
- Bug fixes

Known Bugs

- On this version, on-chip YOLOv5 NMS needs to be compiled using the legacy `fps` command.

API

- `nms_postprocess` model script command now uses relative paths relative to the `alls` script location. In addition, when working with a HAR file that has model script inside, it uses the json from within the HAR
- On `nms_postprocess` model script command, changed the 'yolo' meta_arch to be 'yolov5'
- Layer Analysis Tool now exports its data to a json file, that *could be used with the HTML profiler* to unlock the new Accuracy tab
- ClientRunner APIs
 - New
 - * `analyze_noise`
 - * `optimize_full_precision`
 - Argument changes
 - * New: `analysis_data` in `profile` and `profile_hn_model`
 - * Deprecation warning: `fps` flag in all APIs that compile (`profile_hn_model`, `get_tf_graph`)
 - * Deprecation warning: `ew_add_policy` in `translate_onnx_model`, `translate_tf_model`
 - * Deprecation warning: `apply_model_modifications`
 - * Removed: `model_script_filename` in `load_model_script`
 - * Removed: `is_frozen`, `start_node_name`, `nn_framework` in `translate_tf_model`
 - * Removed: `start_node_name`, `net_input_shape`, `onnx_path` in `translate_onnx_model`
 - Removed
 - * `quantize`
 - * `equalize_params`
 - * `get_hw_representation`

- ✱ revert_state
- Deprecation warning
 - ✱ get_results_by_layer
 - ✱ translate_params
 - ✱ update_params_layer_bias
 - ✱ profile_hn_model
 - ✱ get_mapped_graph
 - ✱ get_params_after_bn
 - ✱ set_original_model
 - ✱ apply_model_modification_commands
- High level APIs
 - add_nms_postprocess (not using a model script command) - removed
 - dead_channels_removal_from_runner - deprecation warning
- CLI tools
 - analyze was renamed to analyze-noise
 - ✱ -data_path renamed to -data-path
 - ✱ -eval-num renamed to -data-count
 - ✱ -calib_path, -alls-path, -quant-mode, -layers, -inverse, -ref-target, -test-target, -analyze-mode removed
 - ✱ old flags exist under analyze command
 - compiler
 - ✱ -alls renamed to -model-script
 - ✱ -auto-alls-path renamed to -auto-model-script
 - har
 - ✱ revert removed
 - parser
 - ✱ ckpt, tf2 removed (just use hailo parser tf *FILE*)
 - ✱ -force-pb and -force-ckpt removed from parser tf
 - profiler
 - ✱ -fps removed
 - ✱ -alls renamed to -model-script
 - ✱ -analysis-data added

Dataflow Compiler v3.20.1 (November 2022)

Parser

- Added support for custom TFLite operators that implement a biased delta activation
- Added support for rank-2 HardSwish activation
- Optimized HardSwish and Gelu implementation
- Added support for the self operators add(x,x), concat(x,x), and mul(x,x) in the TF and ONNX parsers
- Pinned jsonref package to version 0.3.0 to fix installation error

Dataflow Compiler v3.20.0 (October 2022)

Model Optimization

- FPS is improved for large models by Quantization to 4-bit for 20% of the model weights is *enabled by default* on large networks to improve FPS

Kernels and Activations

- Added on-chip support for RGBX->RGB conversion using *input conversion command*
- Added support for ONNX operator InstanceNormalization
- Added support for L2 Normalization layer on TensorFlow

Compiler

- Optimized the performance of compiled models

Parser

- Added a recommendation to use onnxsimpifier when parsing fails
- Added a recommendation to use TFLite parser if TF2 parsing fails (see *conversion guide*, on 4.2.5)
- *TensorFlow parser* detects model type automatically

High Level

- **Refactor logger**
 - Cleaned info and warning messages
 - Log files are duplicated into activated_virtualenv/etc/hailo/
 - Log files could be disabled by an *environment variable*
- *HTML Profiler* report includes model optimization information: compression and optimization levels, model modifications, weight and activation value ranges
- Dataflow Compiler is tested on Windows 10 with WSL2 running Ubuntu 20.04

API

- Compiler automatically separates different connected components to multiple network groups
 - Mostly relevant for *joined networks* with join_action=JoinAction.NONE
 - HailoRT API can be used to activate/deactivate each network group, although it is recommended to use the Scheduler API because it automatically switches between network groups (and .hef files)
 - For more information refer to network_group model script command
- *ClientRunner.compile()* API is introduced (planned to replace runner.get_hw_representation) (preview)
- Updated *platform_param* model script command to optimize compilation for low PCIe bandwidth hosts
- *Model script command for adding NMS on chip* is simplified (preview)

- Deprecation warning for the legacy `-fps` argument, use `performance_param` model script command instead
- Removed the already-deprecated APIs
 - `integrated_preprocess` and `ckpt_path` arguments from `ClientRunner` methods
 - Removed `har-modifier` CLI, and the following related methods: `add_nms_postprocess_from_hn`, `add_nms_postprocess_from_har`, `dead_channels_removal_from_har`, `transpose_hn_height_width_from_hn`, `transpose_hn_height_width_from_har`, `add_yuv_to_rgb_layers`, `add_yuv_to_rgb_layers_from_har`, `add_resize_input_layers`, `add_resize_input_layers_from_har`
 - `npz-csv` (use `params-csv` instead)
- As the parser detects Tensorflow1/2/TFLite automatically, the API for specifying the framework is deprecated
- The argument `onnx_path` of `ClientRunner.translate_onnx_model` was renamed to `model`, and also supports 'bytes' format
- `ClientRunner.load_model_script` can receive either a file object or a string

Note: Ubuntu 18.04 will be deprecated in Hailo Dataflow Compiler future version

Note: Python 3.6 will be deprecated in Hailo Dataflow Compiler future version

3. Dataflow Compiler Installation

Note: This section describes the installation of the Dataflow Compiler **only**. For a complete description of the installation of Hailo Suite, which contains all Hailo SW products, please refer to the [Hailo AI SW Suite User Guide](#).

3.1. System Requirements

The Hailo Dataflow Compiler requires the following minimum hardware and software configuration:

1. Ubuntu® 20.04/22.04, 64-bit (supported also on Windows, under WSL2)
2. 16+ GB RAM (32+ GB recommended)
3. Python 3.8/3.9/3.10, including `pip` and `virtualenv`
4. `python3.X-dev` (according to the Python version), `python3-tk`, `graphviz`, and `libgraphviz-dev` packages. Use the command `sudo apt-get install PACKAGE` for installation.
5. (Optional) Node.js (minimum version: v20.9.0) to start DFC Studio part of SDK package.

The following additional requirements are needed for GPU based hardware emulation:

1. Nvidia's Pascal/Turing/Ampere GPU architecture (such as Titan X Pascal, GTX 1080 Ti, RTX 2080 Ti, or RTX A4000)
2. GPU driver version 555+
3. CUDA 12.5.1
4. CUDNN™ 9.10

Note: The Dataflow Compiler installs and runs Tensorflow, however when Tensorflow is installed from PyPi and runs on the CPU, it will also require AVX instruction support. Therefore, it is recommended to use a CPU that supports AVX instructions. Another option is to compile Tensorflow from sources without AVX.

Warning: These requirements are for the Dataflow Compiler, **which is used to build models**. Running inference using HailoRT works on smaller systems as well. In order to run inference and demos on a Hailo device, the latest [HailoRT](#) needs to be installed as well. See [HailoRT's user guide](#) for more details.

3.2. Installing / Upgrading Hailo Dataflow Compiler

Warning: This installation requires an internet connection (or a local pip server) in order to download Python packages.

Note: If you wish to upgrade both Hailo Dataflow Compiler and HailoRT which are installed in the same virtualenv, update HailoRT first, and then the Dataflow Compiler using the following instructions.

Hailo Dataflow Compiler's Wheel file (.whl) can be downloaded from [Hailo's Developer Zone](#).

For a clean installation

1. Create a virtualenv:

```
virtualenv <VENV_NAME>
```

2. Enter the virtualenv:

```
. <VENV_NAME>/bin/activate
```

3. When inside the virtualenv, use (for 64-bit linux):

```
pip install <hailo_dataflow_compiler-X.XX.X-py3-none-linux_x86_64.whl>
```

4. Perform one of the options:

If the previous version is already installed (v3.15.0 or newer), enter the virtualenv, and install using the line above. The old version will be updated automatically.

If only older versions are installed (<=3.14.0), you have to uninstall it manually from within the existing virtualenv:

```
pip uninstall -y hailo_sdk_common hailo_sdk_client hailo_sdk_server hailo_model_
↪ optimization
```

Install the new package with pip using the method above (the package names were changed from v3.14.0 to v3.15.0).

After installation / upgrade, it is recommended to view Hailo's CLI tool options with:

```
hailo -h
```

Note: You can validate the success of the install/update to latest Hailo packages, by running `pip freeze | grep hailo`.

4. Tutorials

The tutorials below go through the model build and inference steps. They are also available as Jupyter notebook files in the directory `VENV/lib/python.../site-packages/tutorials`.

It's recommended to use the command `hailo tutorial` (when inside the virtualenv) to open a Jupyter server that contains the tutorials.

4.1. Dataflow Compiler Tutorials Introduction

The tutorials cover the Hailo Dataflow Compiler basic use-cases:

Model Compilation:

It is recommended to start with the [Hailo Dataflow Compiler Overview / Model build process](#) section of the user guide.

The Hailo compilation process consists of three steps:

1. Converting a Tensorflow or ONNX neural-network graph into a Hailo-compatible representation.
2. Quantization of a full precision neural network model into an 8-bit model.
3. Compiling the network to binary files (HEF), for running on the Hailo device.

Inference:

1. Blocking inference with the HW-compatible model.
2. Streaming inference with the HW-compatible model.
3. Inference inside a Tensorflow environment.

These use-cases were chosen to show an end-to-end flow, beginning with a Tensorflow / ONNX model and ending with a hardware deployed model.

Throughout this guide the Resnet-v1-18 neural network will be used to demonstrate the capabilities of the Dataflow Compiler. The neural network is defined using Tensorflow checkpoint.

4.1.1. Usage

The HTML and PDF versions are for viewing-only. The best way to use the tutorials is to run them as Jupyter notebooks:

1. The Dataflow Compiler should be installed, either as a standalone Python package, or as part of the Hailo SW Suite.
2. Activate the Dataflow Compiler virtual environment using `source <virtualenv_path>`
 1. When using the Suite docker, the virtualenv is activated automatically.
3. The tutorial notebooks are located in: `VENV/lib/python.../site-packages/hailo_tutorials`.
4. Running the command `hailo tutorial` will open a Jupyter server that allows viewing the tutorials locally by using the link given at the output of the command.
5. Remote viewing from a machine different then the one used to run the Jupyter server is also possible by running `hailo tutorial --ip=0.0.0.0`

4.2. Parsing Tutorial

4.2.1. Hailo Parsing Examples from TensorFlow/Pytorch to HAR

This tutorial describes the steps for parsing models from various frameworks to the HAR format (Hailo Archive).

HAR is a tar.gz archive file that contains the representation of the graph structure and the weights that are deployed to Hailo's runtime.

Note: **Running this code in Jupyter notebook is recommended**, see the Introduction tutorial for more details.

Note: This section demonstrates the Python APIs for Hailo Parser. You could also use the CLI: `hailo parser {tf, onnx} --help`.

More details on Dataflow Compiler User Guide / Building Models / Profiler and other command line tools.

```
[ ]: # General imports used throughout the tutorial
import tensorflow as tf
from IPython.display import SVG

# import the ClientRunner class from the hailo_sdk_client package
from hailo_sdk_client import ClientRunner
```

Set the hardware architecture to be used throughout the tutorial:

```
[ ]: chosen_hw_arch = "hailo8"
# For Mini PCIe modules or Hailo-8R devices, use 'hailo8r'
```

4.2.2. Parsing Example from ONNX to HAR

Choose the ONNX file to be used throughout the example:

```
[ ]: onnx_model_name = "resnet_v1_18"
onnx_path = "../models/resnet_v1_18.onnx"
```

The main API of the Dataflow Compiler that the user interacts with is the ClientRunner class (see the API Reference section on the Dataflow Compiler user guide for more information).

Initialize a ClientRunner and use the `translate_onnx_model` method.

Arguments:

- `model_path`
- `model_name` to use
- `start_node_names` (list of str, optional): Name of the first ONNX node to parse.
- `end_node_names` (list of str, optional): List of ONNX nodes, that the parsing can stop after all of them are parsed.
- `net_input_shapes` (dict, optional): A dictionary describing the input shapes for each of the start nodes given in `start_node_names`, where the keys are the names of the start nodes and the values are their corresponding input shapes. Use only when the original model has dynamic input shapes (described with a wildcard denoting each dynamic axis, e.g. [b, c, h, w]).

As a suggestion try translating the ONNX model without supplying the optional arguments.

```
[ ]: runner = ClientRunner(hw_arch=chosen_hw_arch)
hn, npz = runner.translate_onnx_model(
    onnx_path,
    onnx_model_name,
    start_node_names=["input.1"],
    end_node_names=["192"],
    net_input_shapes={"input.1": [1, 3, 224, 224]},
)
```

4.2.3. Hailo Archive

Hailo Archive is a tar.gz archive file that captures the “state” of the model - the files and attributes used in a given stage from parsing to compilation. Use the `save_har` method to save the runner’s state in any stage and `load_har` method to load a saved state to an uninitialized runner.

The initial HAR file includes: - HN file, which is a JSON-like representation of the graph structure that is deployed to the Hailo hardware. - NPZ file, which includes the weights of the model.

Save the parsed model in a Hailo Archive file:

```
[ ]: hailo_model_har_name = f"{onnx_model_name}_hailo_model.har"
runner.save_har(hailo_model_har_name)
```

Visualize the graph with Hailo’s visualizer tool:

```
[ ]: !hailo visualizer {hailo_model_har_name} --no-browser
SVG("resnet_v1_18.svg")
```

4.2.4. Parsing Example from TensorFlow Lite

The Hailo parser supports inference models as inputs, therefore we advise to use TensorFlow Lite representation for TensorFlow 2 models (TF2 SavedModel format is commonly used for training models).

Parsing the TensorFlow Lite format is similar to parsing ONNX models.

The parser identifies the input format automatically.

The following example shows how to parse a TensorFlow Lite model, using a different model.

```
[ ]: model_name = "dense_example"
model_path = "../models/v3-large-minimalistic_224_1.0_float.tflite"

runner = ClientRunner(hw_arch=chosen_hw_arch)
hn, npz = runner.translate_tf_model(model_path, model_name)
```

4.2.5. Common Conversion Methods from Tensorflow to Tensorflow Lite

The following examples focus on Tensorflow's TFLite converter support for various TF formats, showing how older formats of TF can be converted to TFLite, which can then be used in Hailo's parsing stage.

```
[ ]: # Build a simple Keras model and convert it to tflite
def build_small_example_net():
    inputs = tf.keras.Input(shape=(24, 24, 96), name="img")
    x = tf.keras.layers.Conv2D(24, 1, name="conv1")(inputs)
    x = tf.keras.layers.BatchNormalization(momentum=0.9, name="bn1")(x)
    outputs = tf.keras.layers.ReLU(max_value=6.0, name="relu1")(x)
    model = tf.keras.Model(inputs, outputs, name="small_example_net")
    return model

# Converting the Model to tflite
model = build_small_example_net()
model_name = "small_example"
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.target_spec.supported_ops = [
    tf.lite.OpsSet.TFLITE_BUILTINS, # enable TensorFlow Lite ops.
    tf.lite.OpsSet.SELECT_TF_OPS, # enable TensorFlow ops.
]
tflite_model = converter.convert() # may cause warnings in jupyter notebook, don't
    ↪worry.
tflite_model_path = "../models/small_example.tflite"
with tf.io.gfile.GFile(tflite_model_path, "wb") as f:
    f.write(tflite_model)

# Parsing the model to Hailo format
runner = ClientRunner(hw_arch=chosen_hw_arch)
hn, npz = runner.translate_tf_model(tflite_model_path, model_name)
```

```
[ ]: # Alternatively, convert an already saved TF2.x SavedModel to tflite
model_path = "../models/dense_example_tf2/"
model_name = "dense_example_tf2"
converter = tf.lite.TFLiteConverter.from_saved_model(model_path)
converter.target_spec.supported_ops = [
    tf.lite.OpsSet.TFLITE_BUILTINS, # enable TensorFlow Lite ops.
    tf.lite.OpsSet.SELECT_TF_OPS, # enable TensorFlow ops.
]
tflite_model = converter.convert() # may cause warnings in jupyter notebook, don't
    ↪worry.
tflite_model_path = "../models/dense_example_tf2.tflite"
with tf.io.gfile.GFile(tflite_model_path, "wb") as f:
    f.write(tflite_model)

# Parsing the model to Hailo format
runner = ClientRunner(hw_arch=chosen_hw_arch)
hn, npz = runner.translate_tf_model(tflite_model_path, model_name)
```

4.3. Model Optimization Tutorial

This tutorial describes the process of optimizing the user's model. The input to this tutorial is a HAR file in Hailo Model state (before optimization; with native weights) and the output will be a quantized HAR file with quantized weights.

Note: For full information about Optimization and Quantization, refer to the Dataflow Compiler user guide / Model optimization section.

Requirements:

- Run this code in Jupyter notebook. See the Introduction tutorial for more details.
- The user should review the complete Parsing Tutorial (or created the HAR file in other way)

Recommendation:

- To obtain best performance run this code with a GPU machine. For full information see the Dataflow Compiler user guide / Model optimization section.

Contents:

- Quick optimization tutorial
- In-depth optimization & evaluation tutorial
- Advanced Model Modifications tutorial
- Compression and Optimization levels

```
[ ]: # General imports used throughout the tutorial
# file operations
import json
import os

import numpy as np
import tensorflow as tf
from IPython.display import SVG
from matplotlib import patches
from matplotlib import pyplot as plt
from PIL import Image
from tensorflow.python.eager.context import eager_mode

# import the hailo sdk client relevant classes
from hailo_sdk_client import ClientRunner, InferenceContext

%matplotlib inline

IMAGES_TO_VISUALIZE = 5
```

4.3.1. Quick Optimization Tutorial

After the HAR file has been created (using either `runner.translate_tf_model` or `runner.translate_onnx_model`), the next step is to go through the optimization process.

The basic optimization is performed just by calling `runner.optimize(calib_dataset)` (or the CLI `hailo optimize` command), as described on the user guide on: Building Models / Model optimization / Model Optimization Workflow. The calibration dataset should be preprocessed according to the model's input requirements and it is recommended to have at least 1024 inputs and to use a GPU. During this step it is also possible to use a model script which change the default behavior of the Dataflow Compiler, for example, to add additional layer for normalization. All the model script available commands are described in the user guide on: Building Models / Model optimization / Optimization Related Model Script Commands.

In order to learn how to deal with common pitfalls, image formats and accuracy, refer to the in-depth section.

```
[ ]: # First, we will prepare the calibration set. Resize the images to the correct size and
↳ crop them.
def preproc(image, output_height=224, output_width=224, resize_side=256):
    """imagenet-standard: aspect-preserving resize to 256px smaller-side, then
    ↳ central-crop to 224px"""
    with eager_mode():
        h, w = image.shape[0], image.shape[1]
        scale = tf.cond(tf.less(h, w), lambda: resize_side / h, lambda: resize_side / w)
        resized_image = tf.image.resize(tf.expand_dims(image, 0), [int(h * scale), int(w
↳ * scale)])
        cropped_image = tf.image.resize_with_crop_or_pad(resized_image, output_height,
↳ output_width)

        return tf.squeeze(cropped_image)

images_path = "../data"
images_list = [img_name for img_name in os.listdir(images_path) if os.path.
↳ splitext(img_name)[1] == ".jpg"]

calib_dataset = np.zeros((len(images_list), 224, 224, 3))
for idx, img_name in enumerate(sorted(images_list)):
    img = np.array(Image.open(os.path.join(images_path, img_name)))
    img_preproc = preproc(img)
    calib_dataset[idx, :, :, :] = img_preproc.numpy()

np.save("calib_set.npy", calib_dataset)
```

```
[ ]: # Second, we will load our parsed HAR from the Parsing Tutorial

model_name = "resnet_v1_18"
hailo_model_har_name = f"{model_name}_hailo_model.har"
assert os.path.isfile(hailo_model_har_name), "Please provide valid path for HAR file"
runner = ClientRunner(har=hailo_model_har_name)
# By default it uses the hw_arch that is saved on the HAR. For overriding, use the hw_
↳ arch flag.
```

```
[ ]: # Now we will create a model script, that tells the compiler to add a normalization on
↳ the beginning
# of the model (that is why we didn't normalize the calibration set;
# Otherwise we would have to normalize it before using it)

# Batch size is 8 by default
alls = "normalization1 = normalization([123.675, 116.28, 103.53], [58.395, 57.12, 57.
↳ 375])\n"

# Load the model script to ClientRunner so it will be considered on optimization
runner.load_model_script(alls)

# Call Optimize to perform the optimization process
runner.optimize(calib_dataset)

# Save the result state to a Quantized HAR file
quantized_model_har_path = f"{model_name}_quantized_model.har"
runner.save_har(quantized_model_har_path)
```

That concludes the quick tutorial.

4.3.2. In-Depth Optimization Tutorial

The advanced optimization process (see the diagram in the user guide on: Building Models / Model optimization / Model Optimization Workflow), is comprised of the following steps:

1. Test the parsed `Native model` before any changes are made (still on floating point precision), check to see that the pre and post processing code works well with the start and end nodes provided. The `Native model` will match the results of the original model, in between the `start_node_names` and the `end_node_names` provided by the user during the Parsing stage.
2. Optional: Apply Model Modifications (like input Normalization layer, YUY2 to RGB conversion, changing output activations and others), using a `model script`.
3. Test the `FP Optimized model` (the model after floating point operations and modifications) to see that required results have been achieved.
 - Note: Remember to update the pre and post processing code to match the changes in the model. For example, if normalization has been added to the model, remove the normalization code from the pre-processing code, and feed un-normalized images to the model. If softmax has been added onto the outputs, remove the softmax from the post-processing code. Etc.
4. Now perform `Optimization` to the model, using a calibration set that has been prepared. The result is a `Quantized model`, that has some degradation compared to the pre-quantized model.
 - Note: The format of calibration set is the same as was used as inputs for the modified model. For example, if a normalization layer has been added to the model, the calibration set should not be normalized. If this layer has not been added yet, pre-process and normalize the images.
5. Test the quantized model using the same already-validated code for the pre and post processing.
 - If there is a degradation, it is due to the quantization process and not due to input/output formats, as they were already verified with the pre-quantized model.
6. To increase the accuracy of the quantized model, it is possible to optimize again using a `model script` to affect the optimization process.
 - Note: The most basic method is to raise the `optimization_level`, an example model script command is `model_optimization_flavor(optimization_level=4)`. The advanced method is to use the Layer Analysis Tool, presented on the next tutorial.
 - Note: If the accuracy is good, consider increasing the performance by using 4-bit weights. This is done using `compression_level`, an example model script command is `model_optimization_flavor(compression_level=2)`.
7. During the next tutorials, compilation and on-device inference, input and output values are expected to be similar to the quantized model's values.

The testing (whether on `Native`, `Modified` or `Quantized model`) is performed using our `Emulator` feature, that will be described in this tutorial.

To further understand the advanced optimization process, the following steps are described below.

Preliminary step: Create testing environment

Hailo offers an `Emulator` for testing the model in its different states.

The emulator is implemented as a Tensorflow graph, and its results are the return value of `runner.infer(context, network_input)`.

To get inference results, run this API within the context manager

```
runner.infer_context(inference_context) where the inference context is one of:
[InferenceContext.SDK_NATIVE, InferenceContext.SDK_FP_OPTIMIZED,
InferenceContext.SDK_QUANTIZED];
```

- `InferenceContext.SDK_NATIVE`: Testing method for Step 1 of the optimization process steps (Native model). Runs the model as is without any changes. Use it to make sure the model has been converted properly into Hailo's internal representation. Should yield exact results as the original model.
- `InferenceContext.SDK_FP_OPTIMIZED`: Testing method for Step 3 of the optimization process steps (Modified model). The modified model represents the Hailo model prior to quantization, and is the result of performing model modifications (e.g. normalizing/resizing inputs) and full precision optimizations (e.g. tiled squeeze & excite, equalization). As a result, inference results may vary slightly from the native results.
- `InferenceContext.SDK_QUANTIZED`: Testing method for Step 5 of the optimization process steps (Quantized model). This inference context emulates the hardware implementation, and is useful for measuring the overall accuracy and degradation of the quantized model. This measurement is performed against the original model over large datasets, prior to running inference on the actual Hailo device.

Preliminary Step: Create Pre and Post Processing Functions

```
[ ]: # -----
# Pre processing (prepare the input images)
# -----
def preproc(image, output_height=224, output_width=224, resize_side=256,
    normalize=False):
    """imagenet-standard: aspect-preserving resize to 256px smaller-side, then
    central-crop to 224px"""
    with eager_mode():
        h, w = image.shape[0], image.shape[1]
        scale = tf.cond(tf.less(h, w), lambda: resize_side / h, lambda: resize_side / w)
        resized_image = tf.image.resize(tf.expand_dims(image, 0), [int(h * scale), int(w *
    * scale)])
        cropped_image = tf.image.resize_with_crop_or_pad(resized_image, output_height,
    output_width)

        if normalize:
            # Default normalization parameters for ImageNet
            cropped_image = (cropped_image - [123.675, 116.28, 103.53]) / [58.395, 57.12,
    57.375]

        return tf.squeeze(cropped_image)

# -----
# Post processing (what to do with the model's outputs)
# -----
def _get_imagenet_labels(json_path="../data/imagenet_names.json"):
    imagenet_names = json.load(open(json_path))
    imagenet_names = [imagenet_names[str(i)] for i in range(1001)]
    return imagenet_names[1:]

imagenet_labels = _get_imagenet_labels()

def postproc(results):
    labels = []
    scores = []
    results = [np.squeeze(result) for result in results]
    for result in results:
        top_ind = np.argmax(result)
        cur_label = imagenet_labels[top_ind]
        cur_score = 100 * result[top_ind]
```

(continues on next page)

(continued from previous page)

```

    labels.append(cur_label)
    scores.append(cur_score)
    return scores, labels

# -----
# Visualization
# -----
def mynorm(data):
    return (data - np.min(data)) / (np.max(data) - np.min(data))

def visualize_results(
    images,
    first_scores,
    first_labels,
    second_scores=None,
    second_labels=None,
    first_title="Full Precision",
    second_title="Other",
):
    # Deal with input arguments
    assert (second_scores is None and second_labels is None) or (
        second_scores is not None and second_labels is not None
    ), "second_scores and second_labels must both be supplied, or both not be supplied"
    assert len(images) == len(first_scores) == len(first_labels), "lengths of inputs
    ↪ must be equal"

    show_only_first = second_scores is None
    if not show_only_first:
        assert len(images) == len(second_scores) == len(second_labels), "lengths of
    ↪ inputs must be equal"

    # Display
    for img_idx in range(len(images)):
        plt.figure()
        plt.imshow(mynorm(images[img_idx]))

        if not show_only_first:
            plt.title(
                f"{first_title}: top-1 class is {first_labels[img_idx]}. Confidence is
    ↪ {first_scores[img_idx]:.2f}%, \n"
                f"{second_title}: top-1 class is {second_labels[img_idx]}. Confidence is
    ↪ {second_scores[img_idx]:.2f}%",
            )
        else:
            plt.title(
                f"{first_title}: top-1 class is {first_labels[img_idx]}. Confidence is
    ↪ {first_scores[img_idx]:.2f}%",
            )

```

Step 1: Test Native Model

Load the network to the ClientRunner from the saved Hailo Archive file:

```
[ ]: model_name = "resnet_v1_18"
hailo_model_har_name = f"{model_name}_hailo_model.har"
assert os.path.isfile(hailo_model_har_name), "Please provide valid path for HAR file"
runner = ClientRunner(har=hailo_model_har_name)
# By default it uses the hw_arch that is saved on the HAR. For overriding, use the hw_
↳arch flag.
```

```
[ ]: images_path = "../data"
images_list = [img_name for img_name in os.listdir(images_path) if os.path.
↳splitext(img_name)[1] == ".jpg"]

# Create an un-normalized dataset for visualization
image_dataset = np.zeros((len(images_list), 224, 224, 3))
# Create a normalized dataset to feed into the Native emulator
image_dataset_normalized = np.zeros((len(images_list), 224, 224, 3))
for idx, img_name in enumerate(sorted(images_list)):
    img = np.array(Image.open(os.path.join(images_path, img_name)))
    img_preproc = preproc(img)
    image_dataset[idx, :, :, :] = img_preproc.numpy()
    img_preproc_norm = preproc(img, normalize=True)
    image_dataset_normalized[idx, :, :, :] = img_preproc_norm.numpy()
```

Now call the Native emulator:

```
[ ]: # Notice that we use the normalized images, because normalization is not in the model
with runner.infer_context(InferenceContext.SDK_NATIVE) as ctx:
    native_res = runner.infer(ctx, image_dataset_normalized[:IMAGES_TO_VISUALIZE, :,
↳:, :])

native_scores, native_labels = postproc(native_res)
visualize_results(image_dataset[:IMAGES_TO_VISUALIZE, :, :, :], native_scores,
↳native_labels)
```

Steps 2,3: Apply Model Modifications, and Test Modified Model

The Model Script is a text file that includes model script commands, affecting the stages of the compiler.

In the next steps the following will be performed:

- Create a model script for the Optimization process, that also includes the model modifications.
- Load the model script (it wont be applied yet)
- Call runner.optimize_full_precision() to apply the model modifications (instead, we could call optimize() that also applies the model modifications)
- Then the SDK_FP_OPTIMIZED emulation context can be called

```
[ ]: model_script_lines = [
    # Add normalization layer with mean [123.675, 116.28, 103.53] and std [58.395, 57.12,
↳57.375])
    "normalization1 = normalization([123.675, 116.28, 103.53], [58.395, 57.12, 57.
↳375])\n",
    # For multiple input nodes:
    # {normalization_layer_name_1} = normalization([list of means per channel], [list
↳of stds per channel], {input_layer_name_1_from_hn})\n',
    # {normalization_layer_name_2} = normalization([list of means per channel], [list
↳of stds per channel], {input_layer_name_2_from_hn})\n',
```

(continues on next page)

(continued from previous page)

```
# ...
]

# Load the model script to ClientRunner so it will be considered on optimization
runner.load_model_script("".join(model_script_lines))
runner.optimize_full_precision()
```

```
[ ]: # Notice that we use the original images, because normalization is IN the model
with runner.infer_context(InferenceContext.SDK_FP_OPTIMIZED) as ctx:
    modified_res = runner.infer(ctx, image_dataset[:IMAGES_TO_VISUALIZE, :, :, :])

modified_scores, modified_labels = postproc(modified_res)

visualize_results(
    image_dataset[:IMAGES_TO_VISUALIZE, :, :, :],
    native_scores,
    native_labels,
    modified_scores,
    modified_labels,
    second_title="FP Modified",
)
```

Step 4.5: Optimize the Model and Test its Accuracy

1. Create a calibration dataset (will be the same as the input to the modified model)
2. Run Optimize
3. Test the optimized model accuracy vs. the modified model. Please note that the quantized emulator is not bit-exact with the Hailo hardware but provides good and fast approximation.

```
[ ]: # The original images are being used, just as the input to the SDK_FP_OPTIMIZED emulator
calib_dataset = image_dataset

# For calling Optimize, use the short version: runner.optimize(calib_dataset)
# A more general approach is being used here that works also with multiple input nodes.
# The calibration dataset may also be provided as a dictionary in the following format:
# {input_layer_name_1_from_hn: layer_1_calib_dataset, input_layer_name_2_from_hn:
#   ↳ layer_2_calib_dataset}
hn_layers = runner.get_hn_dict()["layers"]
print("Input layers are: ")
print([layer for layer in hn_layers if hn_layers[layer]["type"] == "input_layer"]) #
# ↳ See available input layer names
calib_dataset_dict = {"resnet_v1_18/input_layer1": calib_dataset} # In our case
# ↳ there is only one input layer
runner.optimize(calib_dataset_dict)
```

```
[ ]: # Notice that the original images are used, because normalization is present in the
# ↳ model
with runner.infer_context(InferenceContext.SDK_QUANTIZED) as ctx:
    quantized_res = runner.infer(ctx, image_dataset[:IMAGES_TO_VISUALIZE, :, :, :])

quantized_scores, quantized_labels = postproc(quantized_res)

visualize_results(
    image_dataset[:IMAGES_TO_VISUALIZE, :, :, :],
    modified_scores,
    modified_labels,
    quantized_scores,
```

(continues on next page)

(continued from previous page)

```
quantized_labels,
first_title="FP Modified",
second_title="Quantized",
)
```

```
[ ]: # Let's save the runner's state to a Quantized HAR
quantized_model_har_path = f"{model_name}_quantized_model.har"
runner.save_har(quantized_model_har_path)
```

Multiple Gpu Examples of inference

This Demo depends on multiple gpu availability Further information for utilizing multiple GPUs is available on the Dataflow Compiler user guide / Model optimization section

```
[ ]: num_gpus = len(tf.config.list_physical_devices("GPU"))
if num_gpus > 1:
    with runner.infer_context(InferenceContext.SDK_NATIVE, gpu_policy="model_
↳parallelization") as ctx:
        native_res = runner.infer(ctx, image_dataset_normalized)

    with runner.infer_context(InferenceContext.SDK_QUANTIZED, gpu_policy="data_
↳parallelization") as ctx:
        native_res = runner.infer(ctx, image_dataset_normalized)
else:
    print(f"To run this cell at least two gpus are needed, there are only {num_gpus}
↳available")
```

Step 6: How to Raise Accuracy

To increase the accuracy of the quantized model, optimize again using a model script to affect the optimization process.

There are several tools that can be used.

- Verify that there is a GPU with at least 1024 images in the calibration set
- Raise the optimization_level value using the model_optimization_flavor command. If it fails on high GPU memory, try lowering the batch_size as described on the last example
- Decrease the compression_level value using the model_optimization_flavor command (default is 0, lowest option)
- Set the output layer(s) to use 16-bit accuracy using the command quantization_param(output_layer_name, precision_mode=a16_w16). Note that the DFC will set 16-bit output automatically for small enough outputs.
- Use the Layer Noise Analysis tools to find layers with low SNR, and affect their quantization using weight or activation clipping (see the next tutorial)
- Experiment with the FineTune parameters (refer to the user guide for more details)

For more information refer the user guide in: Building Models / Model optimization / Model Optimization Workflow / Debugging accuracy.

This completes the in-depth optimization tutorial.

4.3.3. Advanced Model Modifications Tutorial

Adding on-chip input format conversion through model script commands

The next cell will apply model modification commands using a model script. A YUY2->YUV->RGB conversion will be added.

Unlike the normalization layer, which could simulate with the SDK_FP_OPTIMIZED and SDK_QUANTIZED emulators, not all format conversions are supported in the emulator (for more information see the Dataflow Compiler user guide / Model optimization section). Every conversion that runs in the emulator affects the calibration set, and the user should supply the set accordingly. For example, after adding YUV->RGB format conversion layer, the calibration set is expected to be in YUV format. However, for some conversions the user may choose to skip the conversion in emulation and to use the original calibration set instead. For instance, in this tutorial we will use YUY2->YUV layer without emulation because we want the emulator input and the calibration dataset to remain in YUV format. The format conversion layer would be relevant only when running the compiled .hef file on device.

Note: The NV21->YUV conversion is not supported in emulation.

The steps are:

- 1) Initialize Client Runner
- 2) Load YUV dataset
- 3) Load model script with the relevant commands
- 4) Using the optimize() API, the commands are applied and the model is quantized
- 5) Usage:
 - To create input conversion after a specific layer: `yuv_to_rgb_layer = input_conversion(input_layer1, yuv_to_rgb)`
 - To include the conversion in the optimization process: `yuv_to_rgb_layer = input_conversion(input_layer1, yuv_to_rgb, emulator_support=True)`
 - To create input conversion after all input layers: `net_scope1/yuv2rgb1, net_scope2/yuv2rgb2 = input_conversion(yuv_to_rgb)`

```
[ ]: # Let's load the original parsed model again
model_name = "resnet_v1_18"
hailo_model_har_name = f"{model_name}_hailo_model.har"
assert os.path.isfile(hailo_model_har_name), "Please provide valid path for HAR file"
runner = ClientRunner(har=hailo_model_har_name)

# We are using a pre-made YUV calibration set
calib_dataset_yuv = np.load("../model_modifications/calib_dataset_yuv.npz")

# Now we're adding yuy2_to_yuv conversion before the yuv_to_rgb and a normalization
# layer.
# The order of the layers is determined by the order of the commands in the model script:
# First we add normalization to the original input layer -> the input to the network is
# now normalization1
# Then we add yuv_to_rgb layer, so the order will be: yuv_to_rgb1->normalization1->
# original_network
# Lastly, we add yuy2_to_yuv layer, so the order will be: yuy2_to_yuv1->yuv_to_rgb1->
# normalization1->original_network
model_script_commands = [
    "normalization1 = normalization([123.675, 116.28, 103.53], [58.395, 57.12, 57.375])\n",
    "yuv_to_rgb1 = input_conversion(yuv_to_rgb)\n",
    "yuy2_to_yuv1 = input_conversion(input_layer1, yuy2_to_hailo_yuv)\n",
]
runner.load_model_script("".join(model_script_commands))

# Notice that we don't have to call runner.optimize_full_precision(), its only an
# intermediate step
```

(continues on next page)

(continued from previous page)

```
# to be able to use SdkFPOptimize emulator before Optimization.
runner.optimize(calib_dataset_yuv["yuv_dataset"])

modified_model_har_name = f"{model_name}_modified.har"
runner.save_har(modified_model_har_name)

!hailo visualizer {hailo_model_har_name} --no-browser
SVG("resnet_v1_18.svg")
```

Adding On-chip Input Resize Through Model Script Commands

This block will apply on-chip bilinear image resize at the beginning of the network through model script commands:

- Create a bigger (640x480) calibration set out of the Imagenet dataset
- Initialize Client Runner
- Load the new calibration set
- Load the model script with the resize command
- Using the optimize() API, the command is applied and the model is quantized

```
[ ]: images_path = "../data"
images_list = [img_name for img_name in os.listdir(images_path) if os.path.
↳splitext(img_name)[1] == ".jpg"]

idx_to_visualize = None
images_list = images_list[:64]
calib_dataset_new = np.zeros((len(images_list), 480, 640, 3))
for idx, img_name in enumerate(images_list):
    img = Image.open(os.path.join(images_path, img_name))
    resized_image = np.array(img.resize((640, 480), Image.Resampling.BILINEAR))
    calib_dataset_new[idx, :, :, :] = resized_image
    # find an image that will be nice to display
    if idx_to_visualize is None and img.size[0] != 640:
        idx_to_visualize = idx
        img_to_show = img

np.save("calib_set_480_640.npy", calib_dataset_new)
plt.imshow(img_to_show)
plt.title("Original image")
plt.show()
plt.imshow(np.array(calib_dataset_new[idx_to_visualize, :, :, :], np.uint8))
plt.title("Resized image")
plt.show()
```

```
[ ]: model_name = "resnet_v1_18"
hailo_model_har_name = f"{model_name}_hailo_model.har"
assert os.path.isfile(hailo_model_har_name), "Please provide valid path for HAR file"
runner = ClientRunner(har=hailo_model_har_name)

calib_dataset_large = np.load("calib_set_480_640.npy")

# Add a bilinear resize from 480x640 to the network's input size - in this case, 224x224.
# The order of the layers is determined by the order of the commands in the model script:
# First we add normalization to the original input layer -> the input to the network is
↳now normalization1
# Then we add resize layer, so the order will be: resize_input1->normalization1->
↳original_network
```

(continues on next page)

(continued from previous page)

```
model_script_commands = [
    "normalization1 = normalization([123.675, 116.28, 103.53], [58.395, 57.12, 57.
↪375])\n",
    "resize_input1= resize(resize_shapes=[480,640])\n",
]

runner.load_model_script("".join(model_script_commands))
calib_dataset_dict = {"resnet_v1_18/input_layer1": calib_dataset_large} # In our
↪case there is only one input layer
runner.optimize(calib_dataset_dict)

modified_model_har_name = f"{model_name}_resized.har"
runner.save_har(modified_model_har_name)

!hailo visualizer {hailo_model_har_name} --no-browser
SVG("resnet_v1_18.svg")
```

Adding Non-Maximum Suppression (NMS) Layer Through Model Script Commands

This block will add an NMS layer at the end of the network through the model script command: `nms_postprocess`. The following arguments can be used to:

- Configjson: an external json file that allows the changing of the NMS parameters (can be skipped for the default configuration).
- Meta architecture: which meta architecture to use (for example, yolov5, ssd, etc). In this example, yolov5 will be used.
- Engine: defines the inference device for running the nms: `nn_core`, `cpu` or `auto` (this example shows `cpu`).

Usage:

- Initialize Client Runner
- Translate a YOLOv5 model
- Load the model script with the NMS command
- Use the `optimize_full_precision()` API to apply the command (Note that `optimize()` API can also be used)
- Display inference result

```
[ ]: model_name = "yolov5s"
onnx_path = f"../models/{model_name}.onnx"
assert os.path.isfile(onnx_path), "Please provide valid path for ONNX file"

# Initialize a new client runner
runner = ClientRunner(hw_arch="hailo8")
# Any other hw_arch can be used as well.

# Translate YOLO model from ONNX
runner.translate_onnx_model(onnx_path, end_node_names=["Conv_298", "Conv_248",
↪"Conv_198"])
# Note: NMS will be detected automatically, with a message that contains:
# - 'original layer name': {'w': [WIDTHS], 'h': [HEIGHTS], 'stride': STRIDE,
↪'encoded_layer': TRANSLATED_LAYER_NAME}
# Use nms_postprocess(meta_arch=yolov5) to add the NMS.

# Add model script with NMS layer at the network's output.
model_script_commands = [
```

(continues on next page)

(continued from previous page)

```

"normalization1 = normalization([0.0, 0.0, 0.0], [255.0, 255.0, 255.0])\n",
"resize_input1= resize(resize_shapes=[480,640])\n",
"nms_postprocess(meta_arch=yolov5, engine=cpu, nms_scores_th=0.2, nms_iou_th=0.
↪4)\n",
]
# Note: Scores threshold of 0.0 means no filtering, 1.0 means maximal filtering. IoU
↪thresholds are opposite: 1.0 means filtering boxes only if they are equal, and 0.0
↪means filtering with minimal overlap.
runner.load_model_script("".join(model_script_commands))

# Apply model script changes
runner.optimize_full_precision()

# Infer an image with the Hailo Emulator
with runner.infer_context(InferenceContext.SDK_FP_OPTIMIZED) as ctx:
    nms_output = runner.infer(ctx, calib_dataset_new[:16, ...])
HEIGHT = 480
WIDTH = 640
# For each image
for i in range(16):
    found_any = False
    min_score = None
    max_score = None
    # Go over all classes
    for class_index in range(nms_output.shape[1]):
        score, box = nms_output[i][class_index, 4, :], nms_output[i][class_index, 0:4, :]
        # Go over all detections
        for detection_idx in range(box.shape[1]):
            cur_score = score[detection_idx]
            # Discard null detections (because the output tensor is always padded to MAX_
↪DETECTIONS on the emulator interface.
            # Note: On HailoRT APIs (that are used on the Inference Tutorial, and with C++
↪APIs), the default is a list per class. For more information look for NMS on the
↪HailoRT user guide.
            if cur_score == 0:
                continue

        # Plotting code
        if not found_any:
            found_any = True
            fig, ax = plt.subplots()
            ax.imshow(Image.fromarray(np.array(calib_dataset_new[i], np.uint8)))
            if min_score is None or cur_score < min_score:
                min_score = cur_score
            if max_score is None or cur_score > max_score:
                max_score = cur_score
            (
                y_min,
                x_min,
            ) = box[0, detection_idx] * HEIGHT, box[1, detection_idx] * WIDTH
            y_max, x_max = box[2, detection_idx] * HEIGHT, box[3, detection_idx] * WIDTH
            center, width, height = (x_min, y_min), x_max - x_min, y_max - y_min
            # draw the box on the input image
            rect = patches.Rectangle(center, width, height, linewidth=1, edgecolor="r",
↪facecolor="none")
            ax.add_patch(rect)

        if found_any:
            plt.title(f"Plot of high score boxes. Scores between {min_score:.2f} and {max_
↪score:.2f}")
            plt.show()

```


4.3.4. Advanced Optimization - Compression and Optimization Levels

For aggressive quantization (compress significant amount of weights to 4-bits), a higher optimization level will be needed to obtain good result.

For quick iterations it is always recommended to start with the default setting of the model optimizer (optimization_level=2, compression_level=1). However, when moving to production, it is recommended to work at the highest complexity level to achieve optimal accuracy. With regards to compression, users should increase it when the overall throughput/latency of the model is not good enough.

Note that increasing compression would have negligible effect on power-consumption so the motivation to work with higher compression level is mainly due to FPS considerations.

Here the compression level is set to 4 (which means ~80% of the weights will be quantized into 4-bits) using the compression_level param in a model script and run the model optimization again. Using 4-bit weights might reduce the model's accuracy but will help to reduce the model's memory footprint. In this example, it can be seen that the reliability of some examples decreases after changing several layers to 4-bit weights, later the reliability will improve after applying higher optimization_level.

```
[ ]: alls_lines = [
    "normalization1 = normalization([123.675, 116.28, 103.53], [58.395, 57.12, 57.
    ↪375])\n",
    # Batch size is 8 by default; 2 was used for stability on PCs with low amount of RAM /
    ↪VRAM
    "model_optimization_flavor(optimization_level=0, compression_level=4, batch_
    ↪size=2)\n",
    # The following line is needed because resnet_v1_18 is a really small model, and the
    ↪compression_level is always reverted back to 0.'
    # To force using compression_level with small models, the following line should be
    ↪used (compression level=4 equals to 80% 4-bit):
    "model_optimization_config(compression_params, auto_4bit_weights_ratio=0.8)\n",
    # The application of the compression could be seen by the [info] messages: "Assigning
    ↪4bit weight to layer .."
]
# -- Reduces weights memory by 80% !

runner = ClientRunner(har=hailo_model_har_name)

runner.load_model_script("".join(alls_lines))
runner.optimize(calib_dataset)
```

```
[ ]: images = calib_dataset[:IMAGES_TO_VISUALIZE, :, :, :]
with runner.infer_context(InferenceContext.SDK_FP_OPTIMIZED) as ctx:
    modified_res = runner.infer(ctx, images)
with runner.infer_context(InferenceContext.SDK_QUANTIZED) as ctx:
    quantized_res = runner.infer(ctx, images)

modified_scores, modified_labels = postproc(modified_res)
quantized_scores, quantized_labels = postproc(quantized_res)

visualize_results(
    image_dataset[:IMAGES_TO_VISUALIZE, :, :, :],
    modified_scores,
    modified_labels,
    quantized_scores,
    quantized_labels,
    first_title="FP Modified",
    second_title="Quantized",
)
```

Now, repeating the same process with higher optimization level (For full information see the Dataflow Compiler user guide / Model optimization section):

```
[ ]: images = calib_dataset[:IMAGES_TO_VISUALIZE, :, :, :]

alls_lines = [
    "normalization1 = normalization([123.675, 116.28, 103.53], [58.395, 57.12, 57.
    ↪375])\n",
    # Batch size is 8 by default; 2 was used for stability on PCs with low amount of RAM /
    ↪VRAM
    "model_optimization_flavor(optimization_level=2, compression_level=4, batch_
    ↪size=2)\n",
    # The following line is needed because resnet_v1_18 is a really small model, and the
    ↪compression_level is always reverted back to 0.'
    # To force using compression_level with small models, the following line should be
    ↪used (compression level=4 equals to 80% 4-bit):
    "model_optimization_config(compression_params, auto_4bit_weights_ratio=0.8)\n",
    # The application of the compression could be seen by the [info] messages: "Assigning
    ↪4bit weight to layer .."
]
# -- Reduces weights memory by 80% !

runner = ClientRunner(har=hailo_model_har_name)
runner.load_model_script("".join(alls_lines))
runner.optimize(calib_dataset)

with runner.infer_context(InferenceContext.SDK_FP_OPTIMIZED) as ctx:
    modified_res = runner.infer(ctx, images)
with runner.infer_context(InferenceContext.SDK_QUANTIZED) as ctx:
    quantized_res = runner.infer(ctx, images)

modified_scores, modified_labels = postproc(modified_res)
quantized_scores_new, quantized_labels_new = postproc(quantized_res)

visualize_results(
    image_dataset[:IMAGES_TO_VISUALIZE, :, :, :],
    modified_scores,
    modified_labels,
    quantized_scores_new,
    quantized_labels_new,
    first_title="FP Modified",
    second_title="Quantized",
)
```

```
[ ]: print(
    f"Full precision predictions: {modified_labels}\n"
    f"Quantized predictions (with optimization_level=2): {quantized_labels_new} "
    f"({sum(np.array(modified_labels) == np.array(quantized_labels_new))} /
    ↪{len(modified_labels)})\n"
    f"Quantized predictions (with optimization_level=0): {quantized_labels} "
    f"({sum(np.array(modified_labels) == np.array(quantized_labels))} / {len(modified_
    ↪labels)})",
)
```

Finally, save the optimized model to a Hailo Archive file:

```
[ ]: runner.save_har(quantized_model_har_path)
```

4.4. Compilation Tutorial

4.4.1. Hailo Compilation Example from Hailo Archive Quantized Model to HEF

This tutorial will describe how to convert the model into the HEF executable format

Requirements:

- Run the codes below in Jupyter notebook, see the Introduction tutorial for more details.
- A quantized HAR file.

Note: This section demonstrates the Python APIs for Hailo Compiler. You could also use the CLI: try `hailo compiler --help`. More details on Dataflow Compiler User Guide / Building Models / Profiler and other command line tools.

```
[ ]: from hailo_sdk_client import ClientRunner
```

Choose the quantized model Hailo Archive file to use throughout the example:

```
[ ]: model_name = "resnet_v1_18"
quantized_model_har_path = f"{model_name}_quantized_model.har"
```

Load the network to the ClientRunner:

```
[ ]: runner = ClientRunner(har=quantized_model_har_path)
# By default it uses the hw_arch that is saved on the HAR. It is not recommended to
# change the hw_arch after Optimization.
```

Run compilation (This method can take a couple of minutes):

Note: The `hailo compiler` CLI tool can also be used.

```
[ ]: hef = runner.compile()

file_name = f"{model_name}.hef"
with open(file_name, "wb") as f:
    f.write(hef)
```

4.4.2. Profiler tool

Run the profiler tool:

This command will pop-open the HTML report in the browser.

```
[ ]: har_path = f"{model_name}_compiled_model.har"
runner.save_har(har_path)
!hailo profiler {har_path}
```

Note:

The HTML profiler report could be augmented with runtime statistics, that are saved after the `.hef` ran on the device using `hailortcli`.

For more information look under the section: Dataflow Compiler User Guide / Building Models / Profiler and other command line tools / Running the Profiler.

4.5. Inference Tutorial

This tutorial describes the inference process.

Requirements:

- HailoRT installed on the same virtual environment, or as part of the Hailo SW Suite.
- Run this code in Jupyter notebook, see the Introduction tutorial for more details.
- Run the [Compilation Tutorial](#) before running this one.

Note: This section demonstrates PyHailoRT, which is a python library for communication with Hailo devices. For evaluation purposes, refer to `hailortcli run2 --help` (or the alias `hailo run2 --help`). For more details on the HailoRT User Guide / Command Line Tools.

```
[ ]: # General imports used throughout the tutorial
from multiprocessing import Process

import numpy as np

from hailo_platform import (
    HEF,
    ConfigureParams,
    FormatType,
    HailoSchedulingAlgorithm,
    HailoStreamInterface,
    InferVStreams,
    InputVStreamParams,
    InputVStreams,
    OutputVStreamParams,
    OutputVStreams,
    VDevice,
)

from hailo_sdk_client import ClientRunner, InferenceContext
```

4.5.1. Standalone Hardware Deployment

The standalone flow allows direct access to the HW, developing applications directly on top of Hailo core HW, using HailoRT.

This way the Hailo hardware can be used without Tensorflow, and even without the Hailo Dataflow Compiler (after the HEF is built).

A HEF is Hailo's binary format for neural networks. The HEF file contains:

- Low level representation of the model
- Target HW configuration
- Weights
- Metadata for HailoRT (e.g. input/output scaling)

First create the desired target object.

```
[ ]: # Setting VDevice params to disable the HailoRT service feature
params = VDevice.create_params()
params.scheduling_algorithm = HailoSchedulingAlgorithm.NONE

# The target can be used as a context manager ("with" statement) to ensure it's released
# on time.
# Here it's avoided for the sake of simplicity
```

(continues on next page)

(continued from previous page)

```
target = VDevice(params=params)

# Loading compiled HEFs to device:
model_name = "resnet_v1_18"
hef_path = f"{model_name}.hef"
hef = HEF(hef_path)

# Get the "network groups" (connectivity groups, aka. "different networks")
# ↳ information from the .hef
configure_params = ConfigureParams.create_from_hef(hef=hef,
# ↳ interface=HailoStreamInterface.PCIe)
network_groups = target.configure(hef, configure_params)
network_group = network_groups[0]
network_group_params = network_group.create_params()

# Create input and output virtual streams params
# Quantized argument signifies whether or not the incoming data is already quantized.
# Data is quantized by HailoRT if and only if quantized == False .
input_vstreams_params = InputVStreamParams.make(network_group, quantized=False,
# ↳ format_type=FormatType.FLOAT32)
output_vstreams_params = OutputVStreamParams.make(network_group, quantized=True,
# ↳ format_type=FormatType.UINT8)

# Define dataset params
input_vstream_info = hef.get_input_vstream_infos()[0]
output_vstream_info = hef.get_output_vstream_infos()[0]
image_height, image_width, channels = input_vstream_info.shape
num_of_images = 10
low, high = 2, 20

# Generate random dataset
dataset = np.random.randint(low, high, (num_of_images, image_height, image_width,
# ↳ channels)).astype(np.float32)
```

Running Hardware Inference

Infer the model and then display the output shape:

```
[ ]: input_data = {input_vstream_info.name: dataset}

with InferVStreams(network_group, input_vstreams_params, output_vstreams_params)
# ↳ as infer_pipeline:
    with network_group.activate(network_group_params):
        infer_results = infer_pipeline.infer(input_data)
        # The result output tensor is infer_results[output_vstream_info.name]
        print(f"Stream output shape is {infer_results[output_vstream_info.name].shape}
# ↳ ")
```

4.5.2. Streaming Inference

This section shows how to run streaming inference using multiple processes in Python.

Infer will not be used and instead a send and receive model will be employed. The send function and the receive function will run in different processes.

Define the send and receive functions:

```
[ ]: def send(configured_network, num_frames):
    vstreams_params = InputVStreamParams.make(configured_network)
    with InputVStreams(configured_network, vstreams_params) as vstreams:
        configured_network.wait_for_activation(1000)
        vstream_to_buffer = {
            vstream: np.ndarray([1] + list(vstream.shape), dtype=vstream.dtype) for
            vstream in vstreams
        }
        for _ in range(num_frames):
            for vstream, buff in vstream_to_buffer.items():
                vstream.send(buff)

def recv(configured_network, num_frames):
    vstreams_params = OutputVStreamParams.make(configured_network)
    configured_network.wait_for_activation(1000)
    with OutputVStreams(configured_network, vstreams_params) as vstreams:
        for _ in range(num_frames):
            for vstream in vstreams:
                _data = vstream.recv()
```

Define the amount of images to stream and processes, then recreate the target and run the processes:

```
[ ]: # Define the amount of frames to stream
num_of_frames = 1000

# Start the streaming inference
send_process = Process(target=send, args=(network_group, num_of_frames))
recv_process = Process(target=recv, args=(network_group, num_of_frames))
recv_process.start()
send_process.start()
print(f"Starting streaming (hef='{model_name}', num_of_frames={num_of_frames})")
with network_group.activate(network_group_params):
    send_process.join()
    recv_process.join()

# Clean pcie target
target.release()
print("Done")
```

4.5.3. DFC Inference in Tensorflow Environment

Note: This section is not yet supported on the Hailo-15, as it requires the Dataflow Compiler to be installed on the device.

The `runner.infer()` method that was used for emulation in the model optimization tutorial can also be used for running inference on the Hailo device inside the `infer_context` environment. Before calling this function with hardware context, please make sure a HEF file is loaded to a runner, by one of the options: calling `runner.compile()`, loading a compiled HAR using `runner.load_har()`, or setting the HEF attribute `runner.hef`.

First, create the runner and load a compiled HAR:

```
[ ]: model_name = "resnet_v1_18"
compiled_model_har_path = f"{model_name}_compiled_model.har"
runner = ClientRunner(hw_arch="hailo8", har=compiled_model_har_path)
# For Mini PCIe modules or Hailo-8R devices, use hw_arch='hailo8r'
```

Calling `runner.infer()` within inference HW context to run on the Hailo device (`InferenceContext.SDK_HAILO_HW`):

```
[ ]: hef_path = f"{model_name}.hef"
hef = HEF(hef_path)
input_vstream_info = hef.get_input_vstream_infos()[0]
image_height, image_width, channels = input_vstream_info.shape
num_of_images = 10
low, high = 2, 20

with runner.infer_context(InferenceContext.SDK_HAILO_HW) as hw_ctx:
    # Running hardware inference:
    for i in range(10):
        dataset = np.random.randint(low, high, (num_of_images, image_height, image_
        width, channels)).astype(np.uint8)
        results = runner.infer(hw_ctx, dataset)
```

4.5.4. Profiler with Runtime Data

This will demonstrate the usage of the HTML profiler with runtime data:

Note: On the Hailo-15 device:

1. The `hailortcli run2` command should be run on the device itself
2. The created json file should be copied to the Dataflow Compiler environment
3. The `hailo profiler` command should be used

```
[ ]: model_name = "resnet_v1_18"
hef_path = f"{model_name}.hef"
compiled_har_path = f"{model_name}_compiled_model.har"
runtime_data_path = f"runtime_data_{model_name}.json"

# Run hailortcli (can use `hailo` instead) to run the .hef on the device, and save
runtime statistics to runtime_data.json
!hailortcli run2 -m raw measure-fw-actions --output-path {runtime_data_path} set-
net {hef_path}
!hailo profiler {compiled_har_path} --runtime-data {runtime_data_path} --out-path
runtime_profiler.html

# Instead, this command could be used: hailo profiler {compiled_har_path} --collect-
runtime-data --out-path runtime_profiler.html
```

Notes on the Profiler with runtime data

resnet_v1_18 is a small network, which fits in a single device without context-switch (it is called “single context”). Its FPS and Latency are always displayed.

The `--runtime-data` flag is useful with big models, where the FPS and latency cannot be calculated on compile time. With runtime data, the profiler displays the load, config and runtime of the contexts, the fps and latency for multiple batch sizes.

The runtime FPS is also displayed on the hailortcli output.

4.6. Accuracy Analysis Tool Tutorial

This is an advanced tutorial, if the accuracy results obtained were satisfactory it can be omitted. Before using it, make sure that your native (pre-quantization) results are satisfying. For more details refer to [Debugging Accuracy](#) section on the Dataflow Compiler User Guide.

This tutorial will serve as a guide for how model quantization analysis breaks down the quantization noise per layer. The tutorial is intended to guide the user in using Hailo analyze noise tool, by using it to analyze the classification model MobileNet-v3-Large-Minimalistic.

The flow is mainly comprised of:

- Paths definitions: Defining the paths to the model and data for analysis.
- Preparing the model: Initial Parse and Optimize of the model.
- Accuracy analysis: This step is the heart of the tool, and computes the quantization noise of each layer output. For each layer, the layer under analysis is the **only** quantized layer, while the rest of the model is kept in full precision. This highlights the quantization sensitivity of the model to the noise of that specific layer.
- Visualizing the results: Walk through the results of the accuracy analysis and explain the different graphs and information.
- Re-optimizing the model: After debugging the noise we repeat the optimization process to improve the results.

Requirements:

- Run this code in Jupyter notebook, see the Introduction tutorial for more details.
- Verify that you've completed the Parsing tutorial and the Model Optimization tutorial or generated analysis data in another way.

```
[ ]: import os

import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf

from hailo_sdk_client import ClientRunner

%matplotlib inline
```


4.6.1. Input Definitions

- Model path: path to the model to be used in this tutorial
- data_path: path to preprocessed .npy image files for optimization and analysis

```
[ ]: model_name = "v3-large-minimalistic_224_1.0_float"
model_path = "../models/" + model_name + ".tflite"
assert os.path.isfile(model_path), "Please provide valid path for the model"

data_path = "../calib_set.npy"
assert os.path.isfile(data_path), "Please provide valid path for a dataset"
har_path = model_name + ".har"
```

It is highly recommended to use GPU when running the analysis tool.

If there isn't one in the machine, the code will be executed on the CPU and it will take a longer time to run.

```
[ ]: if len(tf.config.list_physical_devices("GPU")) == 0:
    print("Warning: you are running the accuracy analysis tool without a GPU, expect
    ↳ long running time.")
```

4.6.2. Preparing the Model

In this step, the model will be parsed and optimized to prepare it for analysis. For more details checkout the Parsing tutorial and the Model Optimization tutorial.

```
[ ]: runner = ClientRunner(hw_arch="hailo8")
runner.translate_tf_model(model_path, model_name)

model_script = "normalization1 = normalization([127.5, 127.5, 127.5], [127.5, 127.5,
↳ 127.5])\n"
runner.load_model_script(model_script)

runner.optimize(data_path)
```

4.6.3. Accuracy Analysis

Though most models work well with our default optimization, some suffer from high quantization noise that induces substantial accuracy degradation. As an example, we choose the MobileNet-v3-Large-Minimalistic neural network model that, due to its structural characteristics, results in a high degradation of 6% for Top-1 accuracy on the ImageNet-1K validation dataset.

To analyze the source of degradation, the Hailo `analyze_noise` API will be used. The analysis tool uses a given dataset to measure the noise level in each layer and allows to pinpoint problematic layers that should be handled. The analysis tool uses the entire dataset by default, use the `data_count` argument to limit the number of images. It is recommended to use at least 64 images, preferably not from the same calibration set, however, to keep the tool's processing time to a reasonable level, it is also recommended not to use more than 100-200 images.

The following is equivalent to running the CLI command:

```
hailo analyze-noise quantized_model_har_path --data-path data_path --batch-size 2 --data-count 16
```

The output is saved inside the HAR, to be visualized later on by the Profiler.

```
[ ]: runner.analyze_noise(data_path, batch_size=2, data_count=16) # Batch size is 1 by default
runner.save_har(har_path)
```

4.6.4. Visualizing the Results

In this section, a general explanation for the noise analysis report will be provided.

To visualize the accuracy analysis results and debug the quantization noise, the Hailo Model Profiler will be used.

The Hailo Model Profiler will generate an HTML report with all the information for the model.

In the Optimization Details tab of the report, all the relevant information for this tutorial can be found:

```
[ ]: !hailo profiler {har_path}
# Note: When working on a remote computer, manual opening of the HTML file may be required
```

SNR Chart

Displayed on the top ribbon, only if the profiled HAR contains the analyze-noise data.

This chart shows the sensitivity of each layer to quantization (measured separately for each output layer). To measure the quantization noise of each layer's output, iterate over all layers when the given layer is the **only** quantized layer, while the rest are kept in full precision and measure the SNR at each output layer. The number of SNR values will be the number of outputs layer affected by the quantized layer. The graph shows the SNR values in decibels (dB) and any value higher than 10 should be fine (higher is better).

In case an output layer is sensitive (low SNR) across many layers it is recommended to re-quantize with one of the following model script commands (not in the scope of this tutorial):

- Configure the output layer to 16-bit output. For example, using the model script command: `quantization_param(output_layer1, precision_mode=a16_w16)`.
- When possible, offload output activation to the accelerator. For example, the following command adds sigmoid activation to the output layer conv51: `change_output_activation(conv51, sigmoid)` and should be used to offload sigmoid from post-processing code to the accelerator.
- Use massive fine tune which is enabled by default in `optimization_level=2` but can be customized. For example, specific fine-tune command: `post_quantization_optimization(finetime, policy=enabled, learning_rate=0.0001, epochs=8, batch_size=4, dataset_size=4000)`. Other useful attributes to this command are: `loss_layer_names`, `loss_factors` and `loss_types` which allows the user to manually edit the loss function of the fine tune training. In a case where the fine tune failed due to GPU memory, try to use a lower batch_size.
- Increase the optimization level. For example, `model_optimization_flavor(optimization_level=4)` will set the highest optimization level (default is 2).
- Decrease the compression level. For example, `model_optimization_flavor(compression_level=0)` will disable compression (default value is 1).

Layers Information

Displayed on the right when a layer is selected.

This section provide per-layer detailed information that will help debug the local quantization errors in the model, for example, specific layer that is very sensitive for quantization. Note that quantization noise may stem from the layers' weights, activations or both.

- **Weight Histogram:** this graph shows the weights distribution and can help to identify outliers. If outliers exist in the weight distribution, the following command can be used to clip it, for example, clip the kernel values of conv27: `pre_quantization_optimization(weights_clipping, layers=[conv27], mode=percentile, clipping_values=[0.01, 99.99])`
- **Activation Histogram:** this graph shows the activation distribution as collected by the layer noise analysis tool. Wide activation distribution is a major source of degradation source and in general it is strongly recommend to use a model with batch normalization after each layer to limit the layer's extreme activation values. Another important argument that affects the activation distribution is the calibration size that was used during quantization, to raise it, use the following command: `model_optimization_config(calibration, calibset_size=512)`, the default value for calibration is 64. In case of outliers in the layers' activation distribution, we recommend using the activation clipping command, for example: `pre_quantization_optimization(activation_clipping, layers={'*'}, mode=percentile, clipping_values=[0.01, 99.99])`
- **Scatter Plot:** this graph shows a comparison between full precision and quantized values of the layers' activation. The X-axis of each point in this graph is its value in full precision and Y-axis is the value after quantization. Zero quantization noise means the slope would be exactly one. In case of bias noise you expect to find many points above/below the line that represent imperfect quantization, if this is the case, you should use the following commands: `post_quantization_optimization(bias_correction, policy=enabled)` and `post_quantization_optimization(finetune, policy=disabled)`

To examine these results, first plot the SNR graph for this specific model. Note that in general the profiler report should be used but here an alternative visualization will be used.

```
[ ]: def get_snr_results():
    # SNR results are saved in the params statistics object
    params_statistics = runner.get_params_statistics()
    out_layer = "v3-large-minimalistic_224_1_0_float/output_layer1"
    layers = []
    snr = []
    for layer in runner.get_hn_model():
        # We get the SNR for each analyzed layer for a specific output layer (there is only
        # one in this case)
        layer_snr = params_statistics.get(f"{layer.name}/layer_noise_analysis/noise_
        results/{out_layer}")
        if layer_snr is not None:
            layers.append(layer.name_without_scope)
            snr.append(layer_snr[0].tolist())
    return layers, snr

def get_worst_snr_layers(layers, snr):
    worst_snr_layers = [(layers[i], snr[i]) for i in np.argpartition(snr, 3)[:3]]
    print(f"Worst SNR is obtained in the following layers:\n{worst_snr_layers}")
    return worst_snr_layers

def plot_snr_graph(layers, snr):
    fig, ax = plt.subplots(figsize=(12, 3))
    plt.plot(layers, snr)
    plt.title(f"Per-Layer Logits SNR ({model_name}), higher is better.")
    plt.xlabel("Layer")
```

(continues on next page)

(continued from previous page)

```
plt.xticks(rotation=75, fontsize="x-small")
plt.ylabel("SNR")
plt.grid()
plt.show()
```

```
layers, snr = get_snr_results()
get_worst_snr_layers(layers, snr)
plot_snr_graph(layers, snr)
```

4.6.5. Re-Optimizing the Model

Next, we will try to improve the model accuracy results by using specific model script commands. Specifically, we will use the `activation_clipping` command on the problematic layers to clip outliers from the output of the layers and `optimization_level=2`. For further information we refer the user to the full Accuracy report in the profiler HTML.

```
[ ]: runner = ClientRunner(hw_arch="hailo8")
runner.translate_tf_model(model_path, model_name)

model_script_commands = [
    "normalization1 = normalization([127.5, 127.5, 127.5], [127.5, 127.5, 127.5])\n",
    "model_optimization_config(calibration, calibset_size=128)\n",
    "pre_quantization_optimization(activation_clipping, layers=[dw1, conv2, conv3],\n",
    ↪mode=percentile, clipping_values=[0.5, 99.5])\n",
    "pre_quantization_optimization(weights_clipping, layers=[dw1], mode=percentile,\n",
    ↪clipping_values=[0.0, 99.99])\n",
    "model_optimization_flavor(optimization_level=2, compression_level=0)\n",
]
runner.load_model_script("".join(model_script_commands))

runner.optimize(data_path)

runner.analyze_noise(data_path, batch_size=2, data_count=16) # Batch size is 1 by default
runner.save_har(har_path)

!hailo profiler {har_path}
# Note: When working on a remote computer, manual opening of the HTML file may be required
```

After fixing the optimization process, it should be possible to reduce the model degradation to 1% (Top-1 accuracy on the ImageNet-1K validation dataset) which is usually the target goal for classification models.

The improvement can also be seen from the new SNR graph:

```
[ ]: layers, snr = get_snr_results()
get_worst_snr_layers(layers, snr)
plot_snr_graph(layers, snr)
```

4.7. Quantization Aware Training Tutorial

This tutorial is intended for advanced users, If the previous accuracy results were satisfactory, it can be omitted..

This section will describe the steps for performing Quantization Aware Training (QAT) using Hailo's quantized model. It is assumed that the User already has a background in training deep neural networks.

Quantization aware training - refers to a set of algorithms that incorporate full network training in a quantized domain. The technique utilizes the straight-through estimator (STE) concept to allow for backpropagation through non-differentiable operations, such as rounding and clipping, during the training process. In deep learning literature, QAT typically refers to an extended training procedure using the full dataset, labels, and multiple GPUs, similar to the original training process. However, it can also be applied in other scenarios.

The main differences between the quantization-aware training method and the optimization method shown in previous tutorials are:

- QAT enables training using labeled data, whereas the FineTune algorithm ([Model Optimization Tutorial](#)) is limited to training using knowledge distillation from the full precision model.
- QAT supports running on multiple GPUs for faster training.
- QAT allows for the use of a pipeline of networks or the integration of post-processing functions into the training procedure.

In summary, QAT is a useful tool for training quantized models with labeled data and supports multi-GPU training and integration of post-processing functions. Currently, Hailo QAT only supports Keras.

The remainder of this tutorial will cover the following steps:

- Input definitions: In this step, we will prepare the dataset and model for training and testing.
- Full precision training: A short training procedure will be run to initialize the model's weights.
 - In real scenarios, a complete full precision training procedure should take place here. In this notebook, the full precision training has been shortened to simplify the tutorial.
- Translation of the model: The model will be exported to TFLite, parsed, optimized, and evaluated using the Hailo toolchain.
- Running QAT: Finally, quantization-aware training will be performed on the quantized model to optimize its accuracy.

Requirements:

- Run this code in Jupyter notebook, see the Introduction tutorial for more details.

```
[ ]: import keras
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf

from hailo_sdk_client import ClientRunner, InferenceContext
```

4.7.1. Input Definitions

The input definitions step of this tutorial involves using the [MNIST dataset](#) and a simple Convolutional Neural Network (CNN). The code provided will download the dataset and prepare it for training and evaluation.

```
[ ]: # Model parameters
num_classes = 10
input_shape = (28, 28, 1)

# Load the data and split it between train and test sets
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
```

(continues on next page)

(continued from previous page)

```
# Prepare the dataset
x_train = x_train.astype(np.float32) / 255
x_test = x_test.astype(np.float32) / 255
x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)
y_train = keras.utils.to_categorical(y_train, num_classes)
y_test = keras.utils.to_categorical(y_test, num_classes)
print(f"Total number of training samples: {x_train.shape[0]}")
print(f"Total number of testing samples: {x_test.shape[0]}")
```

```
[ ]: # Define the model
model = keras.Sequential(
    [
        keras.Input(shape=input_shape),
        keras.layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
        keras.layers.MaxPooling2D(pool_size=(2, 2)),
        keras.layers.Conv2D(64, kernel_size=(3, 3), activation="relu"),
        keras.layers.MaxPooling2D(pool_size=(2, 2)),
        keras.layers.Flatten(),
        keras.layers.Dropout(0.5),
        keras.layers.Dense(num_classes, activation="softmax"),
    ],
)
model.summary()
```

4.7.2. Full Precision Training

In this step, a short training procedure will be run to initialize the model's weights. Only 5,000 images from the full training dataset will be used. The accuracy of the model will be measured on the test dataset.

```
[ ]: # Run short training (using only 5k images)
model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=[
    ↪ "accuracy"])
model.fit(x_train[:5000], y_train[:5000], batch_size=128, epochs=1)

# Evaluate the results
score = model.evaluate(x_train, y_train)
print(f"Train accuracy: {100 * score[1]:.3f} (Top-1)")
score = model.evaluate(x_test, y_test)
print(f"Test accuracy: {100 * score[1]:.3f} (Top-1)")
```

4.7.3. Translation of the Model

In this step, a trained model will be exported into TFLite format to prepare it for use in the Hailo toolchain. After being translated into TFLite, the model can be parsed, optimized, and inferred using the Hailo DFC. The results of the full precision model will be compared to those of the quantized model. It is important to note that the results of the full precision model should be identical to those obtained from the Keras evaluation, while the quantized model may experience some degradation due to quantization noise.

```
[ ]: # Export the model to TFLite
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()
tflite_model_path = "model.tflite"
with tf.io.gfile.GFile(tflite_model_path, "wb") as f:
    f.write(tflite_model)
```

```
[ ]: # Parse the TFlite model
runner = ClientRunner(hw_arch="hailo8")
runner.translate_tf_model(tflite_model_path)

# Optimize the model: enforce 60% 4-bit weights without optimization
model_script_commands = [
    "model_optimization_config(compression_params, auto_4bit_weights_ratio=0.6)\n"
    "model_optimization_flavor(optimization_level=0)\n",
]

runner.load_model_script("".join(model_script_commands))
runner.optimize(x_train[:1024])

[ ]: # Evaluate the results
with runner.infer_context(InferenceContext.SDK_QUANTIZED) as q_ctx:
    with runner.infer_context(InferenceContext.SDK_FP_OPTIMIZED) as fp_ctx:
        y_infer_fp = runner.infer(fp_ctx, x_test)
        y_infer_q = runner.infer(q_ctx, x_test)

# Hailo Keras model is exported with rank4 layers, expands dimensions for the y_test to
↳match the model output shape
y_test = np.expand_dims(y_test, axis=[1, 2])
full_precision_result = np.count_nonzero(np.argmax(y_infer_fp, axis=-1) == np.
↳argmax(y_test, axis=-1)) / len(y_test)
quantize_result = np.count_nonzero(np.argmax(y_infer_q, axis=-1) == np.argmax(y_
↳test, axis=-1)) / len(y_test)
print(f"Test accuracy (floating point): {100 * full_precision_result:.3f} (Top-1)")
print(f"Test accuracy (quantized): {100 * quantize_result:.3f}% (Top-1)")
print(f"Degradation: {100 * (full_precision_result - quantize_result):.3f}")
```

4.7.4. Running QAT

In this final step, a quantized model will be optimized to enhance its accuracy. The `runner.get_keras_model` API will be used to obtain a Keras model initialized with the quantized weights. The model can then be trained using straight-through estimator (STE) method.

- The `tf.distribute.MirroredStrategy` API is being used to enable synchronous training across multiple GPUs on the same machine.
- The `runner.get_keras_model` API must be used with `trainable=True` to allow training (usage of fit).
- To the Keras model additional layers, post-processing or other models can be added. For example, here a new `tf.keras.layers.Softmax` layer is being added.
- For training, use the `fit` API provided by Keras. Training can be done with customized loss functions and different optimizers.
- After training is complete, update the `ClientRunner` weights with the updated model. This is done using the `runner.set_keras_model` API. Only allowed changes to the Keras model includes weight changes. Once the new weights are updated, compile the model with the new weights using the `runner.compile` API.

```
[ ]: with tf.distribute.MultiWorkerMirroredStrategy().scope():
    with runner.infer_context(InferenceContext.SDK_QUANTIZED) as ctx:
        # Hailo Keras model is exported with rank4 layers, expands dimensions for the y_
        ↳train to match the model output shape
        y_train = np.expand_dims(y_train, axis=[1, 2])

        # move numpy data to tf.data.Dataset to be used by multiple GPUs
        train_data = tf.data.Dataset.from_tensor_slices((x_train, y_train))
```

(continues on next page)

(continued from previous page)

```

train_data = train_data.batch(128)
options = tf.data.Options()
options.experimental_distribute.auto_shard_policy = tf.data.experimental.
↳AutoShardPolicy.OFF
train_data = train_data.with_options(options)

# get the Hailo Keras model for training
model = runner.get_keras_model(ctx, trainable=True)
model.build(train_data)
inputs = keras.Input(input_shape)
x = model(inputs)
outputs = keras.layers.Softmax(axis=0)(x)
new_model = keras.Model(inputs, outputs)

# adding external loss.
# note that this compile API only compiles the Keras model but doesn't compile the
↳model to the Hailo HW.
new_model.build(train_data)
new_model.compile(
    loss=keras.losses.CategoricalCrossentropy(),
    optimizer=keras.optimizers.Adam(learning_rate=1e-6),
    metrics=["accuracy"],
)

# start QAT
log = new_model.fit(train_data, batch_size=128, epochs=10)

# set the Keras model after training. The model is already optimized, so do not run
↳optimize() again.
runner.set_keras_model(model)

# plot training curve
plt.plot(log.history["accuracy"])
plt.title("Model Accuracy")
plt.ylabel("Top-1")
plt.xlabel("Epoch")
plt.grid()
plt.show()

```

```

[ ]: # Evaluate the results
with runner.infer_context(InferenceContext.SDK_QUANTIZED) as q_ctx:
    y_infer_qat = runner.infer(q_ctx, x_test)

qat_result = np.count_nonzero(np.argmax(y_infer_qat, axis=-1) == np.argmax(y_test,
↳axis=-1)) / len(y_test)
print(f"Test accuracy (quantized) before QAT: {100 * quantize_result:.3f} (Top-1)")
print(f"Test accuracy (quantized) after QAT: {100 * qat_result:.3f} (Top-1)")
print(f"Accuracy improvement: {100 * (qat_result - quantize_result):.3f}")

```


4.7.5. Knowledge Distillation and QAT

QAT can gain additional accuracy with training using a teacher (the full precision model) to train the student model (the quantized model) - [knowledge distillation](#). To use the full precision model, call the `runner.get_keras_model` API with a different context and change the loss accordingly. In the following code, a new class `Distiller` is generated to distill the full precision and combine with the supervision of the labels.

- Note that, Hailo's FineTune algorithm works in the same way as well (more information can be found in the DFC user guide).

```
[ ]: class Distiller(keras.Model):
    def __init__(self, student, teacher):
        super().__init__()
        self._teacher = teacher
        self._student = student

    def compile(self, optimizer, metrics, student_loss_fn, distillation_loss_fn,
        ↪alpha=0.1, temperature=3):
        self._teacher.model.compile()
        self._student.model.compile()
        super(Distiller, self).compile(optimizer=optimizer, metrics=metrics)
        self._student_loss_fn = student_loss_fn
        self._distillation_loss_fn = distillation_loss_fn
        self._alpha = alpha
        self._temperature = temperature

    def build(self, input_shape):
        if not self._teacher.model.built:
            self._teacher.model.build(input_shape)
        if not self._student.model.built:
            self._student.model.build(input_shape)

    def train_step(self, data):
        # unpack data (image, label)
        x, y = data

        # forward pass of teacher
        teacher_predictions = self._teacher.model(x, training=False)
        trainable_vars = [v._value for v in self._student.trainable_variables]

        with tf.GradientTape() as tape:
            tape.watch(trainable_vars)
            # forward pass of student
            student_predictions = self._student.model(x, training=True)

            # compute supervised loss
            student_loss = self._student_loss_fn(y, student_predictions / self._
            ↪temperature) * self._temperature

            # compute distillation loss
            distillation_loss = (
                self._distillation_loss_fn(
                    teacher_predictions / self._temperature,
                    student_predictions / self._temperature,
                )
                * self._temperature**2
            )

            total_loss = self._alpha * student_loss + (1 - self._alpha) * distillation_loss

        # compute gradients
        gradients = tape.gradient(total_loss, trainable_vars)
```

(continues on next page)

(continued from previous page)

```
# update weights
self.optimizer.apply_gradients(zip(gradients, trainable_vars))

for metric in self.metrics:
    metric.update_state(y, student_predictions)
results = {m.name: m.result() for m in self._metrics}
results.update(
    {"total_loss": total_loss, "student_loss": student_loss, "distillation_loss":
→ distillation_loss},
)
return results
```

```
[ ]: # Parse the TFlite model
runner = ClientRunner(hw_arch="hailo8")
runner.translate_tf_model(tflite_model_path)

# Optimize the model: enforce 40% 4bit weights without optimization
model_script_commands = [
    "model_optimization_config(compression_params, auto_4bit_weights_ratio=0.6)\n"
    "model_optimization_flavor(optimization_level=0)\n",
]

runner.load_model_script("".join(model_script_commands))
runner.optimize(x_train[:1024])

with runner.infer_context(InferenceContext.SDK_QUANTIZED) as ctx_q:
    with runner.infer_context(InferenceContext.SDK_FP_OPTIMIZED) as ctx_fp:
        # get the Hailo Keras model for training
        student = runner.get_keras_model(ctx_q, trainable=True)

        # get the full precision model for kd
        teacher = runner.get_keras_model(ctx_fp, trainable=False)

        # create the kd model
        distiller = Distiller(student=student, teacher=teacher)
        distiller_input_shapes = (1, *student.model.get_input_shapes()[0])
        distiller.build(distiller_input_shapes)
        distiller.compile(
            optimizer=keras.optimizers.Adam(learning_rate=1e-6),
            metrics=[keras.metrics.SparseCategoricalAccuracy()],
            student_loss_fn=keras.losses.CategoricalCrossentropy(),
            distillation_loss_fn=keras.losses.KLDivergence(),
            alpha=0.5,
            temperature=10,
        )

        # start QAT
        log = distiller.fit(x_train, y_train, batch_size=128, epochs=10)

        # set the Keras model after training
        runner.set_keras_model(student)
```

```
[ ]: # Evaluate the results
with runner.infer_context(InferenceContext.SDK_QUANTIZED) as q_ctx:
    y_infer_qat = runner.infer(q_ctx, x_test)

qat_with_kd_result = np.count_nonzero(np.argmax(y_infer_qat, axis=-1) == np.
→ argmax(y_test, axis=-1)) / len(y_test)
print(f"Test accuracy (quantized) with QAT: {100 * qat_result:.3f} (Top-1)")
```

(continues on next page)

(continued from previous page)

```
print(f"Test accuracy (quantized) with QAT and KD: {100 * qat_with_kd_result:.3f}%  
→(Top-1)")  
print(f"Accuracy improvement: {100 * (qat_with_kd_result - qat_result):.3f}%")
```

5. Building Models

This section describes the process of taking ONNX/TF trained model and compiling them to a Hailo executable binary file (HEF). The main API for this process is the [ClientRunner](#). The client runner is a stateful object that handles all stages. In each stage, the client runner can be serialized into an Hailo archive file (HAR) that can be loaded in the future to initialize a new client runner. There are three main stages: Translation, Optimization and Compilation.

1. **Translation:** this process takes an ONNX/TF model and translates it into Hailo's internal representation. For that, the [translate_tf_model\(\)](#) method or the [translate_onnx_model\(\)](#) method should be used. For examples, see the [Parsing Tutorial](#). At the end of this stage the state of the runner is changed from **Uninitialized** to **Hailo Model** and new functionality is available:

- A. Running inference on [SDK_NATIVE](#) context. For further details refer to: [Model Optimization Tutorial](#).
- B. Profile the model to obtain model overview. For example, using the command line interface: `hailo profiler --help`.

Note: The same functionality can be obtained using the command line interface. For example, `hailo parser {tf, onnx} --help`.

2. **Optimization:** in this stage the model is being optimized before compilation using the [optimize\(\)](#) method. The [optimize](#) method runs several steps of optimization including quantization which may degrade the model accuracy; therefore, evaluation is needed to verify the model accuracy. For further information see [Model Optimization Workflow](#) and [Model Optimization Tutorial](#). The method [load_model_script\(\)](#) can be chosen to use advanced configuration before calling optimize. At the end of the optimization stage, the state of the runner is changed from **Hailo Model** to **Quantized Model** and new functionality is available:

- A. Running inference on [SDK_QUANTIZED](#) context (quantized model emulation). For further details refer to: [Model Optimization Tutorial](#). This step allows the measurement of the degradation due to quantization of the model without executing on the device. It is recommended to evaluate the quantized model in emulation before proceeding to compilation.
- B. Run the [analyze_noise\(\)](#) method to execute the layer noise analysis tool and analyze the model's accuracy. This tool is useful to debug quantization issues in case of large degradation in your quantized model. For further details see the [Layer Noise Analysis Tutorial](#).

An alternative option is to use the [optimize_full_precision\(\)](#) method before calling [optimize\(\)](#) to run only part of the optimization process. In which case, the runner state will be **FP optimized model** and it will include model modifications, such as normalization or resize, but without the quantization process. Runner in this state can run inference with [SDK_FP_OPTIMIZED](#) context, see example in: [Model Optimization Tutorial](#).

Note: The same functionality can be obtained using the command line interface. For example, `hailo optimize --help`

3. **Compilation:** this step takes a runner in state **Quantized Model** and compiles it to a Hailo executable binary file (HEF). At the end of this stage the state of the runner is changed from **Quantized Model** to **Compiled Model**, which allows the exporting of a binary HEF file to run on the Hailo hardware.

- A. Save the HEF file to be used with the HailoRT. For further details refer to the [Compilation Tutorial](#).
- B. Run Inference on hardware. For further details refer to: [Inference Tutorial](#).

Note: The same functionality can be obtained using the command line interface. For example, `hailo compiler --help`

The following block diagram illustrates how the runner states and the API switch between each other.

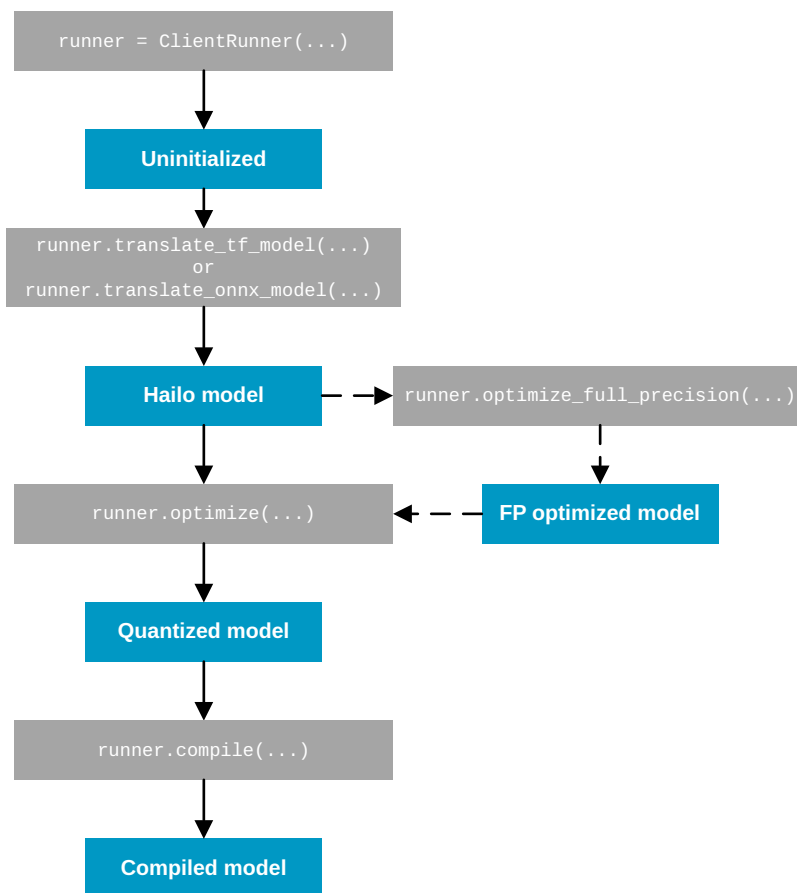


Figure 4. Description of the `ClientRunner` states and its API

5.1. Translating Tensorflow and ONNX Models

5.1.1. Using the Tensorflow Parser

The Dataflow Compiler Tensorflow parser supports the following frameworks:

- Tensorflow v2.18.0, including Keras v3.5.0.
- Tensorflow Lite v2.18.0.

The Parser translates the model to Hailo Archive (.har) format. Hailo Archive is a tar.gz archive file that captures the “state” of the model - the files and attributes used in a given stage from parsing to compilation.

The basic HAR file includes:

- HN file, which is a JSON-like representation of the graph structure that is deployed to the Hailo hardware.
- NPZ file, which includes the weights of the model.

More files are added when the optimization and compilation stages are done.

Note: Advanced users can use the *hailo har* CLI tool to extract the internal files of the HAR.

Note: Tensorflow 1.x models (checkpoints, frozen protobuf) support is planned for deprecation on April 2024. It is recommended to export/convert to TFLite via Keras & Tensorflow's APIs (Python/CLI), see more info on the official ([Tensorflow guide](#)).

Note: APIs that do not create new nodes in the TF graph (such as `tf.name_scope` and `tf.variable_scope`) are not listed because they do not require additional parser support.

Tensorflow models are translated to HAR by calling the `translate_tf_model()` method of the `ClientRunner` object. The `nn_framework` optional parameter tells the Parser whether it's a TF1 or TF2 model. The `start_node_names` and `end_node_names` optional parameters tell the Parser which parts to include/exclude from parsing. For example, the user may want to exclude certain parts of the post-processing and evaluation, so they won't be compiled to the Hailo device.

See also:

The [Parsing Tutorial](#) shows how to use this API.

The supported input formats are:

- TF1/TF2 models – [Deprecated] checkpoints and frozen graphs (.pb), as well as saved_model formats.
- TF Lite models – tflite format.

Supported Layers from Tensorflow

Table 1. Supported Tensorflow Layers Table

Hailo Layer	Keras Layers	TFLite Operators	Additional Info
Argmax	tf.keras.backend.argmax	ARG_MAX	
Average Pooling	tf.keras.ops.mean, tf.keras.layers.AveragePooling2D	AVERAGE_POOL_2D	
Batch Normalization	tf.keras.layers.BatchNormalization		

Continued on next page

Table 1 – continued from previous page

Hailo Layer	Keras Layers	TFLite Operators	Additional Info
<i>Concat</i>	tf.keras.ops.concatenate, tf.keras.ops.tile, tf.keras.layers.Concatenate	CONCATENATION	
<i>Convolution</i>	tf.keras.layers.Conv1D, tf.keras.layers.Conv2D, tf.keras.layers.Conv3D	CONV_2D	Conv3D (preview)
<i>Deconvolution</i>	tf.keras.layers.Conv2DTranspose	TRANSPOSE_CONV	
<i>Depthwise Convolution</i>	tf.keras.layers.DepthwiseConv2D	DEPTHWISE_CONV_2D	
<i>Depth To Space</i>	tf.nn.depth_to_space	DEPTH_TO_SPACE	DCR/CRD modes, can be implemented by combining reshape + transpose, or used in a Lambda layer.
<i>Dense</i>	tf.keras.layers.Dense, tf.keras.ops.matmul	FULLY_CONNECTED	
Dropout	tf.keras.layers.dropout		Ignored
<i>Elementwise Add</i>	tf.keras.ops.add	ADD	
<i>Elementwise Sub</i>	tf.keras.ops.subtract	SUB	
<i>Elementwise Mul</i>	tf.keras.ops.multiply	MUL	
<i>Elementwise Div</i>	tf.keras.ops.divide	DIV	
<i>Elementwise Max</i>	tf.keras.ops.maximum	MAXIMUM	Concat + ReduceMax
<i>Elementwise Min</i>	tf.keras.ops.minimum	MINIMUM	Concat + ReduceMin
<i>External Pad</i>	tf.keras.ops.pad	PAD	
<i>Features Splitter</i>	tf.keras.ops.split	SPLIT	
GRU			Translated as a block of several Hailo layers
Instance Normalization			Translated as a block of several Hailo layers
Layer Normalization			Translated as a block of several Hailo layers
L2 Normalization	tf.math.l2_normalize	L2_NORMALIZATION	Translated as a block of several Hailo layers, can be used in Lambda layer.
LogSoftmax			Translated as a block of several Hailo layers
<i>LSTM</i>			See limitations on Supported RNN/LSTM layers
<i>Matmul</i>	tf.keras.ops.matmul		Matmul (data-driven) or Dense
<i>Max Pooling</i>	tf.keras.layers.MaxPooling1D, tf.keras.layers.MaxPooling2D	MAX_POOL_2D	
<i>Multihead Attention</i>			Atomic in future ONNX versions, currently entire block is translated

Continued on next page

Table 1 – continued from previous page

Hailo Layer	Keras Layers	TFLite Operators	Additional Info
<i>Normalization, Multiplication by scalar</i>	tf.keras.ops.add, tf.keras.ops.subtract, tf.keras.ops.multiply, tf.keras.ops.divide, tf.keras.ops.negative	ADD, SUB, MUL, DIV, NEG	Input normalization (by mean and std) or multiplication by scalar only
OneHot			Convolution with delta activation, axis=-1 only
<i>Pow</i>	tf.keras.ops.pow	POW, SQUARE	Pow(x, 2) or Pow(x, a) where 0<a<1, respectively
<i>Reduce Max</i>	tf.keras.ops.maximum	REDUCE_MAX	Only on the features axis and with keepdims=True
Reduce Min	tf.keras.ops.minimum	REDUCE_MIN	reduce_min(x) == - (reduce_max(-x))
<i>Reduce Mean</i>	tf.keras.ops.mean	AVERAGE_POOL_2D, MEAN	Average pooling if axes are spatial, reduce mean if axes are channels
<i>Reduce Sum</i>	tf.keras.ops.sum	SUM	
Reduce Sum Square			ReduceSum + FeatureMultiplier(square)
Reduce L2			Translated as a block of several Hailo layers, only as successor to layers with rank 4
<i>Reshape</i>	tf.keras.ops.reshape, tf.keras.layers.Reshape	RESHAPE	See limitations on Supported layers / Reshape and Supported layers / Transpose
<i>Resize</i>	tf.image.resize_images, tf.image.resize_bilinear, tf.image.resize_nearest_neighbor, tf.keras.layers.Resizing	RESIZE_BILINEAR, RESIZE_NEAREST_NEIGHBOR	See limitations on supported Resize layers
<i>RNN</i>			See limitations on supported RNN/LSTM layers
<i>Slice</i>	tf.keras.ops.slice	SLICE, STRIDED_SLICE, GATHER	
<i>Softmax</i>	tf.keras.layers.softmax	SOFTMAX	
<i>Space To Depth</i>	tf.nn.space_to_depth	SPACE_TO_DEPTH	DCR/CRD modes, can be implemented by combining reshape + transpose, or used in a Lambda layer.
<i>Square</i>	tf.keras.ops.square	SQUARE, POW	FeatureMultiplier(square)
<i>Transpose</i>	tf.keras.ops.transpose	TRANSPOSE	See limitations on Supported layers / Reshape and Supported layers / Transpose

Group Conv Parsing

Tensorflow v1.15.4 has no group conv operation. The Hailo Dataflow Compiler recognizes the following pattern and automatically converts it to a group conv layer:

- Several (>2) conv ops, which have the same input layer, input dimensions, and kernel dimensions.
- The features are equally sliced from the input layer into the convolutions.
- They should all be followed by the same concat op.
- Bias addition should be before the concat, after each conv op.
- Batch normalization and activation should be after the concat.

Feature Shuffle Parsing

Tensorflow v1.15.4 has no feature shuffle operation. The Hailo Dataflow Compiler recognizes the following pattern of sequential ops and automatically converts it to a feature shuffle layer:

- `tf.reshape` from 4-dim `[batch, height, width, features]` to 5-dim `[batch, height, width, groups, features in group]`.
- `tf.transpose` where the groups and features in group dimensions are switched. In other words, this op interleaves features from the different groups.
- `tf.reshape` back to the original 4-dim shape.

Code example:

```
reshape0 = tf.reshape(input_tensor, [1, 56, 56, 3, 20])
transpose = tf.transpose(reshape0, [0, 1, 2, 4, 3])
reshape1 = tf.reshape(transpose, [1, 56, 56, 60])
```

More details can be found in the [Shufflenet paper](#) (Zhang et al., 2017).

Squeeze and Excitation Block Parsing

Squeeze and excitation block parsing is supported. An example Tensorflow snippet is shown below.

```
out_dim = 32
ratio = 4
conv1 = tf.keras.layers.Conv2D(out_dim, 1)(my_input)
x = tf.keras.layers.GlobalAveragePooling2D()(conv1)
x = tf.keras.layers.Dense(out_dim // ratio, activation='relu')(x)
x = tf.keras.layers.Dense(out_dim, activation='sigmoid')(x)
x = tf.reshape(x, [1, 1, 1, out_dim])
ew_mult = conv1 * x
```

Threshold Activation Parsing

The threshold activation can be parsed from:

```
tf.keras.activations.relu(input_tensor, threshold=threshold)
```

where `threshold` is the threshold to apply.

Delta Activation Parsing

The delta activation can be parsed from:

```
val * tf.sign(tf.abs(input_tensor))
```

where `val` can be any constant number.

5.1.2. Using the Tensorflow Lite Parser

Tensorflow Lite models are translated by calling the `translate_tf_model()` method of the `ClientRunner` object. No additional parameters needed.

Note: Hailo supports 32-bit/16-bit TFLite models, since our Model Optimization stage use the high precision weights to optimize the model for Hailo devices. Models that are already quantized to 8-bit are not supported.

See also:

For more info, and some useful examples on converting models from Tensorflow to Tensorflow-lite, refer to the [Parsing Tutorial](#), or the official Tensorflow guide on ([tf lite converter CLI](#)).

5.1.3. Using the ONNX Parser

ONNX models are translated by calling the `translate_onnx_model()` method of the `ClientRunner` object. The supported ONNX opset versions are 8 and 11-17.

Supported Layers from ONNX

Table 2. Supported ONNX Layers Table

Hailo Layer	PyTorch Layers	ONNX Operators	Additional Info
<i>Argmax</i>	<code>torch.argmax</code>	ArgMax	
<i>Average Pooling</i>	<code>torch.nn.AvgPool2D</code>	AveragePool	
<i>Batch Normalization</i>	<code>torch.nn.BatchNorm2D</code>	BatchNormalization	
<i>Concat</i>	<code>torch.cat</code>	Concat	
<i>Convolution</i>	<code>torch.nn.Conv1d</code> , <code>torch.nn.Conv2d</code> , <code>torch.nn.Conv3d</code> , <code>torch.einsum</code>	Conv	Conv3D (preview)
<i>Deconvolution</i>	<code>torch.nn.ConvTranspose2d</code>	ConvTranspose	
<i>Depthwise Convolution</i>	<code>torch.nn.Conv2d</code>	Conv	(groups=channels)
<i>Depth To Space</i>	<code>torch.nn.PixelShuffle</code> , <code>torch.nn.reshape</code> , <code>torch.nn.transpose</code>	DepthToSpace, Re- shape, Transpose	DCR/CRD modes, can be im- plemented by combining re- shape + transpose.
<i>Dense</i>	<code>torch.nn.Linear</code>	Gemm, MatMul	
Dropout	<code>torch.nn.Dropout2d</code>	Dropout	Ignored
<i>Elementwise Add</i>	<code>torch.add</code> , <code>+</code>	Add	
<i>Elementwise Sub</i>	<code>torch.sub</code> , <code>-</code>	Sub	
<i>Elementwise Mul</i>	<code>torch.mul</code> , <code>*</code>	Mul	
<i>Elementwise Div</i>	<code>torch.div</code> , <code>/</code>	Div	

Continued on next page

Table 2 – continued from previous page

Hailo Layer	PyTorch Layers	ONNX Operators	Additional Info
<i>Elementwise Max</i>	torch.maximum	Max	Concat + ReduceMax
<i>Elementwise Min</i>	torch.minimum	Min	Concat + ReduceMin
<i>External Pad</i>	torch.nn.functional.pad	Pad	
<i>Features Splitter</i>	torch.split	Split, Slice	
GRU	torch.nn.GRU	GRU	Translated as a block of several Hailo layers
Instance Normalization	torch.nn.InstanceNorm2d	InstanceNormalization	Translated as a block of several Hailo layers
Layer Normalization	torch.nn.LayerNorm	LayerNormalization	Translated as a block of several Hailo layers
L2 Normalization			Translated as a block of several Hailo layers
LogSoftmax	torch.nn.LogSoftmax	LogSoftmax	Translated as a block of several Hailo layers
<i>LSTM</i>	torch.nn.LSTM	LSTM	See limitations on Supported RNN/LSTM layers
<i>Matmul</i>	torch.matmul	MatMul	Matmul (data-driven) or Dense
<i>Max Pooling</i>	torch.nn.MaxPool1d, torch.nn.MaxPool2d	MaxPool	
<i>Multihead Attention</i>	torch.nn.MultiheadAttention, torch.nn.functional.scaled_dot_product_attention		Atomic in future ONNX versions, currently entire block is translated
<i>Normalization, Multiplication by scalar</i>	torch.add, torch.sub, torch.mul, torch.div	Add, Sub, Mul, Div, Neg	Input normalization (by mean and std) or multiplication by scalar only
OneHot	torch.nn.functional.one_hot	OneHot	Convolution with delta activation, axis=-1 only
<i>Pow</i>	torch.pow	Pow	Pow(x, 2) or Pow(x, a) where 0<a<1, respectively
<i>Reduce Max</i>	torch.max	ReduceMax	Only on the features axis and with keepdims=True
Reduce Min	torch.min	ReduceMin	reduce_min(x) == -(reduce_max(-x))
<i>Reduce Mean</i>	torch.mean	ReduceMean	Average pooling if axes are spatial, reduce mean if axes are channels
<i>Reduce Sum</i>	torch.sum	ReduceSum	
Reduce Sum Square	torch.nn.MSELoss	ReduceSumSquare	ReduceSum + FeatureMultiplier(square)
Reduce L2	torch.norm(p=2)	ReduceL2	Translated as a block of several Hailo layers, only as successor to layers with rank 4
<i>Reshape</i>	torch.reshape	Reshape	See limitations on Supported layers / Reshape and Supported layers / Transpose

Continued on next page

Table 2 – continued from previous page

Hailo Layer	PyTorch Layers	ONNX Operators	Additional Info
<i>Resize</i>	torch.nn.functional.interpolate, torch.nn.Upsample, torch.nn.UpsamplingBilinear2d, torch.nn.UpsamplingNearest2d	Resize, Upsample	See limitations on supported resize layers
<i>RNN</i>	torch.nn.RNN	RNN	See limitations on supported RNN/LSTM layers
<i>Slice</i>	x[start:end:step]	Slice	
<i>Softmax</i>	torch.nn.Softmax2d	Softmax	
<i>Space To Depth</i>	torch.nn.PixelUnshuffle, torch.nn.reshape, torch.nn.transpose	SpaceToDepth	DCR/CRD modes, can be implemented by combining reshape + transpose.
<i>Square</i>	torch.square	Pow, Mul	FeatureMultiplier(square)
<i>Transpose</i>	torch.transpose	Transpose	See limitations on Supported layers / Reshape and Supported layers / Transpose

Exporting Models from PyTorch to ONNX

The following example shows how to export a PyTorch model to ONNX, note the inline comments which explain each parameter in the export function.

Note: Before trying this small example, make sure Pytorch is installed in the environment.

```
# Building a simple PyTorch model
class SmallExample(torch.nn.Module):
    def __init__(self):
        super(SmallExample, self).__init__()
        self.conv1 = torch.nn.Conv2d(96, 24, kernel_size=(1, 1), stride=(1, 1))
        self.bn1 = torch.nn.BatchNorm2d(24)
        self.relu1 = torch.nn.ReLU6()

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu1(x)
        return x

# Exporting the model to ONNX
torch_model = SmallExample()
torch_model.eval()
inp = [torch.randn((1, 96, 24, 24), requires_grad=False)]
torch_model(*inp)
onnx_path = 'small_example.onnx'

# Note the used args:
# export_params makes sure the weight variables are part of the exported ONNX,
# training=TrainingMode.PRESERVE preserves layers and variables that get folded into
# other layers in EVAL mode (inference),
# do_constant_folding is a recommendation by pytorch to prevent issues with PRESERVED
# mode,
# opset_version selects the desired ONNX implementation (currently Hailo support
# opset versions 8 and 11-17).
torch.onnx.export(torch_model, tuple(inp), onnx_path,
```

(continues on next page)

(continued from previous page)

```
export_params=True,
training=torch.onnx.TrainingMode.PRESERVE,
do_constant_folding=False,
opset_version=13)
```

5.1.4. Supported Activation Functions

Table 3. Supported Activations Table

Hailo Activation	PyTorch Layers	ONNX	TF Layers	TFLite
Abs	torch.abs	Abs	tf.math.abs, tf.keras.ops.abs	ABS
Clip	torch.clip, torch.min, torch.max	Clip, Min, Max, Clamp	tf.clip_by_value, tf.keras.ops.clip	MINIMUM, MAXIMUM
Delta (0 if x == 0 else const)	N/A	N/A	N/A	N/A
Elu	torch.nn.ELU	Elu	tf.nn.elu, tf.keras.activations.elu	ELU
Exp	torch.exp		tf.keras.activations.exponential	EXP
Equal	torch.equal	Equal	tf.math.equal	EQUAL
Gelu (preview)	torch.nn.GELU	Mul, Erf	tf.keras.activations.gelu	CUSTOM
Greater	torch.gt	Greater	tf.math.greater, tf.keras.backend.greater	GREATER
Hard-sigmoid	torch.nn.Hardsigmoid	HardSigmoid	tf.keras.activations.hard_sigmoid	MINIMUM, MAXIMUM, MUL, ADD
Hard-swish (pre-view)	torch.nn.Hardswish	Mul, HardSigmoid	tf.keras.ops.multiply, tf.keras.activations.hard_sigmoid	HARD_SWISH
Leaky Relu	torch.nn.LeakyReLU	LeakyRelu	tf.keras.layers.LeakyReLU, tf.keras.activations.leaky_relu	LEAKY_RELU
Less	torch.lt	Less	tf.math.less, tf.keras.ops.less	LESS
Linear	N/A	N/A	N/A	N/A
Log	torch.log	Log	tf.keras.ops.log	LOG
Max	torch.max	Clip	tf.keras.backend.max	MAXIMUM
Min	torch.min	Clip	tf.keras.backend.min	MINIMUM
Mish	torch.nn.Mish	Mul, Tanh, Softplus	tf.keras.activations.mish	MUL, ADD, TANH, LOGISTIC, EXP, LOG
PRelu	torch.nn.PReLU	PRelu	tf.keras.layers.PReLU	PRELU
Relu	torch.nn.ReLU	Relu	tf.nn.relu, tf.keras.activations.relu	RELU
Relu6	torch.nn.ReLU6	Clip	tf.nn.relu6, tf.keras.activations.relu6	RELU6
Sigmoid	torch.nn.Sigmoid	Sigmoid	tf.nn.sigmoid, tf.keras.activations.sigmoid	LOGISTIC
SiLU	torch.nn.SiLU	Mul, Sigmoid	tf.nn.silu, tf.keras.activations.silu	MUL, LOGISTIC
Softplus	torch.nn.Softplus	Softplus	tf.nn.softplus, tf.keras.activations.softplus	LOG, EXP, ADD
Softsign	torch.nn.Softsign	Softsign	tf.nn.softsign, tf.keras.activations.softsign	ABS, ADD, DIV
Sqrt	torch.sqrt	Sqrt	tf.sqrt, tf.keras.ops.sqrt	SQRT
Swish	N/A	N/A	tf.nn.swish	MUL, LOGISTIC
Tanh	torch.nn.Tanh	Tanh	tf.tanh, tf.keras.activations.tanh	TANH
Threshold (x if x >= threshold else 0)	N/A	N/A	N/A	N/A

5.1.5. NMS Post Processing

- **NMS** is a technique that is used to filter the predictions of object detectors, by selecting final entities (e.g., bounding box) out of many overlapping entities. It consists of two stages: score threshold (filtering low-probability detections by their score), and IoU (Intersection over Union, filtering overlapping boxes).
- The NMS algorithm needs to be fed with bounding boxes, which are calculated out of the network outputs. This process is called **"bbox decoding"**, and it consists of mathematically converting the network outputs to box coordinates.
- The bbox decoding calculations can vary greatly from one implementation to another, and include many types of math operations (pow, exp, log, and more).

Hailo supports the following NMS post processing algorithms:

On neural core:

1. SSD/EfficientDet: bbox decoding, score threshold filtering, IoU filtering
2. CenterNet: bbox decoding, score threshold filtering
3. YOLOv5: bbox decoding, score_threshold filtering (also works for YOLOv7)

On CPU:

1. YOLOv5: bbox decoding, score_threshold filtering, IoU filtering (also compatible with YOLOv7)
2. SSD: bbox decoding, score_threshold filtering, IoU filtering (also compatible with EfficientDet)
3. YOLOX: bbox decoding, score_threshold filtering, IoU filtering
4. YOLOv8: bbox decoding, score_threshold filtering, IoU filtering (also compatible with NanoDet)
5. YOLOv5 SEG: bbox decoding, score_threshold filtering, IoU filtering, segmentation mask per instance

Note: NMS on neural code is only supported in models that are compiled to single context. If the model is compiled with multi-context, undefined runtime behavior might occur. On this case, you are encouraged to either try single context compilation using a model script, or perform the NMS on the host platform.

For implementation on hailo devices:

1. When translating the network using the parser, should supply `end_node_names` parameter with the layers that come **before** the post-processing (bbox decoding) section. For Tensorflow models for example, it is performed using the API `translate_tf_model()` or the CLI tool: `hailo parser tf --end-node-names [list]`.

Note: When `hailo` CLI tool is being used, the arguments are separated by spaces: `--end-node-names END_NODE1 END_NODE2 . .` and so on.

2. The post-processing has to be manually added to the translated (parsed) network using a Model Script command (`nms_postprocess`), which is fed to the `hailo optimize` CLI tool, or is loaded with `load_model_script()` before calling the `optimize()` method. The command adds the relevant postprocess to the Hailo model, according to the architecture (e.g. SSD) and the configuration json file.

Note: The output format of the on-chip post-process can be found on HailoRT guide:

- For Python API, look for `tf_nms_format` and see definitions of *Hailo format* and *TensorFlow format*.
- For CPP API, look for `HAILO_FORMAT_ORDER_HAILO_NMS`. It is similar to the *Hailo format* from the Python API.

3. One can experiment with the output format using the `SDK_FP_OPTIMIZED` or the `SDK_QUANTIZED` emulators, before compiling the model. For more information, refer to the [Model Optimization Workflow](#) section.

SSD

SSD (which is also used by EfficientDet models) post-processing consists of bbox decoding and NMS.

Hailo support the specific SSD NMS implementation from [TF Object Detection API SSD](#), tag v1.13.

It is assumed that the default [configurations](#) file is used.

The `ssd_anchor_generator` is used which utilizes the center of a pixel as the anchors centers (so anchors centers cannot be changed):

```
anchor_generator {
  ssd_anchor_generator {
    num_layers: 6
    min_scale: 0.2
    max_scale: 0.95
    aspect_ratios: 1.0
    aspect_ratios: 2.0
    aspect_ratios: 0.5
    aspect_ratios: 3.0
    aspect_ratios: 0.3333
  }
}
```

It is assumed that each branch ("box predictor") has its own anchors repeated on all pixels.

The bbox decoding function currently supported on the chip can be found [here](#) (see `def _decode` which contains the mathematical transformation needed for extracting the bboxes).

For this NMS implementation, the `end_nodes` that come just-before the bbox decoding might be:

```
end_node_names =
[
  "BoxPredictor_0/BoxEncodingPredictor/BiasAdd",
  "BoxPredictor_0/ClassPredictor/BiasAdd",
  "BoxPredictor_1/BoxEncodingPredictor/BiasAdd",
  "BoxPredictor_1/ClassPredictor/BiasAdd",
  "BoxPredictor_2/BoxEncodingPredictor/BiasAdd",
  "BoxPredictor_2/ClassPredictor/BiasAdd",
  "BoxPredictor_3/BoxEncodingPredictor/BiasAdd",
  "BoxPredictor_3/ClassPredictor/BiasAdd",
  "BoxPredictor_4/BoxEncodingPredictor/BiasAdd",
  "BoxPredictor_4/ClassPredictor/BiasAdd",
  "BoxPredictor_5/BoxEncodingPredictor/BiasAdd",
  "BoxPredictor_5/ClassPredictor/BiasAdd"
]
```

An example for the corresponding SSD NMS JSON is found at: `site-packages/hailo_sdk_client/tools/core_postprocess/nms_ssd_config_example_json_notes.txt`, relatively to the virtual environment where the Dataflow Compiler is installed. This example file is not a valid JSON file since it has in-line comments, but a ready-to-use file is on the same folder.

CenterNet

CenterNet post-processing consists of bbox decoding and then choosing the bboxes with the best scores.

Our CenterNet post-processing corresponds to the `CenterNetDecoder` class on Gluon-CV ([link](#)). Therefore we support any CenterNet post-processing which is equivalent in functionality to the above-mentioned code.

For this implementation, the `end_nodes` that come just-before the bbox decoding might be:

```
end_node_names =
[
    "threshold_confidence/threshold_activation/threshold_confidence/re_lu/Relu",
    "CenterNet0_conv3/BiasAdd",
    "CenterNet0_conv5/BiasAdd"
]
```

An example for the corresponding CenterNet JSON can be found at: `site-packages/hailo_sdk_client/tools/core_postprocess/centerNet_example_json_notes.txt`, relatively to the virtual environment where the Dataflow Compiler is installed. This example file is not a valid JSON file since it has in-line comments, but a ready-to-use file is on the same folder.

YOLOv5

YOLOv5 post-processing (true also for YOLOv7) consists of bbox decoding and NMS. The NMS consists of two parts:

1. Filtering bboxes according to their detection score threshold ("low probability" boxes are filtered).
2. Filtering the remaining bboxes with IoU technique: selecting final entities (e.g., bounding box) out of many overlapping entities.

Hailo implemented the bbox decoding in-chip, as well as score threshold filtering. The IoU section needs to be implemented on host, but since score threshold filtering has been performed, the number of bboxes to deal with has decreased by an order of magnitude.

Support for the post-processing from [the original implementation of YOLOv5](#), tag v2.0. has been tested.

The anchors are taken from [this file](#).

The bbox decoding function is described [here](#), on the `Detect` class.

To add a post-process block from the model script, the model needs to be parsed up to the regression layers that lead into the post-process. These regression layers are given by the `end_nodes_names`. For example, for this implementation, on YOLOv5m (tag v2.0) the `end_node_names` might be:

```
end_node_names =
[
    "Conv_307",
    "Conv_286",
    "Conv_265"
]
```

An example for the corresponding YOLOv5 JSON is found at: `site-packages/hailo_sdk_client/tools/core_postprocess/nms_yolov5_example_json_notes.txt`, relatively to the virtual environment where the Dataflow Compiler is installed. This example file is not a valid JSON file since it has in-line comments, but a ready-to-use file is on the same folder.

YOLOV5 SEG

YOLOv5 SEG NMS for instance segmentation task, the post-processing consists of bbox decoding, score threshold filtering, IoU filtering, and segmentation mask per instance. The post-processing runs on the CPU. The output is per image detection which is in the format of: [N, 1, num_max_proposals, 6 + image_dims[0] * image_dims[1]] where the format of axis -1 is [y_min, x_min, y_max, x_max, score, class, flattened masks]

YOLOv6

YOLOv6 post-processing is supported only for tag 0.2.0 and above (for tag 0.1.0 please use the postprocess of YOLOX). It consists of bbox decoding and NMS. As in other meta-architectures, the NMS postprocess performs score and IoU threshold filtering. YOLOv6 is with an anchor, which means there is no need to configure anchors dimension via the NMS config json.

We have tested support for post-processing from [the original implementation of YOLOv6](#), tag 0.2.1.

An example for the corresponding YOLOv6 JSON can be found at: `site-packages/hailo_sdk_client/tools/core_postprocess/nms_yolov6_example_json_notes.txt`, relatively to the virtual environment where the Dataflow Compiler is installed. This example file is not a valid JSON file since it has in-line comments, but a ready-to-use file is on the same folder.

5.1.6. Layer Ordering Limitations

This section describes the TF and ONNX parser limitations regarding ordering of layers.

- Bias – only before Conv, before DW Conv, after Conv, after DW Conv, after Deconv, or after Dense.

5.1.7. Supported Padding Schemes

The following *padding schemes* are supported in Conv, DW Conv, Max Pooling, and Average Pooling layers:

- VALID
- SAME (*symmetric padding*)
- SAME_TENSORFLOW

Other padding schemes are also supported, and will translate into *External Padding* layers.

5.1.8. Reasons and Solutions for Differences in the Parsed Model

In some cases, the translated model may have some differences as compared to the original model:

1. BatchNorm layer in training mode. The difference in this case is because the BN params are static in the hailo model (and folded on relevant layers kernel/bias), and in the original model framework, training mode means that the layer would first update moving mean/var and then normalize its output in place. To avoid this case:
 - PyTorch: export your model to ONNX in preserve or eval mode. For more information, check [Parsing Tutorial](#).
 - Keras: set the model's learning phase to 0 (test).
2. Otherwise, please contact our [support](#).

5.2. Model Scripts

While it is recommended to optimize and compile using the default configuration (using either the [CLI tools](#) or Python APIs), **Model Scripts** make it possible to change the default behavior of the Dataflow Compiler, and to make modifications to the model.

Example CLI usage:

```
hailo optimize <HAR_PATH> --model-script <MODEL_SCRIPT_PATH>
```

Example Python API usage:

```
client_runner.load_model_script('model_script.alls')
client_runner.optimize(calib_dataset)
compiled_hef = client_runner.compile()
```

The model script is a text file that is optionally fed to the Optimize or Compile functions, that contains commands that serve different purposes. The most frequently used and recommended commands are:

- Full-Precision Optimization stage:
 - **[Important]** The [Model Modification commands](#) are used to modify the parsed model, and to add transformations (that were not originally a part of the original ONNX/TF model) to decrease CPU load. Examples:
 - ✦ Apply [normalization](#) at the inputs.
 - ✦ Apply [format or color conversions](#) at the input.
 - ✦ Apply [resize](#) at the input, from the source resolution to the model's resolution.
 - ✦ Add [post processing](#) to your model, on supported architectures only (if not detected automatically during the parsing stage).
- [Numerical Optimization stage](#):
 - **[Important]** The [Optimization level](#) determines how aggressive are the algorithms that are used to increase the accuracy of the quantized model. Higher optimization level requires more time and system resources, but results in higher accuracy.
 - **[Important]** The [Compression level](#) determines the percentage of 4-bit layers, higher amount increases the performance (FPS) of the compiled model. Requires a high optimization level, to regain the accuracy loss.
 - [Resolution reduction command](#) can be used to run the Optimization stage in lower spatial resolution, to decrease its running time.
 - Advanced commands:
 - ✦ The [precision_mode](#) field of the [quantization_param](#) command can be used to apply 16-bit precision to specific layers or outputs, to increase the model accuracy.
 - ✦ [Weights clipping](#) can be used to ignore outliers on a layer's weights, to increase the accuracy.
 - ✦ [Activation clipping](#) can be used to ignore outliers on a layer's activations, to increase the accuracy.
 - ✦ [Global average pool reduction](#) can be used to split a global average pooling layer with a large input resolution.
 - ✦ The [Post-quantization commands](#) allows to change the parameters of the advanced [post-quantization algorithms](#). Although the algorithms and their parameters are automatically chosen according to the Optimization level, manual configuration is possible. For example, decreasing the [AdaRound](#) batch size if it fails.
 - ✦ When Optimization level < 2, you can manually enable the [checker_cfg](#) in order to collect activation statistics, for further analysis using the profiler (it is enabled by default when Optimization level >= 2).
- Compilation stage:

- **[Important]** The *Performance Mode* can be used to compile the model to the highest possible resource utilization, to maximize performance (FPS). Expect the compilation time to increase dramatically.
- *Suggestions* for the compilation could be supplied (for example: compile for platforms with low PCIe bandwidth).
- The *Automatic model script* can be used to pin the compilation results to a previously compiled version of the same model.

Note: Each stage only considers commands that are relevant for it; If a model script is provided at the Optimization stage, but also contains compilation related commands, those commands will be ignored at the Optimization stage, but will be activated during the compilation stage.

Note: If a new model script is given at the Compilation stage, it will not undo the already executed optimization related commands, but will overwrite any compilation related commands that were loaded at the Optimization stage.

5.3. Model Optimization

This step optimizes the model for deployment. The main objective is translating the models' parameters numerically, from floating point to integer representation, it is also known as quantization (or model optimization). This is a mandatory step in order to run models on the Hailo hardware. This step takes place after translating the model from its original framework and before compiling it. For optimized performance, we recommend using a machine with a GPU when running the model optimization and to prepare calibration data with at least 1024 entries.

5.3.1. Model Optimization Workflow

The model optimization has two main steps: Full Precision Optimization and Quantization Optimization.

Full precision optimization includes any changes to the model in the floating-point precision domain, for example *Equalization* [Meller2019], *TSE* [Vosco2021] and pruning.

It also applies any model modifications from the *model script*, such as color conversions or adding post-processing layers.

The next step, quantization, includes compressing the model from floating point to integer representation of the weights (4/8/16-bits) and activations (8/16-bits) and algorithms to improve the model's accuracy, such as *IBC* [Finkelstein2019], *AdaRound* [Nagel2020], and *QFT* [Finkelstein2022]. Both steps may degrade the model accuracy, therefore, evaluation is needed to verify the model accuracy after each step.

To perform these steps, one can use the simple optimization flow. Use the `hailo optimize` CLI, or the `load_model_script()` method followed by the `optimize()` API. Once the optimization process had finished, continue to the compilation stage. The simple optimization flow is presented in this diagram.

The advanced Python workflow can also be followed for tracking the accuracy of the model throughout the stages of the optimization. This advanced workflow, as well as the simple flows, are presented in the *Model Optimization Tutorial*.

The advanced workflow consists of number of stages, which are depicted in the following chart:

1. A preliminary step would be to test the *Native* model before any changes, right after parsing. This stage is important for ensuring that the parsing was successful, and the preprocessing was built (before the start nodes) and post processing (after the end nodes) correctly. As mentioned, the `SDK_NATIVE` emulator is used for this purpose:

```
import tensorflow as tf
from hailo_sdk_client import ClientRunner, InferenceContext
```

(continues on next page)

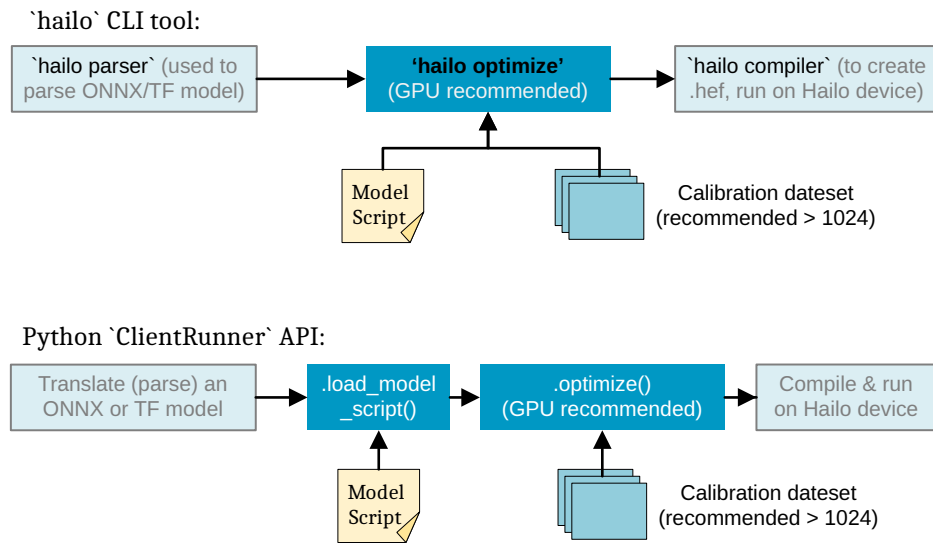


Figure 5. Block diagram of the simple model optimization flow

(continued from previous page)

```

runner = ClientRunner(har=model_path)
with runner.infer_context(InferenceContext.SDK_NATIVE) as ctx:
    output = runner.infer(ctx, input_data)
  
```

The parsed model can also be compared to the original model using the command: *hailo parser* with the flag *-compare*. For more information refer to [reasons](#) section.

2. Load the model script, and use the `optimize_full_precision()` method to apply the model script and the full precision optimizations.
3. Perform full precision validation, when the model is in its final state before the optimization process. This stage is important because it allows to emulate the input and output formats, taking into account the model modifications (normalization, resize, color conversions, etc.). Achieving good accuracy means that the pre/post processing functions are built correctly, and that the infrastructure is ready for testing the quantized model. The `SDK_FP_OPTIMIZED` emulator is used for this purpose:

```

import tensorflow as tf
from hailo_sdk_client import ClientRunner, InferenceContext

runner = ClientRunner(har=model_path)
with runner.infer_context(InferenceContext.SDK_FP_OPTIMIZED) as ctx:
    output = runner.infer(ctx, input_data_modified)
  
```

4. Next, call the model [optimization API](#) to generate an optimized model. To obtain best performance it is recommended to use a GPU machine and a dataset with at least 1024 entries for calibration, which is used to gather activation statistics in the inputs/outputs of each layer. This data is used to optimize the accuracy of the final model. This statistic is being used to map the floating-point values into their integer representation, (a.k.a quantization). Use high quality calibration data (that represents well the validation dataset and the real-life scenario) is crucial for obtaining good accuracy. Supported calibration data types are: Numpy array with shape: [BxHxWxC], NPY file of a Numpy array with shape: [BxHxWxC], directory of Numpy files with each shape: [HxWxC] and `tf.data.Dataset` object with expected return value of: `[{layer_name: input}, _]`.
5. Finally, it is necessary to verify the accuracy of the optimized model to validate the process was successful. In case of large degradation (that doesn't meet the accuracy requirement), re-try the optimization with increased optimization level. **Optimization and Compression levels** allowing the control of the model optimization effort and the model memory footprint. For quick iterations it is recommended to start with the default setting of the

model optimizer (optimization_level=2, compression_level=1). However, when moving to production, work at the highest optimization level (optimization_level=4) to achieve optimal accuracy. With regards to compression, users should increase it when the overall throughput/latency of the application is not satisfactory. Note that increasing compression will have negligible effect on power-consumption so the motivation to work with higher compression level is mainly due to FPS considerations. To verify the accuracy of the quantized model, it is recommended to use the [SDK_QUANTIZED](#) emulator. Note that the quantized model is not bit-exact with the Hailo hardware but provides good and fast approximation of the model accuracy.

```
import tensorflow as tf
from hailo_sdk_client import ClientRunner, InferenceContext

runner = ClientRunner(har=model_path)
with runner.infer_context(InferenceContext.SDK_QUANTIZED) as ctx:
    output = runner.infer(ctx, input_data_modified)
```

Note: Due to known installation issues with Hailo's Docker, GPU usage is possible only when Tensorflow packages are imported before any of Hailo's DFC packages (e.g. client runner, inference context). See code examples above.

A diagram of the advanced optimization flow *is presented below*.

Note: Familiarity with the *runner states diagram* is important for understanding the following diagram.

Note: If problems are encountered with VRAM allocation during stages other than Adaround, it is possible attempt to resolve the issue by disabling the memory growth flag. To do this, set the following environment variable:

```
HAILO_SET_MEMORY_GROWTH=false
```

By doing so, the default memory allocation method for tensorflow GPU will be modified, and the entire VRAM will be allocated and managed internally.

Additionally, if tensorflow is imported, please make sure the SDK is imported before tensorflow is used.

Model Optimization Flavors

The `optimize()` method serves as the model optimization API. This API requires sample dataset (typically ≥ 1024), which is used to collect statistics. After the statistics are collected, they are used to quantize the weights and activations, that is, map the floating point values into integer representation. Hailo's quantization scheme uses uniformly distributed bins and optimizes for the best trade-off between range and precision.

Before calling the `optimize()` API, it is worth considering calling the `load_model_script()` to load a model script (.alls file) that includes commands that modify the model, affect the basic quantization flow and additional algorithms to improve the accuracy and optimize the running time.

To control the optimization grade, it is recommended to set the `optimization_level` argument with the `model_optimization_flavor` command, which will obtain values of 0-4 and control which quantization algorithms will be enabled. Using higher optimization level means the model optimization tool will use more advanced algorithms which are expected to achieve a better accuracy but will take longer to run. Note that optimization levels 2, 4 require at least 1024 images to run and optimization level 3 requires 256. The default setting is `optimization_level=2` unless GPU is not available, or the dataset is not large enough (less than 1024). To reduce the running time, the optimization process uses multiple GPUs when available. To avoid using multiple GPUs use the `model_optimization_config` command. For reference, those are the expected running times for optimizing ResNet-v1-50 with `compression_level=4` using Nvidia A4000 GPU:

- `optimization_level=0`: 59s
- `optimization_level=1`: 206s
- `optimization_level=2`: 256s

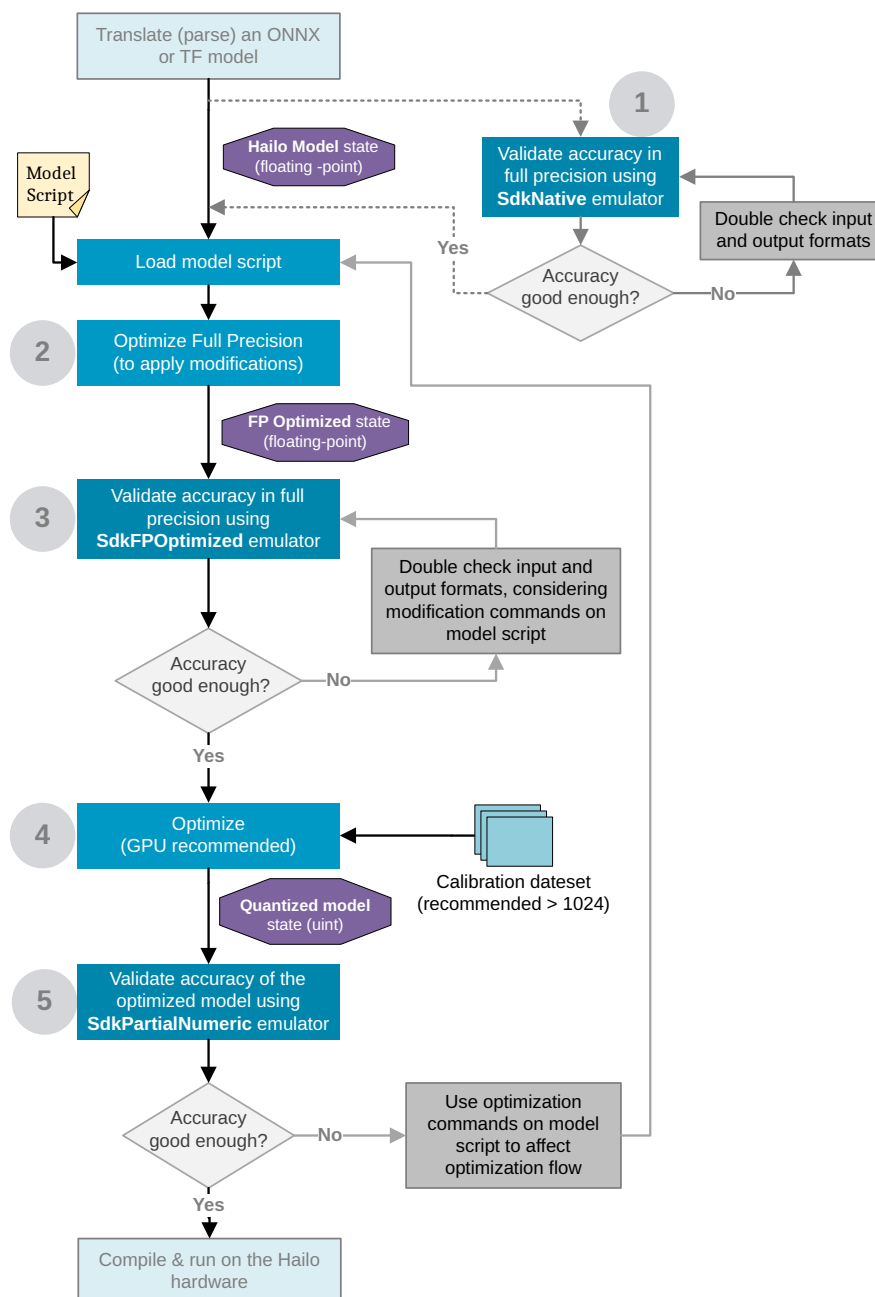


Figure 6. Block diagram of the advanced model optimization flow using Python APIs

- optimization_level=3: 2828s
- optimization_level=4: 11002s

To control the compression degree, use the `compression_level` argument through the `model_optimization_flavor` command, which will obtain values of 0-5 and control the percentage of weights that are quantized to 4-bits (default is using 8-bit precision for weights quantization). Using higher compression level means the compression will be more aggressive and accuracy may be degraded. To recover the accuracy loss, it is recommended to use a higher optimization level as well. High compression rate improves the fps especially for large networks (more than 20M parameters) or when used in a pipeline. The default setting is `Compression_level=1`.

Note: The algorithms that compose each optimization level are expected to change in future versions. To see the current algorithms in use refer to `model_optimization_flavor` command description

The table below displays the results of applying different choices of optimization/compression levels on common CV models.

Table 4. An example of the degradations for the RegNetX-800MF model over various flavor settings. Reported degradations are Top-1 scores over the ImageNet-1K dataset (validation set of 50k images). Note that the RegNetX-800MF model is relatively small (defined as having less than 20M parameters), hence there is only one valid compression level (`compression_level=0`).

	Optimization Level = 0	Optimization Level = 1	Optimization Level = 2	Optimization Level = 3	Optimization Level = 4
Compression Level = 0	0.41	0.16	0.29	-	0.19

Table 5. An example of the degradations for the YOLOv5m model over various flavor settings. Reported degradations are mAP scores over a validation set of 5k samples from the COCO2017 dataset.

	Optimization Level = 0	Optimization Level = 1	Optimization Level = 2	Optimization Level = 3	Optimization Level = 4
Compression Level = 0	4.12	3.35	1.61	-	0.19
Compression Level = 1	4.12	3.26	2.43	1.91	1.25
Compression Level = 4	8.61	7.67	4.78	2.50	1.58

Table 6. An example of the degradations for the DeepLab-v3-MobileNet-v2 model over various flavor settings. Reported degradations are mIoU scores over the PASCAL-VOC dataset. Note that the DeepLab-v3-MobileNet-v2 model is relatively small (defined as having less than 20M parameters), hence there is only one valid compression level (`compression_level=0`).

	Optimization Level = 0	Optimization Level = 1	Optimization Level = 2	Optimization Level = 3
Compression Level = 0	0.72	0.61	1.14	-

Debugging Accuracy

If the quantization accuracy is not sufficient, any of the following methods should be used (after each step, to validate the accuracy of your model):

1. Make sure there are at least 1024 images in the calibration dataset and machine with a GPU.
2. Validate the accuracy of the model in `~hailo_sdk_common.targets.infer_wrapper.InferenceContext.SDK_FP_OPTIMIZED` emulator to ensure pre and post processing are used correctly. Common pitfalls includes mishandling of preprocessing (for example, data normalization) or usage of the wrong data type for calibration.
3. Usage of [BatchNormalization](#) is crucial to obtain good quantization accuracy because it reduces the activation ranges throughout the network, and therefore it is highly recommended to use it during training.
4. Run the layer noise analysis tool to identify the source of degradation. For example, using the CLI command:

```
hailo analyze-noise har_path -data-path data_path
```

5. If you have used `**compression_level**`, lower its value (the default is 0). For example, use the following command in the model script:

```
model_optimization_flavor(compression_level=1)
```

6. Configure higher `**optimization_level**` in the model script, that activates more optimization algorithms and experiment with different optimization levels. For example:

```
model_optimization_flavor(optimization_level=4)
```

7. Configure 16-bit output. Note that using 16-bit output affects the output BW from the Hailo device. For example, using the following model script command:

```
quantization_param(output_layer1, precision_mode=a16_w16)
```

8. Configure 16-bit on specific layers that are sensitive for quantization. Note that if the activation function is not linear/relu/leaky the accuracy might be limited by the activation precision. In addition, note that using 16-bit affects the throughput obtained from the Hailo device. For example, using the following model script command:

```
quantization_param(conv1, precision_mode=a16_w16)
```

9. Try to run with activation clipping using the following model script commands:

```
model_optimization_config(calibration, calibset_size=512), and pre_quantization_optimization(activation_clipping, layers={*}, mode=percentile, clipping_values=[0.01, 99.99])
```

10. Use more data and longer optimization process in Finetune, for example:

```
post_quantization_optimization(finetune, policy=enabled, learning_rate=0.0001, epochs=8, dataset_size=4000)
```

11. Use different loss type in Finetune, for example:

```
post_quantization_optimization(finetune, policy=enabled, learning_rate=0.0001, epochs=8, dataset_size=4000, loss_types=[l2, l2, l2, l2])
```

12. Use quantization aware training (QAT). For more information see [QAT Tutorial](#).

See also:

The [Model Optimization Tutorial](#) which explains how to use the optimization API and the optimization/compression levels and the [Layer Noise Analysis Tutorial](#) which explains how to use the analysis tool.

5.3.2. Flow State Updater

The Flow State Updater module provides advanced debugging capabilities for pinpointing sources of noise in the optimization process. It allows selective modification of model elements, enabling the execution of models with some elements quantized and others in full precision.

This tool can be utilized in two ways:

- Through the Hailo CLI tool.
- Via the Client Runner API.

Specifying Flow State Modifications

Flow state modifications are defined in a YAML configuration file. This file specifies the elements to change using glob syntax and the desired flow target. For example:

```
model_r50_test_sim/ew_add3_to_conv6/*:  
  target: QUANTIZED
```

Flow Target Options

- QUANTIZED: Use the quantized flow.
- NATIVE: Use the full precision flow.

Specifying Elements

Elements can be specified using glob syntax. To ensure accuracy, verify the elements captured by the syntax in the logger output.

Loading the changes specified by the yaml file

Loading Changes Through the CLI

To apply the modifications from the command line, use the `--custom_infer_config` flag with the path to your YAML file. For example:

```
hailomz eval <model_name> --target emulator --custom_infer_config <path_to_yaml>
```

Loading Changes Through the Client Runner API

When using the Client Runner Python API, specify the path to the YAML file in the `custom_infer_config` parameter. For example

```
with runner.infer_context(  
    InferenceContext.SDK_QUANTIZED,  
    custom_infer_config="my_custom_file.yaml",  
) as ctx:  
    runner.infer(ctx, data) # Infer API Optimization Related Model Script Commands
```

5.3.3. Optimization Related Model Script Commands

Information about Model scripts is provided [here](#).

The model script is loaded before running the model optimization by using the `load_model_script()`.

The model script supports model modification commands, which are processed on `optimize()`:

model_modification_commands

In addition, the model script supports 5 optimization commands:

1. *model_optimization_flavor*
2. *model_optimization_config*
3. *quantization_param*
4. *pre_quantization_optimization*
5. *post_quantization_optimization*

model_modification_commands

The model script supports the following model modification commands:

- *input_conversion*
- *transpose*
- *normalization*
- *nms_postprocess*
- *change_output_activation*
- *logits_layer*
- *set_seed*
- *resize*

Note: Each input modification command inserts a layer directly after the input layer, and the commands are applied sequentially as they appear in the script. The final model structure places the most recently executed command's layer immediately after the input layer, resulting in the modification layers appearing in reverse order to the script.

```
# example script to create this structure: input_layer1 -> reshape_yuy2 -> norm_layer1
norm_layer1 = normalization(mean_array, std_array, input_layer1)
reshape_yuy2 = format_conversion(input_layer1, yuy2_to_hailo_yuv)
```

input_conversion

Adds on-chip conversion of the input tensor.

The conversion could be either a **color conversion**:

- *yuv_full_range_to_rgb* - full range YUV conversion from YUV to RGB, implemented by the following kernel: $[[1.0, 1.0, 1.0], [0, -0.343, 1.765], [1.4, -0.711, 0]]$ and bias $[-179.2, 134.912, -225.92]$ terms. Corresponds to `cv::COLOR_YCrCb2RGB` in OpenCV terminology, OpenCV documentation: [OpenCV cv::COLOR_YCrCb2RGB](#). Elaborated matrices and equations can be found at [matrices and equations](#)
- *yuv_to_rgb / yuv601_to_rgb* - in compliance with ITU-R BT.601 standard, implemented by the following kernel: $[[1.164, 1.164, 1.164], [0, -0.392, 2.017], [1.596, -0.813, 0]]$ and bias $[-222.912, 135.616, -276.8]$ terms. Corresponds to `cv::COLOR_YUV2RGB_NV12` in OpenCV terminology, OpenCV documentation: [OpenCV cv::COLOR_YUV2RGB_NV12](#).

- `yuv709_to_rgb` - in compliance with ITU-R BT.709 standard, implemented by the following kernel: $\begin{bmatrix} 1.164 & 1.164 & 0 \\ 0 & -0.213 & 2.112 \\ 1.793 & -0.533 & 0 \end{bmatrix}$ and bias $[-248.128, 76.864, -288.96]$ terms.
- `yuv_full_range_to_bgr` - full range YUV conversion from YUV to BGR, implemented by the following kernel: $\begin{bmatrix} 1.0 & 1.0 & 1.0 \\ 1.765 & -0.343 & 0 \\ 0 & -0.711 & 1.4 \end{bmatrix}$ and bias $[-225.92, 134.912, -179.2]$ terms.
- `yuv_to_bgr` / `yuv601_to_bgr` - in compliance with ITU-R BT.601 standard, implemented by the following kernel: $\begin{bmatrix} 1.164 & 1.164 & 1.164 \\ 2.017 & -0.392 & 0 \\ 0 & -0.813 & 1.596 \end{bmatrix}$ and bias $[-276.8, 135.616, -222.912]$ terms. Corresponds to `cv::COLOR_YUV2BGR` in OpenCV terminology.
- `yuv709_to_bgr` - in compliance with ITU-R BT.709 standard, implemented by the following kernel: $\begin{bmatrix} 1.164 & 1.164 & 1.164 \\ 2.112 & -0.213 & 0 \\ 0 & -0.533 & 1.793 \end{bmatrix}$ and bias $[-288.96, 76.864, -248.128]$ terms.
- `bgr_to_rgb` - which transposes between the R and B channels using an inverse identity matrix as kernel, no bias. Corresponds to `cv2.cvtColor(src, code)` where `src` is a BGR image, and `code` is `cv2.COLOR_BGR2RGB`.
- `rgb_to_bgr` - as the above, transposes between the R and B channels using an inverse identity matrix as kernel, no bias. Corresponds to `cv2.cvtColor(src, code)` where `src` is a RGB image, and `code` is `cv2.COLOR_RGB2BGR`.

Note: The `input_layer` argument is optional. If a layer name is not specified, the conversion will be added after all input layers.

```
rgb_layer = input_conversion(input_layer1, yuv_to_rgb)

# number of return values should match the number of inputs of the network
rgb_layer1, rgb_layer2, ... = input_conversion(yuv_to_rgb)
```

Or a **format conversion**:

- `yuy2_to_hailo_yuv` - Converts the YUY2 format, which is used by some cameras, to YUV. This is useful together with the YUV to RGB layer to create a full vision pipeline YUY2 to YUV to RGB. Corresponds to `cv::COLOR_YUV2RGB_YUY2` in OpenCV terminology.
- `nv12_to_hailo_yuv` - converts the NV12 format, which is used by a growing number of cameras, to YUV format. This is a useful conversion to be used before the first layer to offload this conversion from the host.
- `nv21_to_hailo_yuv` - Converts the NV21 format, which is used by some cameras, to YUV.
- `i420_to_hailo_yuv` - Converts the i420 format, which is used by some cameras, to YUV.
- `tf_rgbx_to_hailo_rgb` - Converts RGBX to Hailo RGB format.

Note: By default, format conversions will only be part of the compiled model but they won't be part of the optimization process. To include emulation supported format conversions - `yuy2_to_yuv`, `tf_rgbx_to_hailo_rgb` and `nv12_to_hailo_yuv` in the optimization process, set `emulator_support=True` inside the command. When setting it to True, the calibration set should be given in the source format.

```
# conversion won't be part of the optimization
yuv_layer = input_conversion(input_layer2, yuy2_to_hailo_yuv)

# conversion will be part of the optimization
yuv_layer = input_conversion(input_layer2, yuy2_to_hailo_yuv, emulator_
    ↪support=True)
```

Or a **hybrid conversion** :

- `yuy2_to_rgb` - which is implemented by adding format conversion `yuy2_to_yuv` and color conversion `yuv_to_rgb`.
- `nv12_to_rgb` - which is implemented by adding format conversion `nv12_to_yuv` and color conversion `yuv_to_rgb`.
- `nv21_to_rgb` - which is implemented by adding format conversion `nv21_to_yuv` and color conversion `yuv_to_rgb`.
- `i420_to_rgb` - which is implemented by adding format conversion `i420_to_yuv` and color conversion `yuv_to_rgb`.

Note: By default, format conversion is part of the hybrid conversion command it behaves as format conversion, i.e. it will be part of the compiled model but not part of the optimization process. To include the supported format conversion - yuy2_to_yuv, tf_rgbx_to_hailo_rgb and nv12_to_hailo_yuv in the optimization process, set `emulator_support=True` inside the command.

```
# yuy2_to_hailo_yuv conversion won't be part of the optimization
yuy2_to_yuv_layer, yuv_to_rgb_layer = input_conversion(input_layer1, yuy2_to_rgb)

# conversion will be part of the optimization
yuy2_to_yuv_layer, yuv_to_rgb_layer = input_conversion(input_layer1, yuy2_to_rgb,
↳emulator_support=True)
```

transpose

Transposes the whole connected component(s) of the chosen input layer(s), so the network runs transposed on chip (improves performance in some cases).

Not supported when there are SpaceToDepth (columns to features) or DepthToSpace (features to columns) reshapes in the network.

HailoRT is responsible for transposing the inputs and outputs on the host side.

```
transpose(input_layer1) # transposing the connected components corresponding to the
↳input layers specified
transpose() # transposing all layers and weights
```

Note: Transposing the network is not supported when the Depth to Space or Space to Depth layers are used.

normalization

Adds on-chip normalization to the input tensor(s).

```
norm_layer1 = normalization(mean_array, std_array, input_layer) # adding
↳normalization layer with the parameters mean & std after the specified input layer.
↳Multiple commands can be used to apply different normalization to each input layer.
norm_layer1, norm_layer2, ... = normalization(mean_array, std_array) # adding
↳normalization layers after all input layers. Return value should match the number of
↳inputs in the network
```

nms_postprocess

For more information about NMS post-processing, refer to [nms_post_processing](#). Hailo's optimized implementation of the NMS post-process is recommended to enhance performance and avoid unnecessary format conversions. This is true for all engines and architectures. In Hailo's implementation, only the boxes with scores above the threshold are converted to FLOAT32 format, which improves significantly overall performance compared to converting the entire output to FLOAT32.

```
# example for adding SSD NMS with config file, architecture is written without ''.
nms_postprocess('nms_config_file.json', meta_arch=ssd)
```

There are a few options for using this command. Note that in each option, the architecture name must be provided, using `meta_arch` argument.

1. Specify only the architecture name.

- If NMS structure was detected during parsing, an autogenerated config file with the values extracted from the original model will be used.
- Otherwise, a default config file will be used.
- Layers that come before the post-process are auto-detected.

For example: `nms_postprocess(meta_arch=ssd)`

2. Specify the architecture name and some of the config arguments.

- If NMS structure was detected during parsing, an autogenerated config file with the values extracted from the original model will be used, edited by provided arguments.
- Otherwise, a default config file will be used, edited by provided arguments.
- Input layers to post-process will be auto-detected.
- The config arguments that can be set via the command are: `nms_scores_th`, `nms_iou_th`, `image_dims`, `classes`.

For example: `nms_postprocess(meta_arch=yolov5, image_dims=[512, 512], classes=70)`

3. Specify the config json path in addition to architecture name.

- The file provided will be used.
- Please note that when providing the config path, do not provide any of the config argument using the command, only inside the file.

For example: `nms_postprocess('config_file_path', meta_arch=centernet)`

The default config files can be found at `site-packages/hailo_sdk_client/tools/core_postprocess/core_postprocess`, relatively to the virtual environment where the Dataflow Compiler is installed:

- `default_nms_config_yolov5.json`
- `default_nms_config_yolov6.json`
- `default_nms_config_yolox.json`
- `default_nms_config_yolo8.json`
- `default_nms_config_centernet.json`
- `default_nms_config_ssd.json`
- `default_nms_config_yolov5_seg.json`

For available architectures see [NMSMetaArchitectures](#).

Networks with YOLOv5 based post-process, perform bbox decoding and score_threshold filtering on the neural core and IOU filtering on CPU by default. Networks with SSD/Centernet based post-process, run on the neural core by default. All other supported post-process architectures run on the CPU by default. Networks with post-process can be configured manually to run either on neural core or on CPU using the engine argument in the relevant model script command. Decoded bounding boxes are normalized between 0 and 1.

There are three supported modes: `nn_core`, `cpu`, `auto`.

- `nn_core` which means the NMS post-process will run on the nn-core, currently supported on YOLOv5, SSD and Centernet.

For example:

```
nms_postprocess(meta_arch=ssd, engine=nn_core)
```

- `cpu` which means the NMS post-process will run on the CPU, currently supported on YOLOv5, YOLOv5 SEG, YOLOv8, SSD and YOLOX:

For example:

```
nms_postprocess(meta_arch=yolov5_seg, engine=cpu, image_dims=[512, 512])
```

Note: The output format of object detection models is [batch_size, num_classes, 5, num_proposals_per_class], where the format of axis 2 dimension is [y_min, x_min, y_max, x_max, score]. The output format of instance segmentation models is [N, 1, num_max_proposals, 6 + image_dims[0] * image_dims[1]] where the format of axis -1 is [y_min, x_min, y_max, x_max, score, class, flattened masks]

- auto currently supported on YOLOv5, performs bbox decoding and score_threshold filtering on the neural core and IoU filtering on CPU.

For example:

```
nms_postprocess('config_file_path', meta_arch=yolov5, engine=auto)
```

Note: When using NMS post-process with the default configuration the *nms_scores_th* value is 0.3. When using NMS post-process on CPU with default configuration the *nms_iou_th* is changed to 0.6.

For performing bbox decoding without NMS use *bbox_decoding_only=True*.

For example:

```
nms_postprocess(meta_arch=yolov5, engine=cpu, bbox_decoding_only=True)
```

Note: Running bbox decoding only on CPU is computationally expensive and may affect the performance, since the decoding is done over all the proposals.

change_output_activation

Changes output layer activation. See the [supported activations](#) section for activation types. If the output layer doesn't support activation, a standalone activation layer will be added.

```
change_output_activation(output_layer, activation) # changing activation function
↳ of specified output layer.
change_output_activation(activation) # changing activation function of all the
↳ output layers.
```

logits_layer

Adds logits layer after an output layer. The supported logits layers are Softmax and Argmax.

Softmax layer can be added under the following conditions:

1. The output layer has rank 2.
2. Total number of softmax layers is less than three.

Argmax layer can be added under the following conditions:

1. The output layer has rank 4.
2. The operation is only on the channels dimension

```
logits_layer1 = logits_layer(output_layer, softmax, 1) # adding logits layer after
↳ the output layer.
logits_layer1, logits_layer2, ..., = logits_layer(argmax, 3) # adding logits layer
↳ after all the output layers.
```

(continues on next page)

set_seed

Sets the global random seed for python random, numpy and tensorflow libraries, and enables operator determinism in tensorflow's backend. Setting the seed ensures reproducibility of quantization results.

Note: When running Finetune algorithm on GPU, tensorflow's back-propagation operators can't perform deterministic results.

Note: Using tensorflow's operator determinism comes at the expense of runtime efficiency, it's recommended to use this feature for debugging only. For more details please refer to tensorflow's [docs](#).

```
set_seed(seed=5)
```

resize:

Performs resize for the input or output tensor(s). The resize can be applied either on-chip or CPU. The default resize method used is bilinear interpolation with align_corners=True, half_pixels=False, and engine=nn_core.

The resize limitations are those of resize bilinear [as described here](#). When the resize ratio is high, the compilation process will be more difficult, as more on-chip memories and sub-clusters are required.

```
resize1 = resize(conv1, resize_shapes=[256,256]) # resize a single layer
resize1 = resize(conv1, resize_shapes=[256,256], resize_method=bilinear, pixels_
↳mode=half_pixels, engine=nn_core)
resize1 = resize(conv1, resize_shapes=[256,256], resize_method=nearest_neighbor,
↳pixels_mode=disable, engine=cpu)
resize1, resize2, ... = resize(resize_shapes=[256,256]) # resize all inputs; return
↳value should match the number of inputs of the network
resize1, resize2, ... = resize(resize_shapes=[256,256], engine=cpu)
```

Note: When using the resize command on an input layer, *resize_shapes* represents the new input shape of the network, while using the command on an output layer *resize_shapes* represents the new output shape of the network

model_optimization_flavor

Configure the model optimization effort by setting compression level and optimization level. The flavor's algorithm will behave as default, any algorithm-specific configuration will override the flavor's default config

Default values:

- compression_level: 1
- optimization_level: 2 for GPU and 1024 images, 1 for GPU and less than 1024 images, and 0 for CPU only.
- batch_size: check default of each algorithm (usually 8 or 32)

Optimization levels: (might change every version)

- -100 nothing is applied - all default algorithms are switched off

- 0 - Equalization
- 1 - Equalization + Iterative bias correction
- 2 - Equalization + Finetune with 4 epochs & 1024 images
- 3 - Equalization + Adaround with 320 epochs & 256 images on all layers
- 4 - Equalization + Adaround with 320 epochs & 1024 images on all layers

Compression levels: (might change every version)

- 0 - nothing is applied
- 1 - auto 4bit is set to 0.2 if network is large enough (20% of the weights)
- 2 - auto 4bit is set to 0.4 if network is large enough (40% of the weights)
- 3 - auto 4bit is set to 0.6 if network is large enough (60% of the weights)
- 4 - auto 4bit is set to 0.8 if network is large enough (80% of the weights)
- 5 - auto 4bit is set to 1.0 if network is large enough (100% of the weights)

Example commands:

```
model_optimization_flavor(optimization_level=4)
model_optimization_flavor(compression_level=2)
model_optimization_flavor(optimization_level=2, compression_level=1)
model_optimization_flavor(optimization_level=2, batch_size=4)
```

Parameters:

Parameter	Values	Default	Re-quired	Description
compression_level	int; $0 \leq x \leq 5$	(Read com-mand doc)	False	Compression level, higher is better but increases degradation, improves fps and latency
optimization_level	int; $100 \leq x \leq 4$	(Read com-mand doc)	False	Optimization level, higher is better but longer, improves accuracy
batch_size	int; $1 \leq x$	(Read com-mand doc)	False	Batch size for the algorithms (adaround, finetune, calibration)

model_optimization_config

- [compression_params](#)
- [negative_exponent](#)
- [globals](#)
- [calibration](#)
- [checker_cfg](#)

compression_params

This command controls layers 4-bit and 16-bit quantization. In 4-bit mode, it reduces some layers' precision mode to a8_w4. The values (between 0 and 1 inclusive) represent how much of the total weight memory usage you want to optimize to 4bit. When the value is 1, all the weights will be set to 4bit, when 0, the weights won't be modified. The 16-bit mode is supported only when setting on the entire network (setting 16-bit value of 1) and without using 4-bit (setting 4-bit value to 0).

Example command:

```
# Optimize 30% of the total weights to use 4bit mode
model_optimization_config(compression_params, auto_4bit_weights_ratio=0.3)
```

Note: If you manually set some layers' precision_mode using quantization_param, the optimization will take it into account, and won't set any weight back to 8bit

Note: If you set 16-bit quantization, all layers activations and weights are quantized using 16 bits. In this case, explicit configuration of layer bias mode is not allowed.

Parameters:

Parameter	Values	Default	Re-quired	Description
auto_4bit_weights_ratio	float; $0 \leq x \leq 1$	0	False	Set a ratio of the model's weights to reduce to 4bit
auto_16bit_weights_ratio	float	0	False	Set a ratio of the model's weights to reduce to 16bit

negative_exponent

During the process of quantization, certain layers may experience bit loss, resulting in reduced precision of the output. To mitigate this issue, this command can be enabled the addition of extra layers. by setting rank to 1 this layer introduces a helper layer that mitigates the the bits lost in the quantized output this can cause a decrease on the FPS of the network. by setting rank to 0 no layer will be introduces and the loss of bits will be delegated to the output.

Example commands:

```
# This will enable the split of conv3 into two layers to not lose precision by a negative
↪exponent >= 1
model_optimization_config(negative_exponent, layers=[conv3], split_threshold=1
↪rank=1)
```

Note: This operation does modify the structure of the model's graph

Parameters:

Parameter	Values	Default	Re-quired	Description
split_threshold	int; 0<x	2	False	Split the layer at the given negative exponent.
rank	int; 0<=x<=1	1	False	How many new layers should be added to the model
auto_clip	{allowed, enabled, disabled}	disabled	False	Clip the range of the accumulator.
auto_remove_offset	{allowed, enabled, disabled}	disabled	False	Remove Offsets that are not reach by the range on calibrations.

globals

Model configuration during the quantization that didn't fit anywhere else...

Example command:

```
model_optimization_config(globals, gpu_policy=auto, max_elementwise_feed_repeat=2)
```

Parameters:

Parameter	Values	Default	Re-quired	Description
multi-proc_policy	{allowed, enabled, disabled}	allowed	False	Select the multiprocessing policy in optimization flow

calibration

During the quantization process, the model will be inferred with small dataset for calibration purposes. The calibration can be configured here. (This replaces the `calib_num_batch` and `batch_size` arguments in `quantize()` API)

Example command:

```
model_optimization_config(calibration, batch_size=4, calibset_size=128)
```

Parameters:

Parameter	Values	Default	Re-quired	Description
batch_size	int; 0<x	8	False	Batch size used during the calibration inference
calibset_size	int; 0<x	64	False	Data items used during the calibration inference

checker_cfg

Checker Config will generate information about the quantization process using the layer analysis tool.

Example commands:

```
# This will disable the algorithm
model_optimization_config(checker_cfg, policy=disabled)
```

Note: This operation does not modify the structure of the model's graph

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{enabled, disabled}	enabled	False	Enable or disable the checker algorithm during the quantization process.
dataset_size	int; 0<x	16	False	Number of images used for profiling.
batch_size	int; 0<x	None	False	Uses the calibration batch_size by default. Number of images used together in each inference step.
analyze_mode	{simple, advanced}	simple	False	The analysis mode that will be used during the algorithm execution (simple/advanced). Simple only execute analysis on the fully quantize net, while advanced also execute layer by layer analysis. Default is simple.
batch_norm_checker	bool	True	False	Set whether the algorithm should display a batch normalization warning message when the gathered layer statistics differ from the expected distribution. Default is True.

quantization_param

The syntax of each quantization_param command in the script is as follows:

```
quantization_param(<layer>, <parameter>=<value>)
```

For example

```
quantization_param(conv1, bias_mode=double_scale_initialization)
```

Multiple parameters can be assigned at once, by simply adding more parameter-value couples, for example:

```
quantization_param(conv1, bias_mode=double_scale_initialization, precision_
↪mode=a8_w4)
```

Multiple layers can be assigned at once when using a list of layers:

```
quantization_param([conv1, conv2], bias_mode=double_scale_initialization,
↪precision_mode=a8_w4)
```

Glob syntax is also supported to change multiple layers at the same time. For example, to change all layers whose name starts with `conv`, use:

```
quantization_param({conv*}, bias_mode=double_scale_initialization)
```

The available parameters are:

1. *bias_mode*
2. *precision_mode*
3. *quantization_groups*
4. *force_range_out*
5. *max_elementwise_feed_repeat*
6. *max_bias_feed_repeat*
7. *null_channels_cutoff_factor*
8. *output_encoding_vector*
9. *gpu_policy*

bias_mode

Sets the layer's bias behavior, there are 2 available bias modes. The modes are:

1. `single_scale_decomposition` when set, the bias is represented by 3 values: `UINT8*INT8*UINT4`.
2. `double_scale_initialization` when set, the layer use 16-bit to represent the bias weight of the layer

Some layers are 16-bit by default (for example, Depthwise), while others are not. Switching a layer to 16-bit, while improving quantization, can have a slightly adverse effect on allocation. If a network exhibits degradation due to quantization, it is strongly recommended to set this parameter for all layers with biases.

All layers that have weights and biases support the `double_scale_initialization` mode.

Example command:

```
quantization_param(conv3, bias_mode=double_scale_initialization)
```

Changed in version 2.8: This parameter was named `use_16bit_bias`. This name is now deprecated.

Changed in version 3.3: `double_scale_initialization` is now the default bias mode for multiple layers.

precision_mode

Precision mode sets the bits available for the layers' weights and activation representation. There are three precision modes that could be set on the model layers using a model script command:

- `a8_w8` - which means 8-bit activations and 8-bit weights. (This is the default)
- `a8_w4` - which means 8-bit activations and 4-bit weights. Can be used to reduce memory consumption. Supported on all layers that have weights. [Compression levels](#) automatically assigns 4-bit to layers in the model, according to the level.
- `a16_w16` - set 16-bit activations and weights to improve accuracy results. Supported on three cases:
 - On any output node (`output_layer_X`)
 - On any supported node(s), see the list below
 - On the full model, in case all its layers are supported (Hailo-8 family only)

Example commands:

```
quantization_param(conv3, precision_mode=a8_w4) # A specific 4bit layer
quantization_param(output_layer1, precision_mode=a16_w16) # A specific 16bit output
↳ layer
quantization_param([conv1, maxpool2], precision_mode=a16_w16) # Multiple 16bit
↳ layers
model_optimization_config(compression_params, auto_16bit_weights_ratio=1) # Full
↳ 16-bit network, in case all layers are supported
```

16-bit precision is supported on the following layers:

- Activations
- Average Pooling
- Concat
- Const Input
- Convolution
- Deconvolution
- Depth to Space
- Depthwise Convolution
- Elementwise Add / Sub*
- External Padding
- Feature Shuffle
- Feature Split
- Fully Connected (dense) [its output(s) must also be 16-bit, or model output layers]
- Max Pooling
- Normalization
- Output Layer
- Reduce Max*
- Reduce Sum*
- Resize*
- Reshape
- Shortcut
- Slice
- Space to Depth

Note: Layers with (*) are supported as long as they are not part of a Softmax chain.

Note: It is recommended to use *Finetune* when using 4-bit weights.

max_bias_feed_repeat

The range is 1-32 (integer only) and the default value is 32.

This parameter determines the precision of the biases. A lower number will result in higher throughput at the cost of reduced precision. This parameter can be switched to 1 for all or some layers, in order to see if higher throughput can be achieved. If this results in high quantization degradation, the source of the degradation should be examined and this parameter should be increased for that layer.

This parameter is not applicable for layers that use the `double_scale_initialization` bias mode.

Example command:

```
quantization_param(conv5, max_bias_feed_repeat=1)
```

quantization_groups

The range is 1-4 (integer only) and the default value is 1.

This parameter allows splitting weights of a layer into groups and quantizing each separately for greater accuracy. When using this command, the weights of layers with more than one quantization group are automatically sorted to improve accuracy.

Using more than one group is supported only by Conv and Dense layers (not by Depthwise or Deconv layers). In addition, it will not be supported if the layers are of conv-and-add kind or rather the last layer of the model (or last layers if there are multiple outputs).

Example command:

```
quantization_param(conv1, quantization_groups=4)
```

force_range_out

This command forces the specified range to the output of given layers in the quantization process.

The expected value for this parameter is a pair of floats `[min, max]` value. `min<=0; max>=0; min<max`. Zero must be within the specified range.

Example command:

```
quantization_param(conv1, force_range_out=[0, 1])
```

max_elementwise_feed_repeat

This command is applicable only for conv-and-add layers. The range is 1-4 (integer only) and the default value is 4.

This parameter determines the precision of the elements in the "add" input of the conv-and-add. A lower number will result in higher throughput at the cost of reduced precision. For networks with many conv-and-add operations, it is recommended to switch this parameter to 1 for all conv-and-add layers, to determine if it's possible to achieve higher throughput. If this results in high quantization degradation, the source of the degradation should be examined and this parameter should be increased for that layer.

Example command:

```
quantization_param(conv5, max_elementwise_feed_repeat=1)
```

null_channels_cutoff_factor

This command is applicable only for layers with fused batch normalization. The default value is 1e-4.

This is used to zero-out the weights of the so called “dead-channels”. These are channels whose variance is below a certain threshold. The low variance is usually a result of the activation function eliminating the results of the layer (for example, a ReLU activation that zeros negative inputs). The weights are zeroed out to avoid outliers that shift the dynamic range of the quantization but do not contribute to the results of the network. The variance threshold is defined by `null_channels_cutoff_factor * bn_epsilon`, where `bn_epsilon` is the epsilon from the fused batch normalization of this layer.

Example command:

```
quantization_param(conv4, null_channels_cutoff_factor=1e-2)
```

output_encoding_vector

This command changes the last layer’s output format, to include a different multiplicative scale for each feature. It raises the accuracy of the model in some cases, in the expense of slightly higher CPU utilization, since the output tensor has to be multiplied with different factor per feature when converting the model outputs back from uint8 or uint16 to floating point precision (a.k.a dequantization).

This command mostly helps when channels with different ranges are concatenated together (for example, some features represent class, and others represent scores).

This command is not available on the following cases:

- Output muxing (an internal feature) has to be disabled: `allocator_param(enable_muxer=False)`.
- When the last layer is a Softmax, NMS, or Resize.
- When HailoRT-postprocess is used:
 - `nms_postprocess` model script command when `engine` is other than `nn_core`.
 - `logits_layer` model script command.

Example command:

```
model_optimization_config(globals, output_encoding_vector=enabled)  
allocator_param(enable_muxer=False)
```

gpu_policy

To enhance your model’s inference capabilities, our client runner supports utilizing multiple GPUs. This functionality accommodates diverse computational needs by offering four distinct modes of operation: *auto*, *data_parallelization*, *model_parallelization*, and *single*. Here’s a succinct overview of each mode:

- *data_parallelization*: Run on parallel data configuration where each gpu will run the same model with different data, batch size will be the same for each gpu. This configuration operates in a data parallel manner. It deploys the same model across multiple GPUs, with each GPU handling a different segment of the data. The batch size remains consistent across all GPUs, ensuring parallel processing of data batches.
- *model_parallelization*: In contrast to the data parallelization approach, the model parallelization mode employs a model parallel strategy. The model is segmented and distributed across multiple GPUs, with different parts of the model running on different GPUs. This setup is beneficial for large models that exceed the memory capacity of a single GPU.
- *single*: Designed for simplicity, the single mode confines the inference process to just one GPU, regardless of the number of available GPUs. This mode is particularly useful for tasks that do not require extensive parallel processing capabilities.

- *auto*: Default configuration. As the default setting, the auto mode intelligently determines the optimal GPU usage strategy based on the available resources and the specific requirements of the task at hand. It defaults to `data_parallelization` mode if the conditions allow, offering a balance between performance and resource utilization."

Example command:

```
runner = ClientRunner(har=model_path)
with runner.infer_context(InferenceContext.SDK_NATIVE, gpu_
    policy=DistributionStrategy.DATA_P) as ctx:
    output = runner.infer(ctx, input_data)
```

pre_quantization_optimization

All the features of this command optimize the model before the quantization process. Some of these commands modify the model structure, and occur before the rest of the commands.

The algorithms are triggered in the following order:

- *dead_channels_removal*
- *zero_static_channels*
- *zero_static_channels per-layer*
- *se_optimization*
- *equalization*
- *equalization per-layer*
- *dead_layers_removal*
- *weights_clipping*
- *activation_clipping*
- *ew_add_fusing*
- *layer_decomposition*
- *smart_softmax_stats*
- *defuse*
- *resolution_reduction*
- *resolution_reduction per-layer*
- *global_avgpool_reduction*
- *add_shortcut_layer*
- *layer_norm_decomposition*
- *matmul_correction*
- *matmul_equalization*
- *matmul_decomposition*
- *switch_concat_with_add*
- *split_ew_mult_by_bit_significance*
- *use_prequantized_weights*
- *conv_decomposition*
- *split_fused_activation*
- *quarot*

- `conv_a16_w4`

dead_channels_removal

Dead channels removal is channel pruning, which removes from the model any layer with both null weights and activation output. This might reduce memory consumption and improve inference time

Example commands:

```
# This will enable the algorithm
pre_quantization_optimization(dead_channels_removal, policy=enabled)
```

Note: This operation will modify the structure of the model's graph

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{enabled, disabled}	disabled	True	Enable or disable the dead channels removal algorithm

zero_static_channels

Zero static channels will zero out the weights of channels that have zero variances to improve quantization.

Example commands:

```
# This will enable the algorithm
pre_quantization_optimization(zero_static_channels, policy=enabled)
```

Note: This operation does not modify the structure of the model's graph

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{enabled, disabled}	enabled	True	Enable or disable the zero static channels algorithm
eps	float; $0 \leq x$	1e-07	False	Threshold value to zero channels for the zero static channels algorithm

zero_static_channels per-layer

This sub-command allows configuring the zerostatic behavior per layer. Example commands:

```
# Disable zero static on conv1 and conv2
pre_quantization_optimization(zero_static_channels, layers=[conv1, conv2],
    ↪ policy=disabled)

# Disable zero static on all conv layers.
pre_quantization_optimization(zero_static_channels, layers={conv*},
    ↪ policy=disabled)
```

Note:

- Not all layers support zero static channels
- if allowed layer gets the behavior from the generic algorithm policy
- if disabled/enabled are explicit, the layer will behave as the LayerFeaturePolicy

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{allowed, enabled, disabled}	allowed	False	Set zero_static behavior to given layer. (default is allowed)

se_optimization

This feature can modify the Squeeze and Excite block to run more efficiently on the Hailo chip. A more detailed explanation of the TSE algorithm can be found here <https://arxiv.org/pdf/2107.02145.pdf>

Example commands:

```
# Apply TSE to the first 3 S&E blocks with tile height of 7
pre_quantization_optimization(se_optimization, method=tse, mode=sequential,
    ↪ count=3, tile_height=7)

# Apply TSE to the first 3 S&E blocks with tile height of 9 to the 1st block, 7 to the 2nd
    ↪ and 5 to the 3rd
pre_quantization_optimization(se_optimization, method=tse, mode=sequential,
    ↪ count=3, tile_height=[9, 7, 5])

# Apply TSE to S&E blocks that start with avgpool1 and avgpool2 layers, with tile height
    ↪ of 7, 5 accordingly
pre_quantization_optimization(se_optimization, method=tse, mode=custom,
    ↪ layers=[avgpool1, avgpool2], tile_height=[7, 5])
```

Note: This operation will modify the structure of the model's graph

Note: An in-depth explanation of the TSE algorithm - <https://arxiv.org/pdf/2107.02145.pdf>

Parameters:

Parameter	Values	Default	Re- quired	Description
method	{tse}	tse	True	Algorithm for Squeeze and Excite block optimization
mode	{sequential, custom, disabled}	disabled	True	How to apply the algorithm on the model
layers	List of {str}	None	False	Required when mode=custom. Set which SE blocks to optimize based on the global avgpool of the block
count	int; 0<x	None	False	Required when mode=sequential. Set how many SE blocks to optimize
tile_height	(int; 0<x) or (List of {int; 0<x})	7	False	Set tile height for the TSE. When list is given, it should match the layers count / the count argument. The tile has to divide the height without residue

equalization

This sub-command allows configuring the global equalization behavior during the pre-quantization process, this command replaces the old equalize parameter from `quantize()` API

Example command:

```
pre_quantization_optimization(equalization, policy=disabled)
```

Note: An in-depth explanation of the equalization algorithm - <https://arxiv.org/pdf/1902.01917.pdf>

Parameters:

Parameter	Values	Default	Re- quired	Description
policy	{enabled, disabled}	enabled	False	Enable or disable the equalization algorithm

equalization per-layer

This sub-command allows configuring the equalization behavior per layer. Allowed policy means the behavior derives from the algorithm config.

Example commands:

```
# Disable equalization on conv1 and conv2
pre_quantization_optimization(equalization, layers=[conv1, conv2],
    policy=disabled)

# Disable equalization on all conv layers.
pre_quantization_optimization(equalization, layers={conv*}, policy=disabled)
```

Note:

- Not all layers support equalization

- Layers are related to other
- Disabling 1 layer, disables all related layers
- Enabling 1 layer won't enable the related layers (it has to be done manually)

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{allowed, enabled, disabled}	allowed	False	Set equalization behavior to given layer. (default is allowed)
equalization_target	{default, activation, weights}	default	False	Set equalization target to activation only or kernel only
force_transparent	{allowed, enabled, disabled}	allowed	False	Force the layer to behave as transparent for equalization

dead_layers_removal

This sub-command allows configuring the dead layers removal

Example command:

```
pre_quantization_optimization(dead_layers_removal, policy=disabled)
```

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{allowed, enabled, disabled}	enabled	False	Enable or disable the dead layers removal algorithm
validate_change	{allowed, enabled, disabled}	enabled	False	if enabled, the algorithm will validate that the removal of the layer by comparing the output of the network before and after the removal

weights_clipping

This command allows changing this behavior for selected layers and applying weights clipping when running the quantization API. This command may be useful in order to decrease quantization related degradation in case of outlier weight values. It is only applicable to the layers that have weights.

- `disabled` mode doesn't take clipping values, and disables any weights clipping mode previously set to the layer.
- `manual` mode uses the clipping values as given.
- `percentile` mode calculates layer-wise percentiles (clipping values are percentiles 0 to 100).
- `mmse` mode doesn't take clipping values, and uses *Minimum Mean Square Estimators* to clip the weights of the layer.

- `mmse_if4b` similar to `mmse`, when the layer uses 4bit weights, and disables clipping when it uses 8-bit weights. (This is the default)

Example commands:

```
pre_quantization_optimization(weights_clipping, layers=[conv2], mode=manual,
↪clipping_values=[-0.1, 0.8])
pre_quantization_optimization(weights_clipping, layers=[conv3], mode=percentile,
↪clipping_values=[1.0, 99.0])
pre_quantization_optimization(weights_clipping, layers={conv*}, mode=mmse)
pre_quantization_optimization(weights_clipping, layers=[conv3, conv4], mode=mmse_
↪if4b)
pre_quantization_optimization(weights_clipping, layers={conv*}, mode=disabled)
```

Note: The dynamic range of the weights is symmetric even if the clipping values are not symmetric.

Parameters:

Parameter	Values	Default	Re-quired	Description
mode	{disabled, manual, percentile, mmse, mmse_if4b}	mmse_if4b	True	Mode of operation, described above
clipping_values	[float, float]	None	False	Clip value, required when mode is percentile or manual

activation_clipping

By default, the model optimization does not clip layers' activations during quantization. This command can be used to change this behavior for selected layers and apply activation clipping when running the quantization API. This command may be useful in order to decrease quantization related degradation in case of outlier activation values.

- `disabled` mode doesn't take clipping values, and disables any activation clipping mode previously set to the layer (This is the default).
- `manual` mode uses the clipping values as given.
- `percentile` mode calculates layer-wise percentiles (clipping values are percentiles 0 to 100).

Note: Percentiles based activation clipping requires several iterations of statistics collection, so quantization might take a longer time to finish.

Example commands:

```
pre_quantization_optimization(activation_clipping, layers=[conv1], mode=manual,
↪clipping_values=[0.188, 1.3332])
pre_quantization_optimization(activation_clipping, layers=[conv1, conv2],
↪mode=percentile, clipping_values=[0.5, 99.5])
pre_quantization_optimization(activation_clipping, layers={conv*}, mode=disabled)
```

Parameters:

Parameter	Values	Default	Re-quired	Description
mode	{disabled, manual, percentile}	disabled	True	Mode of operation, described above
clipping_values	[float, float]	None	False	Clip value, required when mode is percentile or manual
recollect_stats	bool	False	False	Indicates whether stats should be collected after clip

ew_add_fusing

When EW add fusing is enabled, ew add layers will be fused into conv and add layers. Layers with incompatible precision modes won't be fused.

Example commands:

```
# This will enable the algorithm
pre_quantization_optimization(ew_add_fusing, policy=enabled)
```

Note: This operation modifies the structure of the model's graph

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{enabled, disabled}	enabled	True	Enable or disable the ew add fusing optimization
infusible_ew_add_type	{conv, ew_add}	ew_add	False	Decide whether to create a conv or a standalone ew add layer fusing is not possible

layer_decomposition

This sub commands allows toggling layers to decomposition mode, which means 16-bit layers will be implemented with 8-bit layers.

Example commands:

```
# This will decompose a specific layer to increase its precision.
pre_quantization_optimization(layer_decomposition, layers=[conv1],
↪policy=disabled)
pre_quantization_optimization(layer_decomposition, layers=[conv17, conv18],
↪policy=enabled)
```

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{allowed, enabled, disabled}	allowed	False	None

smart_softmax_stats

SmartSoftmaxConfig is an algorithm that collects the stats on a softmax block in an efficient way Example commands:

```
# This will enable the algorithm
pre_quantization_optimization(smart_softmax_stats, policy=enabled)
```

Parameters:

Parameter	Values	Default	Re- quired	Description
policy	{allowed, enabled, disabled}	enabled	False	Enable disable or allow the algorithm
dc_channels	int; 0<x	16	False	Number of channels per group to split the dc component to

defuse

This command allows defusing layer according to the defuse type:

INPUT FEATURES

Defuse input features for a selected dense or conv layer to a selected number of splits. It can also be used to disable defusing of a layer. Example commands:

```
pre_quantization_optimization(defuse, layers=fc1, num_splits=2, defuse_type=INPUT_
↪FEATURES)
# this will disable the fusing of fc2
pre_quantization_optimization(defuse, layers=fc2, num_splits=1, defuse_type=INPUT_
↪FEATURES)
```

Note: num_splits might be overwritten by a larger number due to hw limitations.

MHA

Allows defusing multi-head attention block, represented by its first matmul, to a selected number of splits.

Example commands:

```
pre_quantization_optimization(defuse, layers=matmul1, num_splits=2, defuse_
↪type=MHA)
```

Parameters:

Parameter	Values	Default	Re- quired	Description
num_splits	int	None	False	number of splits required
defuse_type	{input_features, mha}	None	False	defuse type

resolution_reduction

Reduce the model resolution in all input layers in order to optimize the model more efficiently. Marginally affects accuracy. Not supported on models that contain Fully-connected, Matmul an Cross-correlation layers, or when the resolution is too small.

Example commands:

```
# This will enable the algorithm, optimizing over an input shape of [128, 128]
pre_quantization_optimization(resolution_reduction, shape=[128, 128])
```

Note: This operation doesn't modify the structure of the model's graph

Parameters:

Parameter	Values	Default	Re- quired	Description
shape	[int, int]	None	False	The shape to reduce the model resolution to.
interpolation	{disabled, bilinear}	bilinear	False	Interpolation (default) requires dataset in the original model size, disabled required dataset in the reduced resolution.

resolution_reduction per-layer

Sub-command for configuring resolution reduction per input layer, affecting its connected component. Reduce the resolution in order to optimize more efficiently. Marginally affects accuracy. Not supported when containing Fully-connected, Matmul an Cross-correlation layers, or when the resolution is too small.

Example commands:

```
# This will enable the algorithm for input_layer1 connected component, optimizing over
↪ an input shape of [128, 128]
pre_quantization_optimization(resolution_reduction, layers=input_layer1,
↪ shape=[128, 128])
```

Note: This operation doesn't modify the structure of the model's graph

Parameters:

Parameter	Values	Default	Re- quired	Description
shape	[int, int]	None	False	The shape to reduce the component resolution to.
interpolation	{disabled, bilinear}	None	False	Interpolation (default) requires dataset in the original model size, disabled required dataset in the reduced resolution.

global_avgpool_reduction

This command allows reducing the spatial dimensions for global avgpool layers using additional avgpool layer. The kernel size of the added avgpool layer will be [1, h // division_factors[0], w // division_factors[1], 1]

```
pre_quantization_optimization(global_avgpool_reduction, layers=avgpool1, division_
↪factors=[4, 4])
# this will disable the reduction of avgpool1
pre_quantization_optimization(global_avgpool_reduction, layers=avgpool1, division_
↪factors=[1, 1])
```

Parameters:

Parameter	Values	Default	Re-quired	Description
division_factors	[int, int]	None	False	division of the kernel height and width

add_shortcut_layer

Adds an activation layer between “layer” and “target” removes original edge between, activation is linear (by default) before : layer -> target after : layer -> act -> target

Example commands:

```
# Adds activation layer (linear) between conv8 and conv10
pre_quantization_optimization(add_shortcut_layer, layers=conv8, target=conv10)

# Adds activation layer (linear) from conv3 to conv4 and to conv5
pre_quantization_optimization(add_shortcut_layer, layers=conv3, target=[conv4,
↪conv5])
```

Parameters:

Parameter	Values	Default	Re-quired	Description
target	(str) or ([])	None	False	None
name	str	None	False	Name of added shortcut layer. defaults to concatenation of layer-target
activation	str	linear	False	None

layer_norm_decomposition

This sub-command allows configuring norm_layers dec

Example command:

```
pre_quantization_optimization(layer_norm_decomposition, mode=nn_core)
```

Parameters:

Parameter	Values	Default	Re-quired	Description
eq_consumer	bool	False	False	Add normalization layer before the layer norm as equalization consumer

matmul_correction

```
docstring pre_quantization_optimization(matmul_correction, layers=matmul1, correction_type=zp_comp_weights)
pre_quantization_optimization(matmul_correction, layers=[matmul2,matmul4], correction_type=zp_comp_block)
```

Parameters:

Parameter	Values	Default	Re- quired	Description
correction_type	str	zp_comp_weights	False	Type of correction to apply. 'zp_comp_weights', 'zp_comp_block', 'zp_comp_block_2' or 'zp_comp_block_3'

matmul_equalization

```
docstring pre_quantization_optimization(matmul_equalization, layers=matmul1, policy=enabled)
pre_quantization_optimization(matmul_equalization, layers=[matmul2,matmul4], policy=disabled)
```

Parameters:

Parameter	Values	Default	Re- quired	Description
policy	{allowed, enabled, disabled}	allowed	False	Enables matmul Equalization on a given Layer
matmul_bias	{allowed, enabled, disabled}	disabled	False	Adds offset to the matmult to clean the Dc

matmul_decomposition

This sub commands allows toggling Matmul layers to decomposition mode, which means 16-bit layers will be implemented with 8-bit layers.

Example commands:

```
# This will decompose a specific layer to increase its precision.
pre_quantization_optimization(matmul_decomposition, layers=[matmul1],
    ↪ policy=disabled)
pre_quantization_optimization(matmul_decomposition, layers=[matmul1, matmul2],
    ↪ policy=enabled, precision_mode=a16_w8)
```

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{allowed, enabled, disabled}	disabled	False	None
precision_mode	{a8_w8, a8_w4, a16_w16, a16_w8, a16_w4, a8_w4_exp, a16_w16_non_zero, native, a8_w8_a8, a8_w8_a16, a8_w4_a8, a8_w4_a16, a16_w16_a8, a16_w16_a16, a16_w8_a8, a16_w8_a16, a16_w4_a16, a16_w4_a8}	a16_w8	False	Matmul can be decompose into different modes, default is a16_w8, there also can be set to be a16_w16

switch_concat_with_add

If there is concat followed by a conv, this feature converts the concat and the conv to 2 convs and ew-add between them

Example commands:

```
pre_quantization_optimization(switch_concat_with_add, layers=concat1,
↪policy=enabled)
pre_quantization_optimization(switch_concat_with_add, layers={concat*},
↪policy=enabled)
```

Note:

- Relevant only if there is a concat layer with a single output of conv

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{allowed, enabled, disabled}	disabled	False	replace concat and conv with 2 convs and add

split_ew_mult_by_bit_significance

This command allows splitting element-wise multiplication layers by bit significant to allow higher precision.

```
pre_quantization_optimization(split_ew_mult_by_bit_significance, layers=ew_mult1,
↪ num_splits=2)
```

Parameters:

Parameter	Values	Default	Re-quired	Description
num_splits	int	None	False	Number of splits for ew_mult layer

use_prequantized_weights

docstring

```
pre_quantization_optimization(use_prequantized_weights, layers=[matmul2,matmul4], policy=disabled, bits=4)
```

Parameters:

Parameter	Values	Default	Re-quired	Description
mode	{limvals, kernel_vals}	kernel_vals	False	Mode of operation to calc the kernel scale
policy	{allowed, enabled, disabled}	allowed	False	Set pre_weights behavior to given layer. (default is disabled)
bits	int	4	False	Number of bits quantized for the weights currently supports : [4, 8]
groups	int	-1	False	Number of quantization groups currently supports :[-1] (number of channels)

conv_decomposition

This sub commands allows decomosing a single linear/conv layer to multiple layers, to support sub-group quantization. pre_quantization_optimization(conv_decomposition, layers=[conv1],sub_group_size=128)

Parameters:

Parameter	Values	Default	Re-quired	Description
sub_group_size	int	128	False	Size of the sub group
pm_ew_adds	str	a16_w16	False	Precision mode for the ew_adds
allow_eqlize_block	bool	False	False	Allow block equalization
sort_channels_by_stats	bool	False	False	Sort input channels

split_fused_activation

Sub command for splitting fused activation from a main layer.

Example commands:

```
# This will split the activation that fused on conv1 to conv1 layer with linear
↳activation and standalone activation layer.
pre_quantization_optimization(split_fused_activation, layers=conv1,
↳policy=enabled)

# This will split the activations from all conv layers.
pre_quantization_optimization(split_fused_activation, layers={conv*},
↳policy=enabled)
```

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{enabled, disabled, al- lowed}	enabled	False	Split fused activation policy

quarot

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{allowed, enabled, dis- abled}	disabled	False	Enable or disable the quarot algorithm
equalize_inputs	{allowed, enabled, dis- abled}	allowed	False	Enable or disable the equalization of the inputs
equal- ize_outputs	{allowed, enabled, dis- abled}	allowed	False	Enable or disable the equalization of the out- puts

conv_a16_w4

If the convolution layer's kernel size is 1x1, this command decompose the convolution layer work as a8_w4 on a split precision of the input.

Example commands:

```
pre_quantization_optimization(conv_a16_w4, layers=conv1, policy=enabled)
pre_quantization_optimization(conv_a16_w4, layers={conv*}, policy=enabled)
```

Note:

- Relevant only if the convolution layer's kernel size is 1x1.
- This will overwrite the precision mode of that layer set by quantization_param.

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{allowed, enabled, disabled}	disabled	False	enable or disable the a16_w4 decomposition
shift_high	{allowed, enabled, disabled}	disabled	False	Shift the high by 1 (from 7 bit to 8 bit), so that it would suffer less from MAC shift

post_quantization_optimization

All the features of this command optimize the model after the quantization process.

```
post_quantization_optimization(<feature>, <**kwargs>)
```

The features of this command are:

- *bias_correction*
- *bias_correction per-layer*
- *train_encoding*
- *finetune*
- *adaround*
- *adaround per-layer*
- *mix_precision_search*

bias_correction

This sub-command allows configuring the global bias correction behavior during the post-quantization process, this command replaces the old `ibc` parameter from `quantize()` API

Example command:

```
# This will enable the IBC during the post-quantization
post_quantization_optimization(bias_correction, policy=enabled)
```

Note: An in-depth explanation of the IBC algorithm - <https://arxiv.org/pdf/1906.03193.pdf>

Note: Bias correction is recommended when the model contains small kernels or depth-wise layers

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{enabled, disabled}	disabled	False	Enable or disable the bias correction algorithm. When Optimization Level ≥ 1 , could be enabled by the default policy.
cache_compression	{enabled, disabled}	disabled	False	Enable or disable the compression of layer results when cached to disk.

bias_correction per-layer

This sub-command allows enabling or disabling the Iterative Bias Correction (IBC) algorithm on a per-layer basis. Allowed policy means the behavior derives from the algorithm config

Example commands:

```
# This will enable IBC for a specific layer
post_quantization_optimization(bias_correction, layers=[conv1], policy=enabled)

# This will disable IBC for conv layers and enable for the other layers
post_quantization_optimization(bias_correction, policy=enabled)
post_quantization_optimization(bias_correction, layers={conv*}, policy=disabled)
```

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{allowed, enabled, disabled}	allowed	False	Set bias correction behavior to given layer. (default is allowed)

train_encoding

Parameters:

Parameter	Values	Default	Re- quired	Description
policy	{enabled, disabled}	disabled	True	Enable or disable finetune training. When Optimization Level ≥ 1 , could be enabled by the default policy.
dataset_size	int; $0 < x$	1024	False	Number of images used for training; Exception is thrown if the supplied calibration set data stream falls short of that.
batch_size	int; $0 < x$	None	False	Uses the calibration batch_size by default. Number of images used together in each training step; driven by GPU memory constraints (may need to be reduced to meet them) but also by the algorithmic impact opposite to that of learning_rate.
epochs	int; $0 < x$	8	False	Epochs of training
learning_rate	float	None	False	The base learning rate used for the schedule calculation (e.g., starting point for the decay). default value is $0.0002 / 8 * batch_size$. Main parameter to experiment with; start from small values for architectures substantially different from well-performing zoo examples, to ensure convergence.
def_loss_type	{ce, l2, l2rel, cosine}	l2rel	False	The default loss type to use if loss_types is not given
loss_layer_names	List of {str}	None	False	Names of layers to be used for teacher-student losses. Names to be given in Hailo HN notation, s.a. conv20, fc1, etc. Default: the output nodes of the net (the part described by the HN)
loss_types	List of {{ce, l2, l2rel, cosine}}	None	False	(Same length as loss_layer_names) The teacher-student bi-variate loss function types to apply on the native and numeric outputs of the respective loss layers specified by loss_layer_names. For example, ce (standing for 'cross-entropy') is typically used for the classification head(s). Default: the def_loss_type
loss_factors	List of {float}	None	False	(Same length as loss_layer_names) defined bi-variate functions on native/numeric tensors produced by respective loss_layer_names, to arrive at the total loss. Default to 1 for all members.
native_layers	List of {str}	[]	False	Don't quantize given layers during training

Parameters (cont.):

Parameter	Values	Default	Re-quired	Description
native_activations	{allowed, enabled, disabled}	enabled	False	Keep activations native during training.
val_images	int; 0<=x	4096	False	Number of held-up/validation images for evaluation between epochs.
val_batch_size	int; 0<=x	128	False	Batch size for the inter-epoch validation.
stop_gradient_at_loss	bool	False	False	Add stop gradient after each loss layer.
force_pruning	bool	True	False	if true the finetune will force zero weights to stay zeros

Advanced parameters:

Parameter	Values	Default	Re-quired	Description
layers_to_freeze	List of {str}	[]	False	Freeze (don't modify weights&biases for) any layer whose name includes one of this list as a substring. As such, this arg can be used to freeze whole layer types/groups (e.g. pass "conv" to freeze all convolutional).
lr_schedule_type	{co-sine_restarts, exponential, constant}	co-sine_restarts	False	Functional form of the learning rate decay within "decay period" - cosine decay to zero (default), exponential smooth or staircase
decay_rate	float	0.5	False	Decay factor of the learning rate at a beginning of "decay period", from one to the next one. In default case of cosine restarts, the factor of the rate to which learning rate is restarted next time vs. the previous time.
decay_epochs	int; 0<=x	1	False	Duration of the "decay period" in epochs. In the default case of cosine restarts, rate decays to zero (with cosine functional form) across this period, to be then restarted for the next period.
warmup_epochs	int; 0<=x	1	False	Duration of warmup period, in epochs, applied before the starting the main schedule (e.g. cosine-restarts).
warmup_lr	float	None	False	Constant learning rate to be applied during the warmup period. Defaults to 1/4 the base learning rate.
optimizer	{adam, sgd, momentum, rmsprop}	adam	False	set to 'sgd' to use simple Momentum, otherwise Adam will be used.
bias_only	bool	False	False	train only biases (freeze weights).
warmup_strategy	{constant, gradual}	gradual	False	Warmup (learning rate) strategy for warmup stage of the training
wraparound_factor	float; 0<=x	0.1	False	Factor for wraparound loss
shuffle_buffer_size	int; 0<=x	1	False	Buffer size for shuffling the dataset. 0 will use the dataset size

finetune

This sub-command enabled knowledge distillation based fine-tuning of the quantized graph.

Example commands:

```
# enable fine-tune with default configuration
post_quantization_optimization(finetune)

# enable fine-tune with a larger dataset
post_quantization_optimization(finetune, dataset_size=4096)
```

Parameters:

Parameter	Values	Default	Re- quired	Description
policy	{enabled, disabled}	disabled	True	Enable or disable finetune training. When Optimization Level ≥ 1 , could be enabled by the default policy.
dataset_size	int; $0 < x$	1024	False	Number of images used for training; Exception is thrown if the supplied calibration set data stream falls short of that.
batch_size	int; $0 < x$	None	False	Uses the calibration batch_size by default. Number of images used together in each training step; driven by GPU memory constraints (may need to be reduced to meet them) but also by the algorithmic impact opposite to that of learning_rate.
epochs	int; $0 < x$	4	False	Epochs of training
learning_rate	float	None	False	The base learning rate used for the schedule calculation (e.g., starting point for the decay). default value is $0.0002 / 8 * batch_size$. Main parameter to experiment with; start from small values for architectures substantially different from well-performing zoo examples, to ensure convergence.
def_loss_type	{ce, l2, l2rel, cosine}	l2rel	False	The default loss type to use if loss_types is not given
loss_layer_names	List of {str}	None	False	Names of layers to be used for teacher-student losses. Names to be given in Hailo HN notation, s.a. conv20, fc1, etc. Default: the output nodes of the net (the part described by the HN)
loss_types	List of {{ce, l2, l2rel, cosine}}	None	False	(Same length as loss_layer_names) The teacher-student bi-variate loss function types to apply on the native and numeric outputs of the respective loss layers specified by loss_layer_names. For example, ce (standing for 'cross-entropy') is typically used for the classification head(s). Default: the def_loss_type
loss_factors	List of {float}	None	False	(Same length as loss_layer_names) defined bi-variate functions on native/numeric tensors produced by respective loss_layer_names, to arrive at the total loss. Default to 1 for all members.
native_layers	List of {str}	[]	False	Don't quantize given layers during training

Parameters (cont.):

Parameter	Values	Default	Re-quired	Description
native_activations	{allowed, enabled, disabled}	disabled	False	Keep activations native during training.
val_images	int; 0<=x	4096	False	Number of held-up/validation images for evaluation between epochs.
val_batch_size	int; 0<=x	128	False	Batch size for the inter-epoch validation.
stop_gradient_at_loss	bool	False	False	Add stop gradient after each loss layer.
force_pruning	bool	True	False	if true the finetune will force zero weights to stay zeros

Advanced parameters:

Parameter	Values	Default	Re-quired	Description
layers_to_freeze	List of {str}	[]	False	Freeze (don't modify weights&biases for) any layer whose name includes one of this list as a substring. As such, this arg can be used to freeze whole layer types/groups (e.g. pass "conv" to freeze all convolutional).
lr_schedule_type	{co-sine_restarts, exponential, constant}	co-sine_restarts	False	Functional form of the learning rate decay within "decay period" - cosine decay to zero (default), exponential smooth or staircase
decay_rate	float	0.5	False	Decay factor of the learning rate at a beginning of "decay period", from one to the next one. In default case of cosine restarts, the factor of the rate to which learning rate is restarted next time vs. the previous time.
decay_epochs	int; 0<=x	1	False	Duration of the "decay period" in epochs. In the default case of cosine restarts, rate decays to zero (with cosine functional form) across this period, to be then restarted for the next period.
warmup_epochs	int; 0<=x	1	False	Duration of warmup period, in epochs, applied before the starting the main schedule (e.g. cosine-restarts).
warmup_lr	float	None	False	Constant learning rate to be applied during the warmup period. Defaults to 1/4 the base learning rate.
optimizer	{adam, sgd, momentum, rmsprop}	adam	False	set to 'sgd' to use simple Momentum, otherwise Adam will be used.
bias_only	bool	False	False	train only biases (freeze weights).
warmup_strategy	{constant, gradual}	constant	False	Warmup (learning rate) strategy for warmup stage of the training
wraparound_factor	float; 0<=x	0	False	Factor for wraparound loss
shuffle_buffer_size	int; 0<=x	1	False	Buffer size for shuffling the dataset. 0 will use the dataset size

adaround

Adaround algorithm optimizes layers' quantization by training the rounding of the kernel layer-by-layer. To enable it, use high optimization_level (≥ 3), or use the explicit command:

```
post_quantization_optimization(adaround, policy=enabled)
```

It is used by the highest optimization level to recover any degradation caused by quantization, and as such, it is time consuming and requires strong system in order to run.

To reduce some of the memory usage of the algorithm, it is recommended to:

- Ensure dali package is installed
 - For example: `pip install --extra-index-url https://developer.download.nvidia.com/compute/redist nvidia-dali-cuda110 nvidia-dali-tf-plugin-cuda110`
 - DALI is an external package which is being used by AdaRound algorithm to accelerate the running time (see warning raised during the run for more information)
- Use a lower batch size
 - For example, using the `alls` command: `post_quantization_optimization(adaround, policy=enabled, batch_size=8)`
 - Lowering the batch size can reduce the RAM memory consumption but will increase the running time (default is 32)
- Enabled/ disabled cache_compression
 - For example, the `alls` command: `post_quantization_optimization(adaround, cache_compression=enabled, policy=enabled)` enables cache compression.
 - Enables compression on the disk to reduce disk space usage at the expense of increased running time (default is disabled).
- Use smaller dataset_size
 - For example, using the `alls` command: `post_quantization_optimization(adaround, policy=enabled, dataset_size=256)`
 - Using a smaller dataset for Adaround would reduce the memory consumption but might affect the final accuracy (default is 1024)
- Disable bias training
 - For example, using the `alls` command: `post_quantization_optimization(adaround, policy=enabled, train_bias=False)`
 - Disabling bias training can help to reduce running time but might affect the final accuracy (default is true)
- Reduce the number of epochs
 - For example, using the `alls` command: `post_quantization_optimization(adaround, policy=enabled, epochs=100)`
 - Reducing the number of epochs can help to reduce the running time of the algorithm but might affect the final accuracy (default is 320)

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{enabled, disabled}	disabled	False	Enable or disable the adaround algorithm. When Optimization Level ≥ 1 , could be enabled by the default policy.
learning_rate	float; $0 < x$	0.001	False	Learning rate used for gradient descent by the optimizer.
batch_size	int; $0 < x$	32	False	batch size of the ada round algorithm
dataset_size	int; $0 < x$	1024	False	Data samples for adaptive round algorithm
epochs	int; $0 < x$	320	False	Number of train epochs
warmup	float; $0 \leq x \leq 1$	0.2	False	Ratio of warmup epochs out of epochs
weight	float; $0 < x$	0.01	False	Regularization weight; higher value emphasizes rounding cost over reconstruction loss (MSE).
train_bias	bool	True	False	Whether to train bias as well or not (will apply bias correction if layer is not trained)
bias_correction_count	int	64	False	Data count for bias correction
mode	{train_4bit, train_all}	train_4bit	False	default train behavior

Parameters (cont.):

Parameter	Values	Default	Re-quired	Description
cache_compression	{enabled, disabled}	disabled	False	Enable or disable the compression of layer results when cached to disk.

Advanced parameters:

Parameter	Values	Default	Re-quired	Description
b_range	[float, float]	[20, 2]	False	Max, min for temperature decay
decay_start	float; $0 \leq x \leq 1$	0	False	Ratio of round train without round regularization decay (b)

adaround per-layer

This sub commands allow toggling layers in the adaround algorithm individually

Example commands:

```
# This will enable AdaRound for a specific layer
post_quantization_optimization(adaround, layers=[conv1], policy=disabled)
post_quantization_optimization(adaround, layers=[conv17, conv18], policy=enabled)
```

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{allowed, enabled, disabled}	allowed	False	None
epochs	int	None	False	Amount of train epochs for a specific layer
weight	float; 0<x	None	False	Weight of round regularization
b_range	[float, float]	None	False	Temperature decay range
decay_start	float; 0<=x<=1	None	False	Ratio of round train without round regularization decay (b)
train_bias	bool	None	False	Toggle bias training
warmup	float; 0<=x<=1	None	False	Ratio of warmup epochs out of epochs
dataset_size	int; 0<x	None	False	Data samples count for the train stage of the specified layer
batch_size	int; 0<x	None	False	Batch size for train / infer of a layer

mix_precision_search

This algorithm aims to identify the optimal precision configuration for a model by utilizing the signal to noise ratio (SNR). SNR quantifies the extent to which a signal is corrupted by noise. In this context, it aids in determining the trade-off between the compression applied on operations and the error (or noise) introduced as a result of this compression.

Parameters:

Parameter	Values	Default	Re-quired	Description
policy	{enabled, disabled}	disabled	False	Enable or disable
dataset_size	int; 0<x	16	False	Number of images used for profiling.
batch_size	int; 0<x	8	False	Uses the calibration batch_size by default. Number of images used together in each inference step.
snr_cap	int; 0<x	140	False	The maximum SNR value to be considered in the search.
compressions_markers	List of {float}	[0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.2]	False	This will be the compression markers
optimizer	{linear, pareto}	linear	False	Linear, Pareto
output_regulizer	{harmony}	harmony	False	What function will be use to regulate the output
comprecision_metric	{macs, bops, weighs}	bops	False	None

5.4. Model Compilation

5.4.1. Basic Compilation Flow

For Inference Using TAPPAS or With Native HailoRT API

Calling `compile()` compiles the model without loading it to the device returning a binary that contains the compiled model, a HEF file.

Note: The default compilation target is Hailo-8. To compile for different architecture (Hailo-8R for example), use `hw_arch='hailo8r'` as a parameter to the translation phase. For example see the tutorial referenced on the next note.

See also:

The [Compilation Tutorial](#) shows how to use the `compile()` API.

For Inference using ONNX Runtime

After compiling a model, as described in the previous section, that originated from an ONNX model one may choose to extract a new ONNX model that contains the entire network in the original model, with the nodes segmented by the start and end note arguments, replaced by the compiled HEF, by calling `get_hailo_runtime_model()`. This is required if you wish to run inference using [OnnxRT with HailoRT](#).

The CLI: `hailo har-onnx-rt COMPILED-HAR-FILE` can also be used.

This feature is currently in preview, with the following limitations:

- The validated opset versions are 8 and 11-17.
- The model needs to be dividable to three sections:
 - Pre-processing, which connects only to the Main model

- Main model, which connects only to the Post-processing
- Post-processing
- The start_nodes will completely separate the pre-processing from the Main model. No connections from the pre-processing are allowed into the main model, unless they are marked as start_nodes.
- The end_nodes need to separate the main model from the post-processing completely.

`get_hailo_runtime_model()` returns an ONNX model, that you can either pass directly to an ONNXRT session, or first save to a file and then load unto a session.

```
hef = runner.compile() # the returned HEF is not needed when working with ONNXRT
onnx_model = runner.get_hailo_runtime_model() # only possible on a compiled model
onnx_file = onnx.save(onnx_model, onnx_file_path) # save model to file
```

See also:

The [Parsing Tutorial](#) shows how to load a network from an existing model and setting the start and end node arguments.

More information on using OnnxRT with HailoRT is available [here](#).

Changed in version 3.9: Added [context switch](#) support using an allocation script command. The context switch mechanism allows to run a big model by automatically switching between several contexts that together constitute the full model.

For Inference with Python Using TensorFlow

First, to get a runner loaded with compiled model, use one of the options: calling `compile()`, loading a compiled HAR using `load_har()`, or setting the HEF using `hef()`.

To run inference on the model, enter the context manager `infer_context()` and call `infer()` to get the results.

5.4.2. Compilation Related Model Script Commands

Information about Model scripts is provided [here](#).

The compilation related model script commands affect the resources allocation stage of the compilation. Except for the recommended [Performance Mode](#) command, most of these commands are for advanced and edge cases, as the Dataflow Compiler already maximizes the performance by taking many factors into account.

Note: This section uses terminology that is related to the Hailo neural core. Full description of the core architecture is not in the scope of this guide.

Usage

The script is a separate file which can be given to the `load_model_script()` method of the `ClientRunner` class.

For example:

```
client_runner.load_model_script('x.all.s')
compiled_model = client_runner.compile()
```

Automatic Model Script

After the compilation process, in addition to the binary .hef file, the compiled HAR (*Hailo ARchive*) file is created. This HAR file contains the final compilation results, as well as the **automatic model script (.auto.all)** file, that contains the exact instructions for the compiler for creating the same binary file (for the specific Dataflow Compiler version). This model script can be used to compile the model again (from the corresponding quantized HAR file), for a quick compilation.

Extraction of the automatic model script out of the compiled HAR file is done with the command:

```
hailo har extract <COMPILED_HAR_PATH> --auto-model-script-path auto_model_script_file.all.
```

The extracted model script can be used in this manner:

```
hailo compiler <QUANTIZED_HAR_PATH> --model-script auto_model_script_file.all.
```

Context Switch Parameters

Definition

```
context_switch_param(param=value)
```

Example

```
context_switch_param(mode=enabled)
```

Description This command modifies context switch policy and sets several parameters related to it:

- **mode** - Context switch mode. Set to `enabled` to enable context switch: Automatic partition of the given model to several contexts will be applied. Set to `disabled` to disable context switch. Set to `allowed` to let the compiler decide if multi context is required. Defaults to `allowed`.
- **allow_auto_merge_in_multicontext** - Set to `True` to allow auto-merging of layers in multi-context mode. Defaults to `False`. Should be used in conjunction with *Performance Mode* or *resources_params* set to higher utilization.

Allocator Parameters

Definition

```
allocator_param(param=value)
```

Example

```
allocator_param(automatic_ddr=False)
```

Description This sets several allocation parameters described below:

- **timeout** - Compilation timeout for the whole run. By default, the timeout is calculated dynamically based on the model size. The timeout is in seconds by default. Can be given a postfix of 's', 'm', or 'h' for seconds, minutes or hours respectively. e.g. `timeout=3h` will result to 3 hours.
- **automatic_ddr** - when enabled, DDR portals that buffer data in the host's RAM over PCIe are added automatically when required. DDR portals are added when the data needed to be buffered on some network edge exceeds a threshold. In addition, DDR portal is added only when there are enough resources on-chip to accommodate it. Defaults to `True`. Set to `False` to ensure the HEF compatibility to platforms that don't support it, such as Ethernet based platforms.

- `automatic_rescales` - When enabled, Format Conversion (Reshape) layers might be added to networks boundary inputs and outputs. They will be added when supported, and when we have enough resources on-chip to accommodate these functions. When disabled, format conversion layers won't be added to boundary inputs and outputs. on chip. Defaults to allowed (compiler's decision to enable or disable).
- `merge_min_layer_utilization` - Threshold of minimum utilization of the 'control' resource, to start the layer auto merger. Auto-merger will try to optimize on-chip implementation by sharing resources between layers, to reach this control threshold. Auto-merger will not fail if target utilization cannot be reached.
- `enable_lcu_ecc` - When enabled, ECC calculation is enabled. to reduce power, disable the calculation.

Resource Calculation Flow Parameters

Definition

```
resources_param(param=value)
```

Example

```
resources_param(strategy=greedy, max_control_utilization=0.9, max_
↪compute_utilization=0.8)
context0.resources_param(max_utilization=0.25)
```

Description This sets several resources calculation flow parameters described below.

- `strategy` - Resources calculation strategy. When set to `greedy`, adding more resources to the slowest layers iteratively (Maximum FPS search), to reach the highest possible network FPS (per context). Defaults to `greedy`.
- `max_control_utilization` - Number between 0.0 and 1.2. Threshold for `greedy` strategy. Maximum-FPS search will be stopped when the overall control resources on-chip exceeds the given threshold (per context). Defaults to 0.75.
- `max_compute_utilization` - Number between 0.0 and 1.0. Threshold for `greedy` strategy. Maximum-FPS search will be stopped when the overall compute resources on-chip exceeds the given threshold (per context). Defaults to 0.75.
- `max_memory_utilization` - Number between 0.0 and 1.0. Threshold for `greedy` strategy. Maximum-FPS search will be stopped when the overall weights-memory resources on-chip exceeds the given threshold. Defaults to 0.75.
- `max_utilization` - Number between 0.0 and 1.0. Threshold for `greedy` strategy. Maximum-FPS search will be stopped when on-chip utilization of any resource (control, compute, memory) exceeds the given threshold. The parameter overrides default thresholds but not the user provided thresholds specified above.

Two formats are supported - the first one affects all contexts, and the second one only affects the chosen context (see example #2).

Place

Definition

```
place(cluster_number, layers)
```

Example

```
place(2, [layer, layer2])
```

Description This points the allocator to place layers in a specific `cluster_number`. Layers which are not included in any `place` command, will be assigned to a cluster by the Allocator automatically.

Shortcut

Definition

```
shortcut(layer_from, layers_to)
```

Examples

```
shortcut1 = shortcut(conv1, conv2)
shortcut2 = shortcut(conv5, [batch_norm2, batch_norm3])
```

Description This command adds a shortcut layer between directly connected layers. The `layers_to` parameter can be a single layer or a list of layers. The shortcut layer copies its input to its output.

Portal

Definition

```
portal(layer_from, layer_to)
```

Example

```
portal1 = portal(conv1, conv2)
```

Description This command adds a portal layer between two directly connected layers. When two layers are connected using a portal, the data from the source layer leaves the cluster before it gets back in and reaches the target layer. The main use case for this command is to solve edge cases when two layers are manually placed in the same cluster. When two layers are in different clusters, there is no need to manually add a portal between them.

L4 Portal

Definition

```
l4_portal(layer_from, layer_to)
```

Example

```
portal1 = l4_portal(conv1, conv2)
```

Description This command adds a L4-portal layer between two directly connected layers. This command is essentially the same as `portal`, with the key difference that the data will be buffered in L4 memory, as opposed to a regular `portal` which buffers the data in L3 memory. The main use case for this command is when a large amount of data needs to be buffered between two endpoints, and it is required for this data to be buffered in another memory hierarchy.

DDR Portal

Definition

```
ddr(layer_from, layer_to)
```

Example

```
ddr1 = ddr(conv1, conv2)
```

Description This command adds a DDR portal layer between two directly connected layers. This command is essentially the same as `portal`, with the key difference that the data will be buffered in the host, as opposed to a regular `portal` which buffers the data in on-chip memory. Note that this command is supported only in HEF compilations and will work only on supported platforms (i.e. when using the PCIe interface).

Concatenation

Definition

```
concat(layers_from, layer_to)
```

Example

```
concat0 = concat([conv7, conv8], concat1)
```

Description Add a concat layer between several input layers and an output layer. This command is used to split a "large" concat layer into several steps (For example, three concat layers with two inputs instead of a single concat layer with four inputs).

Note: For now this command only supports two input layers (in the argument `layers_from`).

De-fuse

Definition

```
defuse(layer, defuse_number, defuse_type)
```

Examples

```
maxpool1_1, maxpool1_2, maxpool1_c = defuse(maxpool1, 2) # Defuse by output
↳ features
conv4a, conv4b, conv4c, conv4concat = defuse(conv4, 3, defuse_type=SPATIAL) #
↳ Defuse by output columns
dw10_fs, dw10_d0, dw10_d1, dw10_dc = defuse(dw10, 2, defuse_type=INPUT_
↳ FEATURES) # Defuse by input features
maxpool11_fs, maxpool11_d0, maxpool11_d1, maxpool11_dc = defuse(maxpool11, 2,
↳ defuse_type=INPUT_FEATURES) # Defuse by input features
```

Description Defusing splits a logical layer into multiple physical layers in order to increase performance. This command orders the Allocator to defuse the given layer to `defuse_number` physical layers that share the same job, plus an additional concat layer merges all outputs together (and an input feature splitter in case of feature splitter). Like most mechanisms, the defuse mechanism happens automatically, so no user intervention is required.

Several types of defuse are supported, the most common are:

- Feature defuse: Each physical layer calculates part of the output features. Supported layers: Conv, Deconv, Maxpool, Depthwise conv, Avgpool, Dense, Bilinear resize, NN resize.

- Spatial defuse: Each physical layer calculates part of the output columns. Supported layers: Conv, Deconv, Depthwise conv, Avgpool, Argmax, NN resize.
- Input features defuse: Each physical layer receives a part of the input features. Supported layers: Maxpool, Depthwise conv, Avgpool, NN resize, Bilinear resize.

For Feature defuse, don't use the `defuse_type` argument (see examples).

Merge

Definition

```
merge(layer1, layer2)
```

Examples

```
merged_layer = merge(conv46, conv47)
merged_layer_conv12_dw5 = merge(conv12, dw5)
merged_layer_conv15_dw6 = merge(conv15, dw6)
merged_layer_conv18_conv19 = merge(conv18, conv19)
```

Description Merging is a mechanism that uses the same hardware resources to compute two layers. The FPS of the layer will be lower than the two original layers, but unless it is a bottleneck layer, it could save resources and result in total higher FPS. It is supported for a subset of layers and connectivity types. Automatic merging of layers is performed on single context when needed, and could be affected with the `allocator_param(merge_min_layer_utilization)` command.

Compilation Parameters

Definition

```
compilation_param(layer, param=value)
```

Example

```
compilation_param(conv1_d0, resources_allocation_strategy>manual_scs_
↪selection, number_of_subclusters=8, use_16x4_sc=enabled)
```

Description This will update the given layer's compilation param. The command in the example sets the number of subclusters of a specific layer to 8. In addition, it forces 16x4 mode, which means that each subcluster handles 16 columns and 4 output features at once. This is instead of the default of 8 and 8 respectively.

Supported compilation params:

- `resources_allocation_strategy` - defaults to `automatic_scs_selection`, which selects number of subclusters according to global network performance optimization. Change to `manual_scs_selection` to manually choose the number of subclusters (Conv, Dense and DW layers only).
- `use_16x4_sc` - can use 16 pixels multiplication by 4 features - instead of the default 8 pixels by 8 features. This is useful when the number of features is smaller than 8. A table of supported layers is given below (layers that are not mentioned are not supported).
- `no_contexts` - change to True in order to accumulate all the needed inputs for each output row computation in the L3 memory. A table of supported layers is given below (layers that are not mentioned are not supported).
- `balance_output_multisplit` - change to False in order to allow unbalanced output buffers. This can be used to save memory when there are "long" skip connections between layers.

- `number_of_subclusters` - force the usage of a specific number of subclusters. Make sure the `resource_allocation_strategy` value is set to `manual_scs_selection`. This is only applicable to Conv and Dense layers.
- `fps` - force a layer to reach this throughput, possibly higher than the FPS used for the rest of the model. This parameter is useful to reduce the model's latency, however it is not likely to contribute to the model's throughput which is dominated by the bottleneck layer.

Glob syntax is supported to change many layers at once. For example:

```
compilation_param({conv*}, resources_allocation_strategy=min_scs_match_fps)
```

will change the resources allocation strategy of all the layers that start with `conv`.

Table 7. 16x4 mode support

Kernel type	Kernel size (HxW)	Stride (HxW)	Dilation (HxW)	Padding
Conv	1x1	1x1	1x1	SAME SAME_TENSORFLOW VALID
Conv	3x3	1x1, 2x1	1x1 2x2 (stride=1x1 only) 3x3 (stride=1x1 only) 4x4 (stride=1x1 only)	SAME SAME_TENSORFLOW
Conv	5x5	1x1, 2x1	1x1	SAME SAME_TENSORFLOW
Conv	7x7	1x1, 1x2, 2x1, 2x2	1x1	SAME SAME_TENSORFLOW
Conv	1x3, 1x5, 1x7	1x1	1x1	SAME SAME_TENSORFLOW
Conv	3x5, 3x7, 5x3, 5x7, 7x3, 7x5	1x1	1x1	SAME SAME_TENSORFLOW
Conv	3x4, 5x4, 7x4, 9x4	1x1	1x1	SAME SAME_TENSORFLOW
Conv	3x6, 5x6, 7x6, 9x6	1x1	1x1	SAME SAME_TENSORFLOW
Conv	3x8, 5x8, 7x8, 9x8	1x1	1x1	SAME SAME_TENSORFLOW
Conv	9x9	1x1	1x1	SAME SAME_TENSORFLOW
DW	3x3	1x1	1x1, 2x2	SAME SAME_TENSORFLOW
DW	5x5	1x1	1x1	SAME SAME_TENSORFLOW

Table 8. No contexts mode support

Kernel type	Kernel size (HxW)	Stride (HxW)	Dilation (HxW)
Conv	3x3	1x1, 1x2, 2x1, 2x2	1x1
Conv	7x7	2x2	1x1

HEF Parameters

Definition

```
hef_param(should_use_sequencer=value, params_load_time_compression=value)
```

Example

```
hef_param(should_use_sequencer=True, params_load_time_compression=True)
```

Description This will configure the HEF build. The command in the example enables the use of Sequencer and weights compression for optimized device configuration.

Supported hef parameters:

- `should_use_sequencer` - Using the Sequencer allows faster configurations load to device over PCIe during network activation, but removes Ethernet support for the created HEF. It defaults to `True`.
- `params_load_time_compression` - defaults to `True` and enables compressing layers parameters (weights) in the HEF for allowing faster load to device during network activation. Note that load time compression doesn't reduce the required memory space. This parameter also removes Ethernet support for the created HEF when enabled.
- `strip_mapping_info` - defaults to `False`. Remove mapping information from the HEF.

Outputs Multiplexing

Definition

```
output_mux(layers)
```

Example

```
output_mux1 = output_mux([conv7, fc1_d3])
```

Description The outputs of the given layers will be multiplexed into a single tensor before sending them back from the device to the host. Contrary to concat layers, output mux inputs do not have to share the same width, height, or numerical scale.

From TF

Definition

```
layer = from_tf(original_name)
```

Example

```
my_conv = from_tf('conv1/BiasAdd')
```

Description This command allows the use of the original (TF/ONNX) layer name in order to make sure that the correct layers are addressed, as the HN layers names and the original layers names differ.

Note: Despite its name, this commands supports original names from both TF and ONNX.

Buffers

Definition

```
buffers(layer_from, layer_to, number_of_rows_to_buffer)
buffers(layer_from, layer_to, number_of_rows_cluster_a, number_of_rows_
→cluster_b)
```

Example

```
buffers(conv1, conv2, 26)
```

Description This command sets the size of the inter-layer buffer in units of `layer_from`'s output rows. Two variants are supported. The first variant sets the total number of rows to buffer. The second variant sets two such buffer sizes, in case the compiler adds a cluster transition between these layers. The first size sets the number of rows to buffer before the cluster transition, and the second number sets the number of rows after the transition. If there is no cluster transition, only the first number is used. The second variant is mainly used in autogenerated scripts returned by `save_autogen_allocation_script()`.

Feature Splitter

Definition

```
feature_splitter(layer_from, layers_to)
```

Example

```
aux_feature_splitter0 = feature_splitter(feature_splitter0, [conv0, conv1])
```

Description Add a feature splitter layer between an existing feature splitter layer and some of its outputs. This command is used to break up a "large" feature splitter layer with many outputs into several steps.

Shape Splitter

Definition

```
shape_splitter(split_type, layer_from, layers_to)
```

Example

```
row_splitter1 = shape_splitter(SPLIT_HEIGHT, row_splitter0, [conv0, conv1])
```

Description Add a shape splitter layer between an existing shape splitter layer and some of its outputs. This command is used to break up a "large" shape splitter layer with many outputs into several steps.

Supported split types:

- `SPLIT_HEIGHT` - split the input tensor by height.
- `SPLIT_WIDTH` - split the input tensor by width.
- `SPLIT_FEATURES` - split the input tensor by features, behaves the same as `feature_splitter` command.

Platform Param

Definition

```
platform_param(param=value)
```

Examples

```
platform_param(targets=[ethernet])
platform_param(hints=[low_pcie_bandwidth])
```

Description This sets several parameters regarding the platform hosting Hailo as described below:

- **targets** - a list or a single value of hosting target restrictions such as `Ethernet` which requires disabling a set of features.

Current supported targets: `Ethernet`, which disables the following features:

- DDR portals, since the DDR access through PCIe is not available
- Context Switch (multi contexts), since DDR access is not available
- Sequencers (a fast PCIe-based model loading)

- **hints** - a list of hints or a single hint about the hosting platform such as `Low PCIE bandwidth` which optimizes performance for specific scenarios.

Current supported hints: `low_pcie_bandwidth`, Adjusts the compiler optimizations for low PCIe bandwidth.

Performance Param

Definition

```
performance_param(compiler_optimization_level=max, optimize_for_
↪batch=value)
```

Description Setting this parameter enters *performance mode*, in which the compiler will try as hard as it can to find a solution that will fit in a single context, with the highest performance. This method of compilation will require significantly longer time to complete, because the compiler tries to use very high utilization levels, that might not allocate successfully. If it fails to allocate, it automatically tries lower utilization, until it finds the highest possible utilization.

- **optimize_for_batch** - Setting this parameter will optimize the model for a specific batch size. The value should be given in number of samples.
- **compiler_optimization_level** - supports 0, 1 (default), 2, and max.

0 - returns the first feasible solution found, 1 - returns the best solution under default utilization, 2 (or max) - exhausts searches over best utilization.

Remove Node

Definition

```
remove_node(layer_name)
```

Example

```
remove_node(conv1)
```

Description removing layer from the network. This command is useful to remove layers that are given by the hn but we can remove them. Should be used internally only and with caution.

- **layer_name** - the name of the layer to remove.

5.5. Supported Layers

The following section describes the layers and parameters range that the Dataflow Compiler supports.

However, the *Parser* (that translates the original model to Hailo's internal representation) support varies across frameworks, so that some layers that are supported might not be supported on all frameworks, and vice-versa. Therefore, please also refer to the summary tables which detail all layers and their corresponding APIs in *PyTorch/ONNX* and *TensorFlow*.

5.5.1. Convolution

Convolution layers are supported with any integer values of kernel size, stride, dilation. Padding types supported are: VALID, SAME, and SAME_TENSORFLOW. The following table displays the current optimized params.

Table 9. Convolution kernel optimized parameters

Kernel (HxW)	Stride (HxW)	Dilation (HxW)	Padding
1x1	1x1, 2x1, 2x2	1x1	SAME SAME_TENSORFLOW VALID
3x3	1x1, 1x2, 2x1, 2x2	1x1 2x2 (stride=1x1 only) 3x3 (stride=1x1 only) 4x4 (stride=1x1 only) 6x6 (stride=1x1 only) 8x8 (stride=1x1 only) 16x16 (stride=1x1 only)	SAME SAME_TENSORFLOW VALID
2x2	2x1	1x1	SAME SAME_TENSORFLOW VALID
2x2, 2x3, 2x5, 2x7, 3x2, 5x2, 7x2	2x2	1x1	SAME SAME_TENSORFLOW
5x5, 7x7	1x1, 1x2, 2x1, 2x2	1x1	SAME SAME_TENSORFLOW VALID (stride=1x1 only)
6x6	2x2	1x1	SAME SAME_TENSORFLOW VALID
1x3, 1x5, 1x7	1x1, 1x2	1x1	SAME SAME_TENSORFLOW VALID (stride=1x1 only)
3x5, 3x7, 5x3, 5x7, 7x3, 7x5	1x1, 1x2	1x1	SAME SAME_TENSORFLOW VALID (stride=1x1 only)
3x1	1x1, 2x1	1x1	SAME SAME_TENSORFLOW VALID

Continued on next page

Table 9 – continued from previous page

Kernel (HxW)	Stride (HxW)	Dilation (HxW)	Padding
5x1, 7x1	1x1	1x1	SAME SAME_TENSORFLOW VALID
1x9, 3x9, 5x9, 7x9	1x1	1x1	SAME SAME_TENSORFLOW VALID
9x1, 9x3, 9x5, 9x7, 9x9	1x1	1x1	SAME SAME_TENSORFLOW VALID
3x4, 5x4, 7x4, 9x4	1x1	1x1	SAME SAME_TENSORFLOW VALID
3x6, 5x6, 7x6, 9x6	1x1	1x1	SAME SAME_TENSORFLOW VALID
3x8, 5x8, 7x8, 9x8	1x1	1x1	SAME SAME_TENSORFLOW VALID
1xW	1x1	1x1	SAME SAME_TENSORFLOW VALID
MxN	MxN	1x1	SAME SAME_TENSORFLOW VALID
MxN, where M,N in {1..16}	AxB, where A,B in {1..4}	CxD, where C,D in {1..9}	SAME SAME_TENSORFLOW VALID
Any other	Any other	Any other	SAME SAME_TENSORFLOW VALID

'W' refers to the width of the layer's input tensor, in this case the kernel width is equal to the image width

Table 10. Convolution & add kernel supported parameters

Kernel (HxW)	Stride (HxW)	Dilation (HxW)	Padding
1x1, 1x3, 1x5, 1x7	1x1	1x1	SAME SAME_TENSORFLOW VALID
3x1, 3x3, 3x5, 3x7	1x1	1x1	SAME SAME_TENSORFLOW VALID
5x1, 5x3, 5x5, 5x7	1x1	1x1	SAME SAME_TENSORFLOW VALID
7x1, 7x3, 7x5, 7x7	1x1	1x1	SAME SAME_TENSORFLOW VALID

Note: Convolution kernel with elementwise addition supports the addition of two tensors only.

Conv3D is supported with the following parameters:

Table 11. 3D Convolution supported parameters

Kernel (HxWxD)	Stride (HxWxD)	Dilation (HxWxD)	Padding	Notes
3x3x3	1x1x1	1x1x1	SAME SAME_TENSORFLOW VALID	PREVIEW
3x3xAny	1x1xAny	1x1x1	SAME SAME_TENSORFLOW VALID	PREVIEW

Note: Current limitations of Conv3D layer:

1. Models that contain Conv3D layer must have rank-4 input and output (4 dimensions at most), so the Conv3D layer must reside inside a "2D" model.
2. The input to the first 3D Conv needs to be created using a Concat layer on the Disparity dimension (after Unsqueeze).
3. The last Conv3D in the chain must have output_features = 1 (HxWxDx1), followed by a Squeeze operation, then a Conv2D or a Resize layer.

Note: Number of weights per layer <= 8MB (for all Conv layers).

Note: Padding type definitions are:

- SAME: *Symmetric padding*.
- SAME_TENSORFLOW: Identical to Tensorflow SAME padding.
- VALID: No padding, identical to Tensorflow VALID padding.

5.5.2. Max Pooling

Table 12. Max pooling kernel supported parameters

Kernel (HxW)	Stride (HxW)	Padding
2x2	1x1, 2x1, 2x2	SAME SAME_TENSORFLOW VALID
1x2	1x2	SAME SAME_TENSORFLOW VALID

Continued on next page

Table 12 – continued from previous page

Kernel (HxW)	Stride (HxW)	Padding
3x3	1x1	SAME SAME_TENSORFLOW VALID
3x3	2x2	SAME SAME_TENSORFLOW VALID
5x5, 9x9, 13x13	1x1	SAME SAME_TENSORFLOW VALID
Any other	Any other	SAME SAME_TENSORFLOW VALID

“Any other” means any kernel size or stride between 2 and the tensor’s dimensions, for example $2 \leq k_h \leq H$ where k_h is the kernel height and H is the height of the layer’s input tensor.

5.5.3. Dense

Dense kernel is supported. It is supported only after a Dense layer, a Conv layer, a Max Pooling layer, a Global Average Pooling layer, or as the first layer of the network.

When a Dense layer is after a Conv or a Max Pooling layer, the data is reshaped to a single vector. The height of the reshaped image in this case is limited to 255 rows.

5.5.4. Average Pooling

Average Pooling layers are supported with any integer values of kernel size, stride, and dilation. Padding types supported are: VALID, SAME, and SAME_TENSORFLOW. The following table displays the current optimized params.

Table 13. Average pooling kernel optimized parameters

Kernel (HxW)	Stride (HxW)	Padding
2x2	2x2	SAME SAME_TENSORFLOW VALID
3x3	1x1, 2x2	SAME SAME_TENSORFLOW
3x4	3x4	SAME SAME_TENSORFLOW VALID
5x5	1x1, 2x2	SAME SAME_TENSORFLOW
hxW	hxW	VALID
Global	n/a	n/a

‘W’ means the width of the layer’s input tensor. In other words, in this case the kernel width equals to the image width.
‘h’ means any height, from 1 up to the input tensor height.

5.5.5. Concat

This layer requires 4-dimensional input tensors (batch, height, width, features), and concatenates them in the features dimension. It supports up to 4 inputs.

5.5.6. Deconvolution

Table 14. Deconvolution kernel supported parameters

Kernel (HxW)	Rate (HxW)	Padding
16x16	8x8	SAME_TENSORFLOW
8x8	8x8	SAME_TENSORFLOW
8x8	4x4	SAME_TENSORFLOW
4x4	4x4	SAME_TENSORFLOW
4x4	2x2	SAME_TENSORFLOW
3x3	2x2	SAME_TENSORFLOW
2x2	2x2	SAME_TENSORFLOW
1x1	1x1	SAME_TENSORFLOW

5.5.7. Depthwise Convolution

Depthwise Convolution layers are supported with any integer values of kernel size, stride, and dilation. Padding types supported are: VALID, SAME, and SAME_TENSORFLOW. Utilizing a Depthwise 1x1 stride 1x1 kernel with elementwise addition, supports the addition of two tensors only

Table 15. Depthwise convolution kernel optimized parameters

Kernel (HxW)	Stride (HxW)	Dilation (HxW)	Padding
1x1	1x1	1x1	SAME SAME_TENSORFLOW VALID
2x2	2x2	1x1	SAME SAME_TENSORFLOW VALID
3x3	1x1, 2x2	1x1 2x2 (stride=1x1 only) 4x4 (stride=1x1 only)	SAME SAME_TENSORFLOW VALID (stride=1x1, dilation=1x1 only)
3x5, 5x3	1x1	1x1	SAME SAME_TENSORFLOW VALID
5x5	1x1, 2x2	1x1	SAME SAME_TENSORFLOW VALID (stride=1x1, dilation=1x1 only)
9x9	1x1	1x1	SAME SAME_TENSORFLOW

5.5.8. Group Convolution

Group Convolution is supported with all supported Convolution kernels.

For Conv 1x1/1, 1x1/2, 3x3/1, and 7x7/2, any number of output features is supported. For all other supported Conv kernels, only $OF \% 8 = 0$ or $OF < 8$ is supported, where OF is the number of output features in each group.

5.5.9. Group Deconvolution and Depthwise Deconvolution

Group Deconvolution is supported with all supported Deconvolution kernels. Only $OF \% 8 = 0$ or $OF < 8$ is supported, where OF is the number of output features in each group.

Depthwise Deconvolution is a sub case of Group Deconvolution.

5.5.10. Elementwise Multiplication and Division

Elementwise operations require:

1. Two input tensors with the same shape.
Example: [N, H, W, F], [N, H, W, F]
2. Two tensors with the same batch and spatial dimensions, one tensor has features dimension 1.
Example: [N, H, W, F], [N, H, W, 1]
3. Two tensors with the same batch and feature dimensions, one of them has spatial dimension [1, 1].
Example: [N, H, W, F], [N, 1, 1, F].
4. Two tensors with the same batch dimension, one of them has feature and spatial dimension [1, 1, 1].
Example: [N, H, W, F], [N, 1, 1, 1].

Note: The *resize layer* can broadcast a tensor from (batch, 1, 1, F) to (batch, height, width, F), where F is the number of features. This may be useful before the Elementwise Multiplication layer.

5.5.11. Add and Subtract

Add and subtract operations are supported in several cases:

1. Bias addition after Conv, Deconv, Depthwise Conv and Dense layers. Bias addition is always fused into another layer.
2. Elementwise addition and subtraction: When possible, elementwise add / sub is fused into a Conv layer as detailed above. Elementwise add / sub is supported on both "Conv like" and "Dense like" tensors, with shapes in the format shown on [Elementwise Multiplication and Division](#)
3. Addition of a constant scalar to the input tensor.
4. AddN (only in TFlite)

5.5.12. Input Normalization

Input normalization is supported as the first layer of the network. It normalizes the data by subtracting the given mean of each feature and dividing by the given standard deviation.

5.5.13. Multiplication by Scalar

This layer multiplies its input tensor by a given constant scalar.

5.5.14. Batch Normalization

Batch normalization layer is supported. When possible, it is fused into another layer such as Conv or Dense. Otherwise, it is a standalone layer.

Calculating Batch Normalization statistics in runtime using the Hailo device is not supported.

5.5.15. Resize

Two methods are supported: Nearest Neighbor (NN) and Bilinear. In both methods, the scaling of rows and columns can be different.

These methods are supported in three cases:

1. When the columns and rows scale is a float (for rows also ≤ 4096), the new sizes are integers, where `half_pixels` and `align_corners` satisfies one of the following: `align_corners=True & half_pixels=False`, `align_corners=False & half_pixels=True`, `align_corners=False & half_pixels=False`.
2. When the input shape is (batch, H, 1, F) and the output shape is (batch, rH, W, F). The number of features F stays the same and the height ratio r is integer. This case is also known as "broadcasting" (NN only).
3. When the input shape is (batch, H, W, 1) and the output shape is (batch, H, W, F). The height H and the width W stay the same. This case is also known as "features broadcasting" (NN only).

Note: `align_corners`: If True, the centers of the 4 corner pixels of the input and output tensors are aligned, preserving the values at the corner pixels. See definition [here](#) (PyTorch) and [here](#) (TensorFlow).

`half_pixel`: Relevant for Pytorch / ONNX, as defined on the [ONNX Operators](#) page, under `coordinate_transformation_mode`.

5.5.16. Depth to Space

Depth to space rearranges data from depth (features) into blocks of spatial data.

Two modes are supported (check out ONNX operators spec for more info - <https://github.com/onnx/onnx/blob/main/docs/Operators.md#depthtospace>):

1. "DCR" mode – the default mode, where elements along the depth dimension from the input tensor are rearranged in the following order: depth, column, and then row.
2. "CRD" mode – elements along the depth dimension from the input tensor are rearranged in the following order: column, row, and the depth.

MxN block size is supported, where M, N are integers, in both modes.

Table 16. Depth to space kernel supported parameters

Block size (HxW)
1x2
2x1
2x2

Depth to space is only supported when $IF \% (B_W \cdot B_H) = 0$, where IF is the number of input features, B_W is the width of the depth to space block and B_H is the height of the block.

5.5.17. Space to Depth

Space to depth rearranges blocks of spatial data into the depth (features) dimension.

1. "Classic" variant – The inverse of the Depth to Space kernel. It is identical to Tensorflow's `space_to_depth` operation. Supports MxN block size, where M, N are integers.
2. "Focus" variant – It supports the 2x2 block size. Used by models such as YOLOv5, YOLOP. It is defined by the following Tensorflow code:

```
op = tf.concat([inp[:, ::block_size, ::block_size, :], inp[:, 1::block_size, ::block_size, :],
               inp[:, ::block_size, 1::block_size, :], inp[:, 1::block_size, 1::block_size, :]], axis=3)
```

where `inp` is the input tensor.

5.5.18. Softmax

Softmax layer is supported in three cases:

1. After a "Dense like" layer with output shape (batch, features). In this case, Softmax is applied to the whole tensor.
2. After another layer, if the input tensor of the Softmax layer has a single column (but multiple features). In this case, Softmax is applied row by row.
3. After another layer, even if it has multiple columns. In this case Softmax is applied pixel by pixel on the feature dimension. This case is implemented by breaking the softmax layer to other layers.

5.5.19. LogSoftmax

Implemented by breaking the softmax layer to other layers.

5.5.20. Argmax

Argmax kernel is supported if it is the last layer of the network, and the layer before it is has a 4-dimensional output shape (batch, height, width, features).

Note: Currently argmax supports up to 64 features.

5.5.21. Reduce Max

Reduce Max is supported along the features dimension, and if the layer before it is has a 4-dimensional output shape (batch, height, width, features).

5.5.22. Reduce Sum

If the layer before it is has a 4-dimensional output shape (batch, height, width, features), the Reduce Sum layer is supported along all axes. If the layer before it has a 2-dimensional output shape (batch, features), the Reduce Sum layer is supported along the features dimension.

5.5.23. Reduce Sum Square

Reduce Sum Square is supported along the features or the spatial dimensions.

5.5.24. Reduce Min

Reduce Min is supported along the features dimension.

5.5.25. Reduce Mean

Reduce Mean is supported along the all axes.

5.5.26. Feature Shuffle

Feature shuffle kernel is supported if $F \% G = 0$, where G is the number of feature groups.

5.5.27. Features Split

This layer requires 4-dimensional input tensors (batch, height, width, features), and splits the feature dimension into sequential parts. Only static splitting is supported, i.e. the coordinates cannot be data dependent.

5.5.28. Slice

This layer requires 4-dimensional input tensors (batch, height, width, features), and crops a sequential part in each coordinate in the height, width, and features dimensions. Only static cropping is supported, i.e., the coordinates cannot be data dependent.

5.5.29. Reshape

Reshape is supported in the following cases:

“Conv like” to “Dense like” Reshape Reshaping from a Conv or Max Pooling output with shape (batch, height, W' , F') to a Dense layer input with shape (batch, F), where $F = W' \cdot F'$.

“Dense like” to “Conv like” Reshape Reshaping a tensor from (batch, F) to (batch, 1, W' , F'), where $F = W' \cdot F'$ and $F' \% 8 = 0$.

Features to Columns Reshape Reshaping a tensor from (batch, height, 1, F) to (batch, height, W' , F'), where $F = W' \cdot F'$.

Transpose, on the other side, permutes the order of the dimensions without changing them.

5.5.30. External Padding

This layer implements zeros padding as a separate layer, to support custom padding schemes that are not one of three schemes that are supported as a part of other layers (VALID, SAME and SAME_TENSORFLOW).

5.5.31. Matmul

This layer implement data driven matrices multiplication $X \times Y = Z$. Input sizes should obey matrices multiplication rules.

Support is currently available only as part of a Multi Head Attention block.

5.5.32. Multi Head Attention

This layer is a major building block for Transformer models. It receives (K, Q, V) matrices, and implements the formula:

$$\text{Softmax} \left(\frac{Q_i \cdot K_i^T}{\sqrt{d_k}} \right) \cdot V_i$$

When Q_i, K_i, V_i are matrices that result from multiplying the input matrices K, Q, V by W_i^K, W_i^Q, W_i^V respectively (W are learned matrices), i ranges from 0 to #heads - 1. Then concatenating the results after multiplying by a learned weights vector W^0 .

Hailo supports 3-dimensional tensors as inputs to this layer. An example code:

```
# keep previous shape; This code is PyTorch, on which the input shape is channels-first.
b, in_channels, h, w = prev_output.shape

# reshape [b, channels, h, w] -> [b, channels, h*w=N]
x = prev_output.flatten(2)

# transpose [b, channels, h*w] -> [b, h*w, channels] or [h*w, b, channels]
if self.batch_first:
    x = x.permute(0, 2, 1)
else:
    x = x.permute(2, 0, 1)

# self.q, self.k and self.v were defined as Linear transformations, for example: `self.
    ↪ v = nn.Linear(channels, self.d_v)`
# self.mha = nn.MultiheadAttention(..., batch_first=self.batch_first)
mha_output = self.mha(self.q(x), self.k(x), self.v(x))[0]

# transpose [b, h*w, channels] or [h*w, b, channels] -> [b, channels, h*w], then
    ↪ reshape [b, channels, h*w] -> [b, channels, h, w]
_, _, out_channels = mha_output.shape
if self.batch_first:
    unflattened = mha_output.permute(0, 2, 1).reshape(b, out_channels, h, w)
else:
    unflattened = mha_output.permute(1, 2, 0).reshape(b, out_channels, h, w)
```

5.5.33. RNN and LSTM

RNNs (Recurrent Neural Networks) and LSTMs (Long Short Term Memory) are mainly used on sequential or time series data. By using a feedback loop and an internal state, they utilize information from prior inputs to influence the current output and update the state. The sequence length of an RNN or LSTM block is the number of past or future inputs that affect the current one.

Since Hailo does not allow feedback loops, those layers are supported by the Unrolling technique, which duplicates each RNN or LSTM block by sequence length times. Therefore, high sequence lengths (more than 10 for forward/backward, or 5 for bidirectional) may lead to performance degradation.

Hailo supports the following layer flavors:

- Forward: Current input utilizes information from previous inputs; Supported for RNN and LSTM.
- Bidirectional: Current input utilizes information from previous and future inputs; Supported for LSTM.

5.5.34. Transpose

The Transpose operator permutes two dimensions of the input tensor.

Hailo supports the following Transpose operations:

- Transpose of Width <-> Column dimensions.
- Transpose of Height <-> Width dimensions, only in tensors where their complete quantized size is smaller than 1.5MB. This type of transpose is not optimal for performance, since it requires the buffering of the whole tensor, creating a "pipeline stop" that raises the latency of the model.
- Transpose of Height <-> Feature, with the same disclaimer as above.

5.5.35. Activations

See the [supported activations](#) summary table for more information.

Note: Activations are usually fused into the layer before them, however they are also supported as standalone layers when they can't be fused.

5.5.36. Square, Pow

- Square operator ($x*x$) is supported.
- Pow operator is currently supported with exponent=2 or fraction < 1.

5.5.37. Elementwise Min/Max

- Elementwise min/max are supported for two input tensors of the same shape.

5.5.38. L2 Operators

- ReduceL2 is supported.
- L2Normalization is supported.

5.5.39. Note about Symmetric Padding

The Hailo Dataflow Compiler supports symmetric padding as supported by other frameworks such as Caffe. As the SAME padding in Tensorflow is not symmetric, the only way to achieve this sort of padding is by explicitly using `tf.pad` followed by a convolution operation with `padding= 'VALID'`. The following code snippet shows how this would be done in Tensorflow (the padding generated by this code is supported by the Dataflow Compiler):

```
pad_total_h = kernel_h - 1
if strides_h == 1:
    pad_beg_h = int(ceil(pad_total_h / 2.0))
else:
    pad_beg_h = pad_total_h // 2
pad_end_h = pad_total_h - pad_beg_h

# skipping the same code for pad_total_w

inputs = tf.pad(
    inputs,
    [[0, 0], [pad_beg_h, pad_end_h], [pad_beg_w, pad_end_w], [0, 0]])
```

6. Profiler and Other Command Line Tools

6.1. Using Hailo Command Line Tools

The Hailo Dataflow Compiler offers several command line tools that can be executed from the Linux shell. Before using them, the virtual environment needs to be activated. This is explained in the [tutorials](#).

To list the available tools, run:

```
hailo --help
```

The `--help` flag can also be used to display the help message for specific tools. The following example prints the help message of the Profiler:

```
hailo profiler --help
```

The command line tools cover major parts of the Dataflow Compiler's functionality, as an alternative to using the Python API directly:

6.1.1. Model Conversion Flow

- The `hailo parser` command line tool is used to translate ONNX / TF models into Hailo archive (HAR) files.

Note: Consult [Translating Tensorflow and ONNX models](#) and `hailo parser {tf, onnx} --help` for further details on the parser arguments.

- The `hailo optimize` command line tool is used to optimize models' performance.

Note: Consult [Model Optimization](#) and `hailo optimize --help` for further details on quantization arguments.

- The `hailo compiler` command line tool is used to compile the models (in HAR format) into a hardware representation.

Note: Consult [Compilation](#) and `hailo compiler --help` for further details on compilation arguments.

6.1.2. Analysis and Visualization

The list below describes the Hailo command line interface functions for visualization and analysis:

- The `hailo analyze-noise` command is used to analyze per-layer quantization noise. Consult [Model Optimization Workflow](#) for further details.
- The `hailo params-csv` command is used to generate a CSV report with weights statistics, which is useful for analyzing the quantization.
- The `hailo tb` command is used to convert HAR or CKPT files to Tensorboard.
- The `hailo visualizer` command is used to visualize HAR files.
- The `hailo har` command is used to extract information from HAR files.

6.1.3. Tutorials

- The `hailo tutorial` command opens Jupyter with the tutorial notebooks folder.
- Select one of the tutorials to run.

6.2. Running the Profiler

The **Model Profiler** analyzes the expected performance of a compiled model on hardware and displays the optimization analysis.

To run the Profiler, use the following command:

```
hailo profiler network.har
```

The user has to set the path of the HAR file to profile, additional optional parameters may be needed.

Profiler Modes:

- **Default operation mode**
 - For runner (or HAR file) in Native state (before quantization): Presents model overview.
 - For runner (or HAR file) in Quantized state (after optimization): Presents Optimization details.
 - For runner (or HAR file) in Compiled state (or Quantized state + `--hef` flag): Presents Optimization details and compilation data, (*note*).
- **Profiler with Runtime Data:** By using the `--runtime-data <JSON_FILE>` flag with a runner (or HAR file) in Compiled state (or Quantized + `--hef`), the profiler will show **full compilation and performance data**. The JSON file is generated using `hailortcli run2 -m raw measure-fw-actions set-net <HEF-PATH>` command on the **target** platform. In case HailoRT is installed on the same machine as the Dataflow Compiler, the `--collect-runtime-data` profiler argument can be used to run the compiled model on this platform and display the full report. See example at the bottom of the [Inference Tutorial](#).
- **Accuracy Profiler:** By default, when running after quantization, only partial noise/accuracy data is displayed. The user can add the full analysis information by running the profiler on a HAR file that is a result of `hailo analyze-noise <har-path> --data-path <data-path>` tool. Another option is to add `model_optimization_config(checker_cfg, policy=enabled, analyze_mode=advanced)` to the model script before the optimization stage. See example at the [Layer Noise Analysis Tutorial](#).

Note: For single-context networks, the profiler report calculates the proposed FPS and latency of the whole model. However, on hosts with low PCIe bandwidth, it might not reflect actual performance. If the performance is worse than the profiler report values, it is recommended to try and [disable DDR buffers](#).

Note: For single-context networks, `--stream-fps` argument can be used to normalize the power and bandwidth values according to the FPS of the input stream.

Note: For big models (when the compilation results in multi-context), performance data will not be available since it depends on various runtime factors. To present performance data for those models, use the **Profiler with Runtime Data** mode.

6.2.1. Understanding the Profiler Report

The Hailo Model Profiler's report consists of the following tabs:

- **Model Overview** – Presents a summary of the model and its performance (runtime data will be required for presenting the performance of big models)
- **Optimization Details** – Presents global Optimization-related information, and also per-layer statistics, both native and quantized, used for gaining insights about degradation factors
- **Compilation & Runtime Details** – The percentage of the device(s) resources to be used by the target model, and per-layer resources information. Presents simulated performance information for small models, and, when runtime data file is provided, presents the measured performance for small and big models (see the [note](#) above).

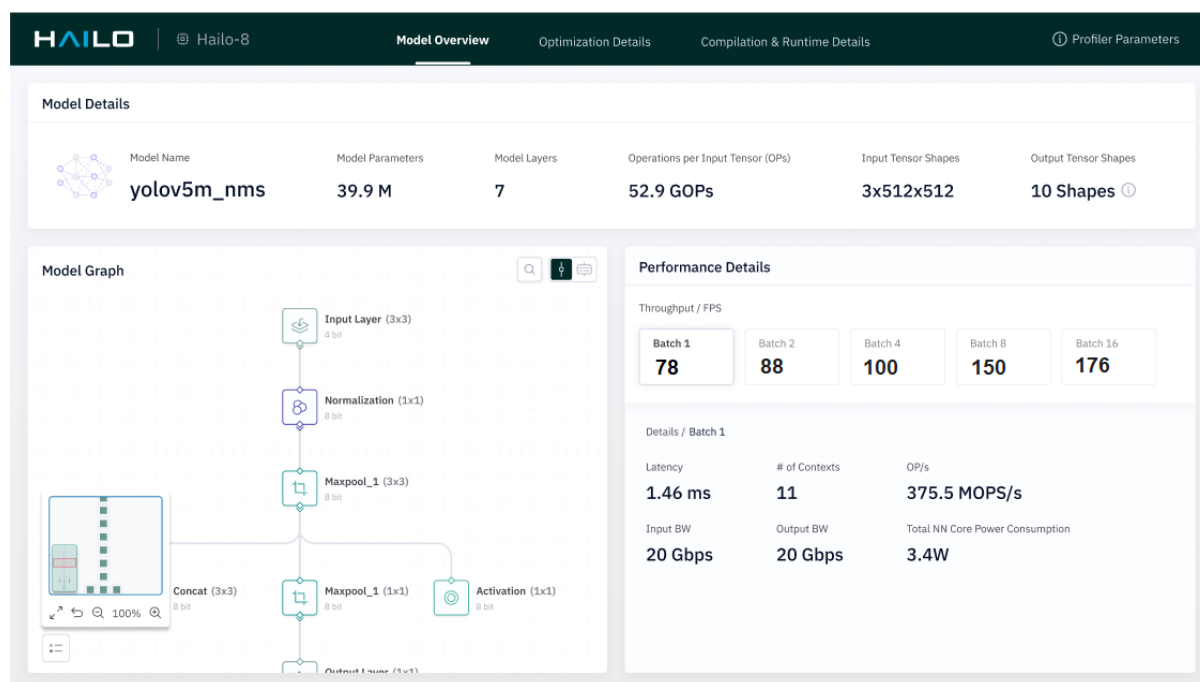


Figure 7. Model Overview Tab

The following sections describe all tabs of the report and define the fields in each one:

6.2.2. Header

Device The device that the model is compiled for. Hailo-8 for example.

Tabs There are three main tabs.

Profiler Parameters An icon on the top right corner, shows information about the Dataflow Compiler version and profiling mode.

6.2.3. Model Overview tab

Model Details (top drawer)

Model Name The model name (for example, Resnet18).

Model Parameters The number of model parameters (weights and biases), without any hardware-related overheads.

Model Layers The number of layers on the model.

Operations per Input Tensor (OPS) Total operations per input image.

Input Tensors Shapes The resolution of the model's input image (for example, HxWxC = 224x224x3).

Output Tensors Shapes The resolution of the model's output shape (for example, HxWxC = 1x1x1000).

Model Graph

Graph representation of the model that is parsed using the Hailo Parser. If the model is in FP-Optimized or more advanced state, it shows the model with the requested [model modifications](#) and further optimizations. Allows scrolling, zooming in/out, and selecting a specific layer to display Kernel, Activation and Batch Norm information.

Performance Details

Throughput / FPS The overall network FPS, per batch size (for small models, the same FPS is achieved across all batch sizes). The selection of a FPS-per-batch affects the next values.

Latency The number of milliseconds it takes the network to process an image / batch of images.

of Contexts The amount of consecutive allocations that are used for the compilation of the model on the device. Small models require one context. Large models consist of two or more contexts.

Operations per Second (OP/s) The total operations per second, based on the FPS rate.

Total NN Core Power Consumption The estimated power consumption of the neural core in watts at standard 25°C. This field excludes power consumed by the chip top and interfaces. Only appears for small models (that fit into a single context), and with accuracy of +/-20%.

Input Throughput (Input BW) The model's total input tensor throughput (bytes per second), based on the FPS rate.

Output Throughput (Output BW) The model's total tensor output throughput (bytes per second), based on the FPS rate.

6.2.4. Optimization Details Tab

Global Optimization Details (top drawer)

Optimization Level [Complexity of the optimization algorithm](#) that was used to quantize the model.

Compression Level Level of [weights compression](#) to 4-bit that was used (0 corresponds to 0% 4-bit weights, and 5 corresponds to 100% 4-bit weights).

Calibration Set Size Calibration set size that was used to optimize the model.

Ratio of Weights Resulted percentage of 4/8/16-bit weights.

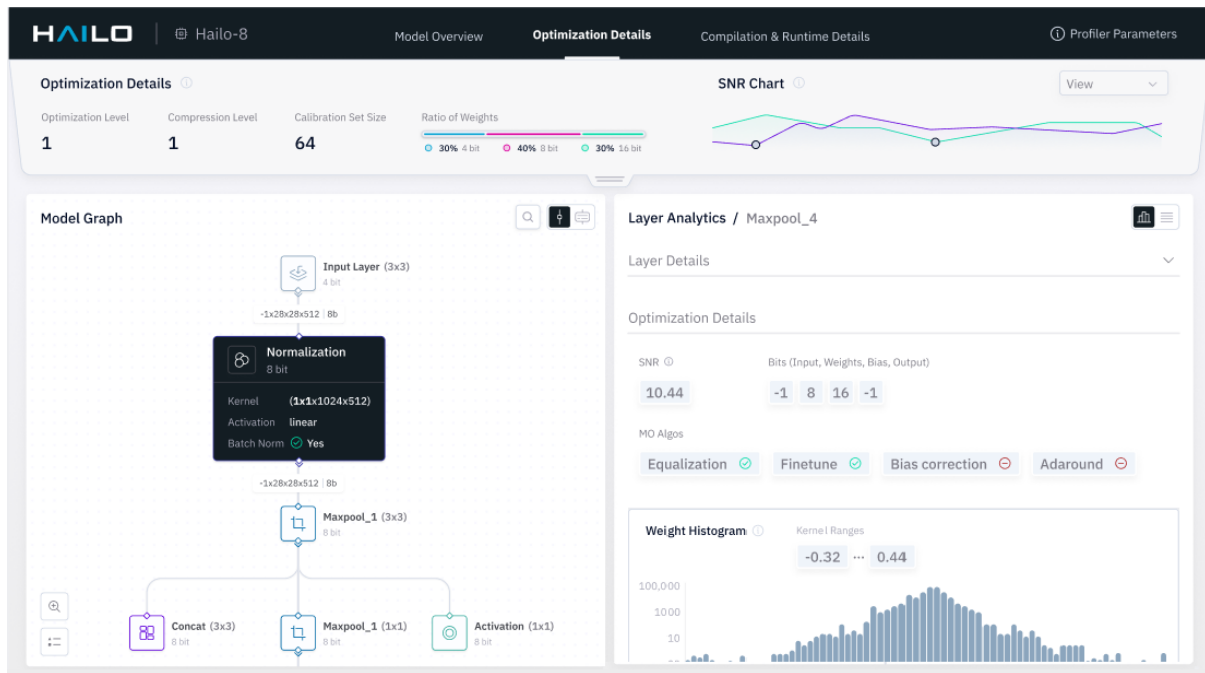


Figure 8. Optimization Details tab

Model Modifications (top drawer)

Input Conversion Input color/format conversions that were added to the model using a model script command.

Input Resize Input resize that was added to the model using a model script command.

Transpose Model (H<->W) The model was transposed using a model script command.

Normalization Input normalization that was added to the model using a model script command.

Post Processing Post-processing that was added to the model using a model script command. Can be either a single Op (like Softmax or Sigmoid) or a complex method (like NMS).

SNR Chart (top drawer)

A plot of signal-to-noise ratio between the full precision and quantized model. The SNR value is measured at the output layer(s) of the model and in each measurement, only a single layer is quantized. This graph shows the sensitivity of each layer to quantization measured in dB. The most sensitive layers (< 10dB) are highlighted. In cases where there are multiple output layers, multiple graphs will be shown.

Layer(s) with low SNR could be improved using [the following techniques](#).

Model Graph

Similar to the model graph on the Model Overview tab.

Layer Analytics

This view is the default view on the bottom-right side, it can be switched to the Table view with the icon on its top right corner.

Layer Details

For each layer, the fields presented below describe it's properties:

Layer Name The name of the layer, as defined in the HN/HAR.

Layer Type The type of operation performed by this layer (for example, convolution or max pooling).

Operations The number of multiply-accumulate operations, required by the layer.

Parameters The number of layer parameters (weights and biases), required by the layer.

Input Shape The shape of the input tensor processed by this layer.

Output Shape The shape of the output tensor processed by this layer.

Kernel Shape The shape of the kernel weights matrix (for example: A Conv layer with 3x3x64x64 means 3x3 kernel, 64 input features and 64 output features).

Strides The kernel stride.

Dilation The kernel dilation.

Groups The number of groups the kernel is split into. On most cases, groups are calculated independently. For example, convolution layers with more than one group are called "group convolution" layers.

Activation Specifies the activation function type that is performed on the output of the layer.

Batch Norm Specifies whether batch normalization was used during training on this layer.

Original Names The original name(s) of the layer(s) in the original model file (TF or ONNX), that are merged into this layer (for example a Conv layer, a Batch Norm and an Activation).

Optimization Details

Displays statistics per each layer, collected by passing the calibration set through the model:

SNR (on layer) Signal-to-noise ratio between the full precision and quantized model, measured **at this layer's output, when all the layers are quantized**. It helps to understand what is the SNR at this point on the quantized model, considering all previous layers have been quantized. Expect low on-layer SNR at the final nodes of the model, compared to the on-layer SNR at the beginning. Note the difference between this measure to the SNR chart on top drawer, which shows the SNR at the model's outputs, when only one layer is quantized at a time.

Bits (Input, Weights, Bias, Output) The amount of bits used to represent the [Input, Weights, Bias, Output] of the layer.

MO Algorithms Which algorithms were used on this layer in the Optimization phase of the model: [Equalization](#), [FineTune](#), [Bias Correction](#), and [AdaRound](#).

Weight Histogram This histogram shows the full precision weights distribution. Outliers in the distribution might cause degradation. **Kernel Ranges** are the minimum and maximum values of the weights of the layer.

Activations Histogram This histogram shows the full precision activations distribution. Outliers in the distribution might cause degradation. **Input Ranges** are the minimum and maximum values at the layer's inputs (before quantization). **Output Ranges** are the minimum and maximum values at the layer's outputs (before quantization).

Scatter Plot This graph shows the difference for representative activation values between the full precision and quantized model as measured at the output of the layer. Better quantization means the trend should be closer to a slope of 1 (that represents zero quantization noise). If a layer has some outliers (far from the slope=1 line), or the values resemble a "cloud" instead of a straight linear line, it may point to quantization errors. Use [the following techniques](#) to try and improve it.

Model Table

This view can be switched into by using the icon on the top right corner on the right side of the tab. You can select which fields are displayed, and scroll horizontally if not all the fields are visible.

The displayed fields are all the fields that appear on the [Layer Analytics / Layer Details](#), plus the SNR (per layer), and Bits information.

6.2.5. Compilation & Runtime Details Tab

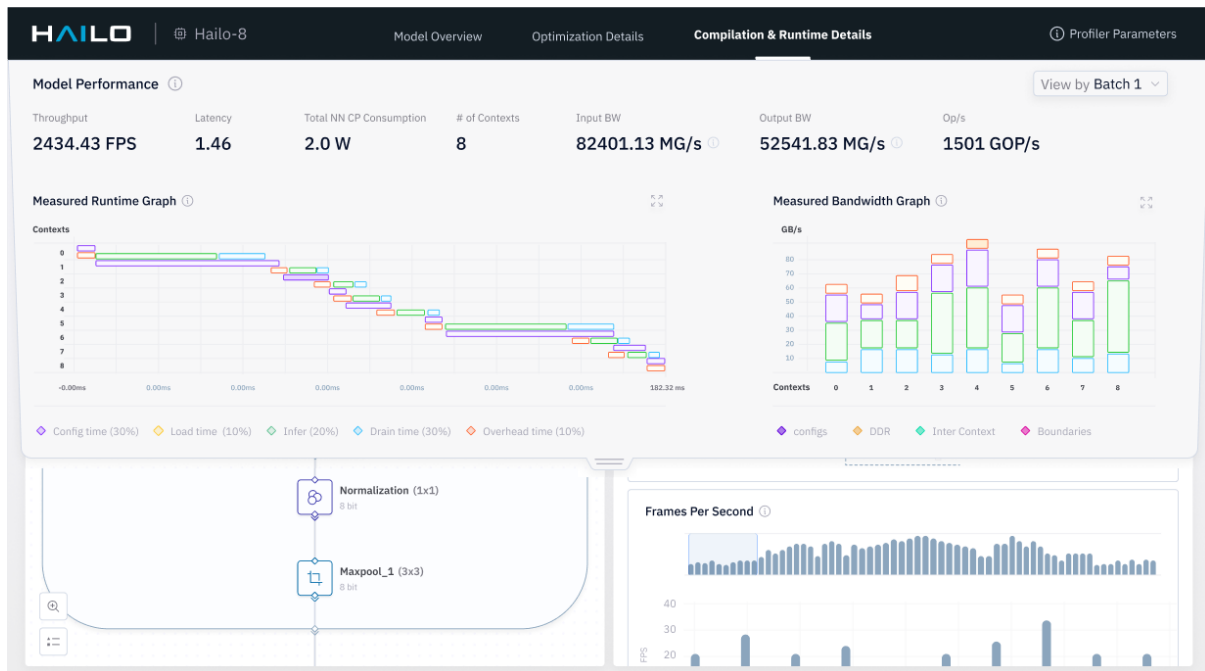


Figure 9. Compilation & Runtime tab

Model Performance (top drawer)

Displays the performance information of the model. Available for small (single context) models, or for big (multi context) models with runtime data.

The fields are the same fields from the [Model Overview tab / Performance Details](#) section.

Measured Runtime Graph (top drawer)

For large (multi context) models only. A timeline graph that shows the consecutive loading and execution of the model's contexts on the device, to complete a single inference (of a specific batch of images). Available only when `--runtime-data` is provided.

Each context consists of five phases:

- Config time – Time required to fetch weights and configurations over the DDR interface.
- Load time – Some of the fetched data needs to be prepared and loaded into the resources of the device.
- Inference time – The time it takes for the first layer to complete processing the batch.
- Drainage time – The time it takes for the last layer to complete processing the batch, measured from the end of the Inference time.
- Overhead time – Initializing / finalizing the resources before / after the inference.

Measured Bandwidth Graph (top drawer)

For large (multi context) models only. A graph describing the DDR bandwidth utilized by each context of the network (averaged over the context length). Available only when `--runtime-data` is provided.

There are four factors that contribute to DDR usage:

- **Weights/Configs** – The weights of the next context, and its configuration registers.
- **DDR Buffers** – Some contexts might include long skip connections, so *the DDR is being used* for buffering this large amount of data.
- **Inter-context tensors** – The intermediate tensors that are passed between the contexts.
- **Boundary tensors** – The boundary (edge) tensors that are fed into the model, and the outputs of the model.

Compiled Model Graph

Unlike the graph on the Model Overview and the Optimization Details tabs, this model is the result of the compilation. It may include slightly different layers, like the addition of shortcuts and inter-context nodes.

The graph starts with a “Context View” that shows the different contexts that the model was compiled into. By choosing a context, the layers that are included in it can be observed. Also, the right side of the screen will show the “Context Details” view. When a layer is selected, the right side of the screen will show the “Table View” with this layer highlighted.

Context Analytics

This is the view on the right side of the screen, when a context is selected. You can switch from this view to the Table View (per-layer) by using the icon on the top-right corner of this region.

The Context Analytics section displays information regarding the whole context in general - statistics and utilization. In case of small (single context) model, since it has only one context, the performance details of the whole model are determined from this context.

Note: The following section uses a terminology that is related to the internal structure of the neural core.

The Context Analytics view includes multiple sub-views:

- **Context Utilization**
 - **Compute Usage** The percentage of the device compute resources to be used by the target network. Can be expanded to view breakdown to sub-clusters (SCs), input aligners (IAs), and activation/pooling units (APUs).
 - **Memory Usage** The percentage of the device memory resources to be used by the target network. This figure includes both weights and intermediate results memory. Can be expanded to view breakdown to L2 (sub-cluster resource), L3 (cluster resource), and L4 (device resource) memories.
 - **Control Usage** The percentage of the device control (LCU = Layer Controller Unit) resources to be used by the target network.
- **Frames Per Second** Breakdown of the FPS of the context's layers, with the lowest (bottleneck) layer highlighted.
- **Latency Breakdown** Displays a simulation of the layers as if they were running on the device. Displays a simulation of three input tensors.

Table View

This view can be accessed by using the icon on the top right corner on the right side of the tab. Select which fields are displayed, and scroll horizontally if not all the fields are visible.

The displayed fields consist of some of the fields that appear on **Layer Analytics** tab / **Layer Details**: * Layer Name (This column stays even when scrolling) * Layer Type * Input Shape * Output Shape * Kernel Shape * Stride * Dilation * Groups * MACs * Parameters

and in addition, Hailo performance parameters:

FPS For many frames per second this layer processes. The ratio between the real layer's computed features, to the actual computed features that include padding in the width dimension. The ratio between the real layer's computed features, to the actual computed features that include padding in the width dimension. The ratio between the real layer's computed features, to the actual computed features that include padding in the width dimension.

LCUs How many Layer Controllers this layer requires (for producing the layer's FPS).

Subclusters How many sub-clusters this layer requires (for producing the layer's FPS).

Latency How much time it takes from the moment the layer starts processing an input data, until the first output is generated

Power The expected power to be consumed by the hardware resources that run this layer (an estimation; for producing the layer's FPS).

APUs How many Activation and Pooling Units this layer requires (for producing the layer's FPS).

IAs How many Input Aligners this layer requires (for producing the layer's FPS).

L3 weight cuts The relative amount of L3 (cluster-level) memory required by the layer's weights.

L3 output cuts The relative amount of L3 (cluster-level) memory required for holding the layer's outputs.

L4 cuts The relative amount of L4 (device-level) required by the layer.

defuse_mode Whether this layer was defused into multiple sub-layers, and how.

ew_add_enabled Whether this layer was merged with a nearby element-wise add operation.

active_mac_util The utilization of the this layer's code; The relative amount of cycles that the multiply-and-accumulate units are working.

width_align_util The ratio between the real layer's computed features, to the actual computed features that include padding in the width dimension.

feature_align_util The ratio between the real layer's computed features, to the actual computed features that include padding in the features dimension.

balance_fps_util How much time this layer is working. The layer with the lowest FPS has balance_fps_util = 1. Other layers are IDLE at times, therefore the utilization is lower.

mac_layers_util Of this layer's subclusters, how many are used for the calculation of the output features (not including intermediate helper operations).

effective_mac_util Multiplication of the previous factors; What is the effective (actual) MAC utilization of this layer, considering all above factors.

6.3. Using The Dataflow Compiler Studio

The Dataflow Compiler Studio allows users to parse and visualize neural network graphs efficiently.

Start by launching the Dataflow Compiler studio using:

```
hailo dfc-studio
```

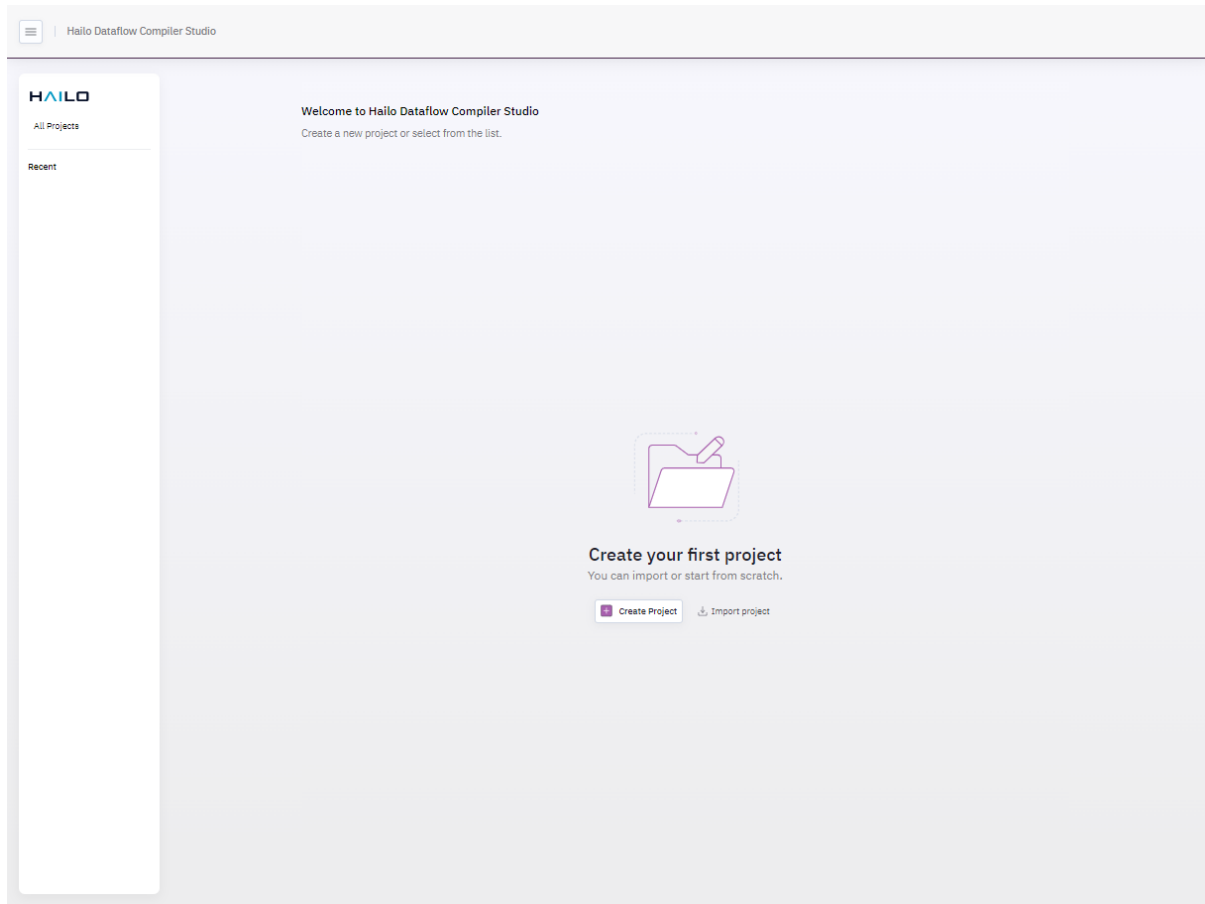


Figure 10. Dataflow Compiler Studio Welcome Page (Empty State)

On the welcome page, you can start a new project, import a project from the file system, or select an existing project that is already open on this machine. After creating a project, add and load new model file, then select the desired hardware architecture. The supported formats for original model are .onnx or .tflite.

Note: Each project can contain multiple models, but each model is parsed separately.

Note: It's also possible to load a .HAR file, but this will allow visualization of the Hailo graph only, while it won't be possible to change start and end nodes and trigger parsing again. In addition, the original graph will not be available.

Each model opens in a new, separate tab. Click 'Continue' to initiate the parsing process (Assuming .onnx/.tflite files are loaded). A red dot on the tab will indicate any unsaved changes.

You will be presented with a side-by-side view of the original graph and Hailo's graph, using the start/end nodes suggested by the Hailo parse.

To see matching layers, right-click a layer in original or hailo's grap.

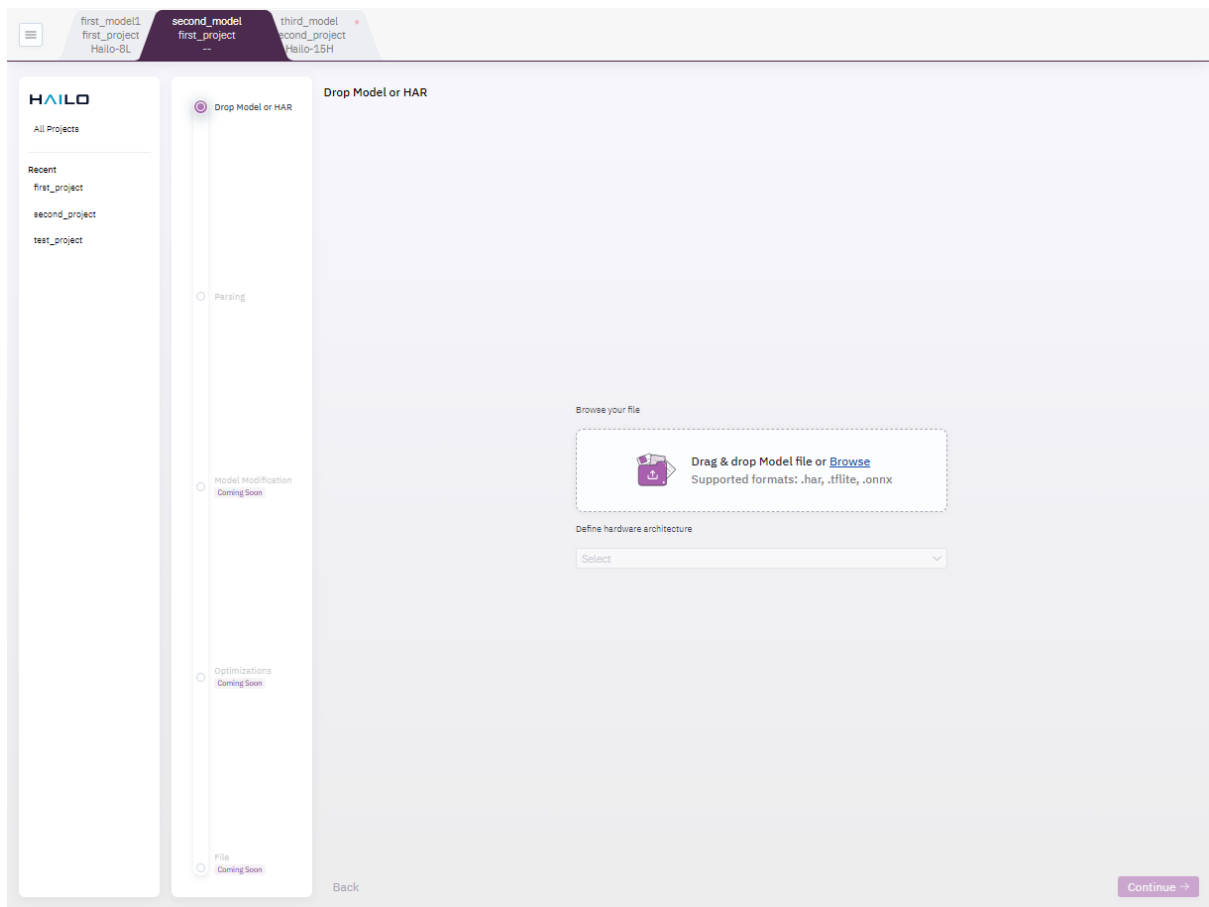


Figure 11. Dataflow Compiler Studio Browse New Model

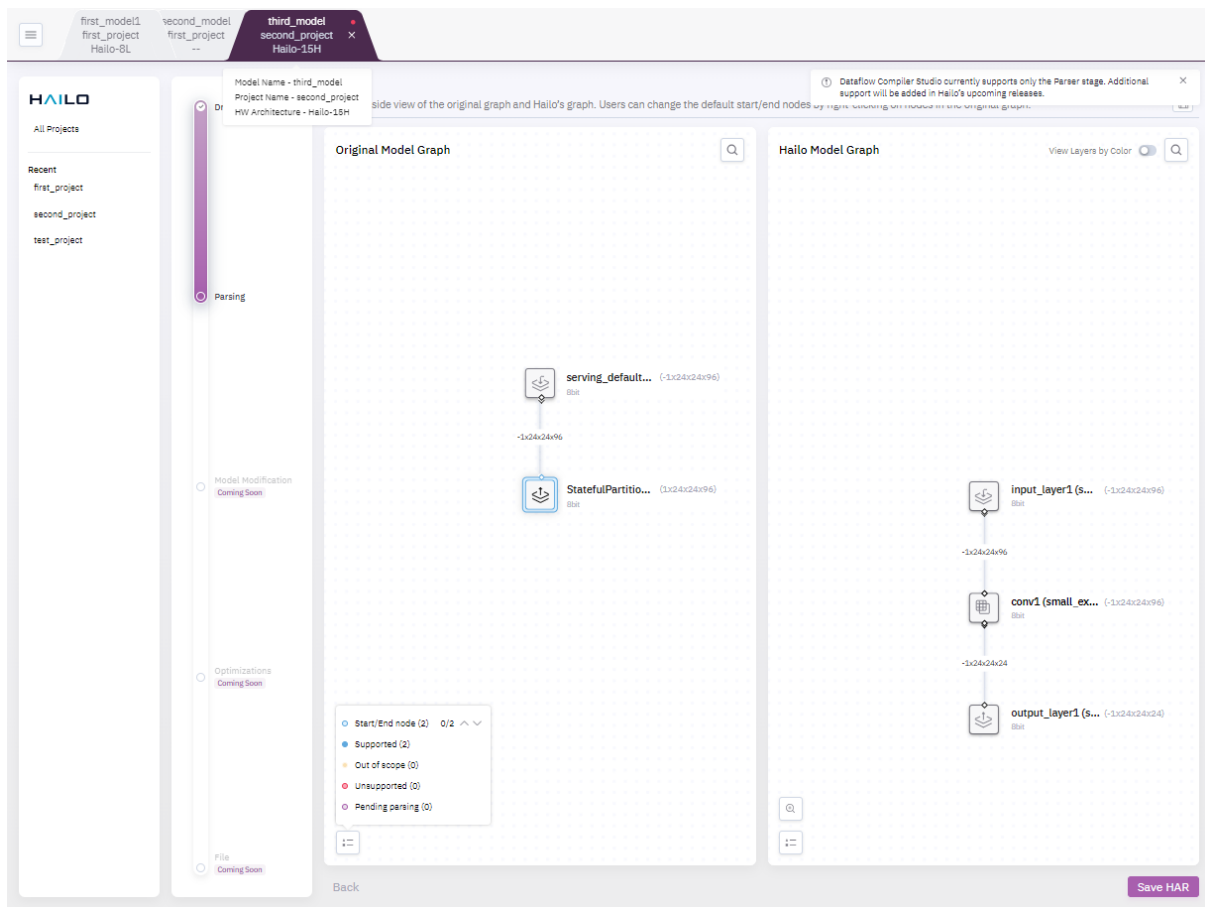


Figure 12. Dataflow Compiler Studio side-by-side View

In the bottom-left corner, you'll find a legend that illustrates the color code for each type of node: start/end nodes, supported, unsupported, nodes outside the parsing scope, or nodes awaiting parsing. This occurs if there's an attempt to modify the start/end nodes. You can navigate through the start/end nodes by using the directional arrows within the legend. To alter the start/end nodes, simply right-click on the node you wish to designate and initiate the parsing again to refresh the display. To continue with your workflow, you can save the .HAR file and move on to the subsequent stages using the command line interface (CLI).

Note: Future releases will unlock additional features. Stay tuned for updates.

Note: Currently, it's impossible to make changes to other tabs (models) while parsing is in progress in another tab. This will be fixed in Future.

7. Additional Topics

7.1. Environment Variables

In order to adjust the Dataflow Compiler behavior, the following optional functional variables could be set:

- `HAILO_CLIENT_LOGS_ENABLED`: Set to `false` to disable the log files of the Dataflow Compiler.
- `HAILO_SDK_LOG_DIR`: Defines which directory to write the logs into. Default to the working directory.
- `HAILO_SET_MEMORY_GROWTH`: Set to `false` if VRAM allocation problems occur. It disables the memory growth flag, which affects the way TensorFlow allocates and manages its memory. More information is provided [here](#).

Part II

API Reference

8. Model Build API Reference

8.1. hailo_sdk_client.runner.client_runner

Hailo DFC API client.

```
class hailo_sdk_client.runner.client_runner.ClientRunner(hn=None, ...)
    Bases: object
```

Hailo DFC API client.

```
__init__(hn=None, hw_arch=None, har=None)
    DFC client constructor
```

Parameters

- **hn** – Hailo network description (HN), as a file-like object, string, dict, or [HailoNN](#). Use None if you intend to parse the network description from Tensorflow later. Notice: This flag will be deprecated soon.
- **hw_arch** (str, optional) – Hardware architecture to be used. Defaults to hailo8.
- **har** (str or HailoArchive, optional) – Hailo Archive file path or Hailo Archive object to initialize the runner from.

```
property model_script
```

```
property modifications_meta_data
```

```
force_weightless_model(weightless=True)
    DFC API to force the model to work in weightless mode.
```

When this mode is enabled, the software emulation graph can be received from `get_tf_graph()` even when the parameters are not loaded.

Note: This graph cannot be used for running inference unless the model does not require weights.

Parameters `weightless` (bool) – Set to True to enable weightless mode. Defaults to True.

```
set_keras_model(model: hailo_model_optimization.flows.inference_flow.SimulationTrainingModel)
    Set Keras model after quantization-aware training. This method allows you to set the model after editing it externally. After setting the model, new quantized weights are generated.
```

Parameters `model` (SimulationTrainingModel) – model to set.

```
get_keras_model(context: hailo_sdk_client.exposed_definitions.ContextInfo, trainable=False) → ...
    Get a Keras model for inference. This method returns a model for inference in either native, fp-optimized, quantized, or HW mode. Editing the keras model won't affect quantization/compilation unless set_keras_model() API is being used.
```

Parameters

- **context** ([ContextInfo](#)) – inference context generated by `infer_context`.
- **trainable** (bool, optional) – indicate whether the returned model should be trainable or not. `set_keras_model()` only supports trainable models.

Example

```
>>> with runner.infer_context(InferenceContext.SDK_NATIVE) as ctx:
>>> result = runner.get_keras_model(ctx)
```

`infer(context: hailo_sdk_client.exposed_definitions.ContextInfo, dataset, ...)`

DFC API for inference. This method infers the given dataset on the model in either full-precision, emulation (quantized), or HW and returns the output.

Parameters

- `context` (`ContextInfo`) – inference context generated by `infer_context`
- `dataset` – data for Inference. The type depends on the `data_type` parameter.
- `data_type` (`InferenceDataType`) – dataset's data type, based on enum values:
 - `auto` – Automatically detection.
 - `np_array` – `numpy.ndarray`, or dictionary with input layer names as keys, and values types of `numpy.ndarray`.
 - `dataset` – `tensorflow.data.Dataset` object with a valid signature. signature should be either `((h, w, c), image_info)` or `({'input_layer1': (h1, w1, c1), 'input_layer2': (h2, w2, c2)}, image_info)` `image_info` can be an empty dict for inference
 - `npy_file` – path to a npy or npz file
 - `npy_dir` – path to a npy or npz dir, assumes the same shape to all the items
- `data_count` (`int`) – optional argument to limit the number of elements for inference
- `batch_size` (`int`) – batch size for inference

Returns list: list of outputs. Entry `i` in the list is the output of input `i`. In case the model contains more than one output, each entry is a list of all the outputs.

Example

```
>>> with runner.infer_context(InferenceContext.SDK_NATIVE) as ctx:
>>> result = runner.infer(
...     ctx,
...     dataset=tf.data.Dataset.from_tensor_slices(np.ones((1, 10))),
...     batch_size=1
... )
```

`load_model_script(model_script=None, append=False)`

DFC API for manipulation of the model build params. This method loads a script and applies it to the existing HN, i.e., modifies the specific params in each layer, and sets the model build script for later use.

Parameters

- `model_script` (`str`, `pathlib.Path`) – A model script is given as either a path to the ALLS file or commands as a string allowing the modification of the current model, before quantization / native emulation / profiling, etc. The SDK parses the script, and applies the commands as follows:
 1. Model modification related commands – These commands are executed during optimization.
 2. Quantization related commands – Some of these commands modify the HN, so after the modification, each layer (possibly) has new quantization parameters. Other commands are executed during optimization.
 3. Allocation and compilation related commands – These commands are executed during compilation.
- `append` (`boolean`) – Whether to append the commands to a previous script (if exists) or use only the new script. Addition is allowed only in native mode. Defaults to `False`.

Returns A copy of the new modified HN (JSON dictionary).

Return type dict

`load_params(params, params_kind=None)`
Load network params (weights).

Parameters

- `params` – If a string, this is treated as the path of the npz file to load. If a dict, this is treated as the params themselves, where the keys are strings and the values are numpy arrays.
- `params_kind(str, optional)` – Indicates whether the params to be loaded are native, native after BN fusion, or quantized.

Returns Kind of params that were actually loaded.

Return type str

`save_params(path, params_kind='native')`
Save all model params to a npz file.

Parameters

- `path(str)` – Path of the npz file to save.
- `params_kind(str, optional)` – Indicates whether the params to be saved are native, native after BN fusion, or quantized.

`compile()`
DFC API for compiling current model to Hailo hardware.

Returns Data of the HEF that contains the hardware representation of this model.

Return type bytes

Example

```
>>> runner = ClientRunner(har="my_model.har")
>>> compiled_model = runner.compile()
```

`infer_context(inference_context: hailo\_sdk\_client.exposed\_definitions.InferenceContext, ...)`
DFC API for generating context for inference. The context must be used with the *infer* API.

Parameters

- `inference_context(InferenceContext)` – Enum to control which inference types to use.
- `device_ids(list of str, optional)` – device IDs to create VDevice from, call `Device.scan()` to get a list of all available devices. Excludes 'params'.
- `nms_score_threshold(float, optional)` – score threshold filtering for on device nms. Relevant only when nms is used.
- `custom_infer_config` – debugging capabilities for pinpointing sources of noise in the optimization process. See Flow State Handler.
- `gpu_policy(str, Optional)` – Sets the gpu policy for emulation based inference, AUTO will distribute the inference across available GPUS using a Mirrored Strategy (Parallel DATA)
- `lora_adapter_name(str, optional)` – optional argument to specify the lora adapter name for inference.

Raises

- `HailoPlatformMissingException` – In case, HW inference is requested but HailoRT is not installed.

- `InvalidArgumentsException` - In case, `InferenceContext` is not recognized.

Example

```
>>> with runner.infer_context(InferenceContext.SDK_NATIVE) as ctx:
>>> result = runner.infer(
...     ctx,
...     dataset=tf.data.Dataset.from_tensor_slices(np.ones((1, 10))),
...     batch_size=1
... )
```

`translate_onnx_model(model=None, net_name='model', start_node_names=None, ...)`
 DFC API for parsing an ONNX model. This creates a runner with loaded HN (model) and parameters.

Parameters

- `model` (str or bytes or `pathlib.Path`) - Path or bytes of the ONNX model file to parse.
- `net_name` (str) - Name of the new HN to generate.
- `start_node_names` (list of str, optional) - List of ONNX nodes that parsing will start from.
- `end_node_names` (list of str, optional) - List of ONNX nodes, that the parsing can stop after all of them are parsed.
- `net_input_shapes` (dict or list, optional) - A dictionary describing the input shapes for each of the start nodes given in `start_node_names`, where the keys are the names of the start nodes and the values are their corresponding input shapes. Use only when the original model has dynamic input shapes (described with a wildcard denoting each dynamic axis, e.g. [b, c, h, w]). Can be a list (e.g. [b, c, h, w]) for a single input network.
- `augmented_path` - Path to save a modified model, augmented with tensors names (where applicable).
- `disable_shape_inference` - When set to True, shape inference with ONNX runtime will be disabled.
- `disable_rt_metadata_extraction` - When set to True, runtime metadata extraction will be disabled. Generating a model using `get_hailo_runtime_model()` won't be supported in this case.
- `net_input_format` - (dict of str to list of `Dims`, optional): A dictionary describing the input format for each of the start nodes given in `start_node_names`, where the keys are the names of the start nodes and the values are their corresponding input format (list of `Dims`). The defaults are as follows: - rank 2 input: [Dims.BATCH, Dims.CHANNELS] - rank 3 input: [Dims.BATCH, Dims.WIDTH, Dims.CHANNELS] - rank 4 input: [Dims.BATCH, Dims.CHANNELS, Dims.HEIGHT, Dims.WIDTH] - rank 5 input: [Dims.BATCH, Dims.CHANNELS, Dims.DISPARITY, Dims.HEIGHT, Dims.WIDTH] usage example: `net_input_format={'Conv_3': [Dims.BATCH, Dims.CHANNELS, Dims.HEIGHT, Dims.WIDTH]}`

Note: Using a non-default `start_node_names` requires the model to be shape inference compatible, meaning either it has a real input shape, or, in the case of a dynamic input shape, the `net_input_shapes` field is provided to specify the input shapes of the given start nodes. The order of the output nodes is determined by the order of the `end_node_names`.

Returns The first item is the HN JSON as a string. The second item is the params dict.

Return type tuple

`translate_tf_model(model_path=None, net_name='model', start_node_names=None, ...)`

DFC API for parsing a TF model given by a checkpoint/pb/savedmodel/tflite file. This creates a runner with loaded HN (model) and parameters.

Parameters

- `model_path` (str) – Path of the file to parse. Possible formats (recommend to move to TFLite, see user guide for more details): * SavedModel (TF2): [Deprecated] Saved model export from Keras, file named `saved_model.pb` | `pbt.txt` from the model dir. * TFLite: Tensorflow lite model, converted from ckpt/frozen/Keras to file with `.tflite` suffix.
- `net_name` (str) – Name of the new HN to generate.
- `start_node_names` (list of str, optional) – List of TensorFlow nodes that parsing will start from. If this parameter is specified, `start_node_name` should remain empty.
- `end_node_names` (list of str, optional) – List of Tensorflow nodes, which the parsing can stop after all of them are parsed.
- `tensor_shapes` (dict, optional) – [Deprecated] A dictionary containing names of tensors and shapes to set in the TensorFlow graph. Use only for placeholders with a wildcard shape.

Note:

- The order of the output nodes is determined by the order of the `end_node_names`.
- TF1.x (.ckpt/.pb) and TF2.x (.pb) models support were deprecated, it is recommended to use TFLite models (see user guide for more details).

Returns The first item is the HN JSON, as a string. The second item is the params dict.

Return type tuple

Example

```
>>> model = keras.Sequential(
...     layers.Conv2D(32, 3, activation="relu"),
...     layers.Conv2D(64, 3, activation="relu"),
...     layers.MaxPooling2D(3))
>>> model.predict(random.uniform(shape=(1, 32, 32, 3), minval=-1,
↪maxval=1))
>>> converter = tf.lite.TFLiteConverter.from_keras_model(model)
>>> tflite_model = converter.convert()
>>> with tf.io.gfile.GFile('my_model.tflite', "wb") as f:
...     f.write(tflite_model)
>>> runner = ClientRunner(hw_arch='hailo8')
>>> hn, params = runner.translate_tf_model(
...     'my_model.tflite', 'MyCoolModel', ['sequential/Conv1'], [
↪'sequential/Maxpool'])
```

`join(runner, scope1_name=None, scope2_name=None, join_action=JoinAction.NONE, join_action_info=None)`

DFC API to join two models, so they will be compiled together.

Parameters

- `runner` (`ClientRunner`) – The client runner to join to this one.
- `scope1_name` (dict or str, optional) – In case dict is given, mapping between existing scope names to new scope names for the layers of this model (see example below). In case str is given, the scope name will be used for all layers of this model. A string can be used only when there is a single scope name.

- `scope2_name(dict or str, optional)` - Same as `scope1_name` for the runner to join.

Example

```
>>> net1_scope_names = {'net1_scope1': 'net_scope1',
...                     'net1_scope2': 'net_scope2'}
>>> net2_scope_names = {'net2': 'net_scope3'}
>>> runner1.join(runner2, scope1_name=net1_scope_names,
...               scope2_name=net2_scope_names)
```

join_action (JoinAction, optional): Type of action to run in addition to joining the models:

- **NONE**: Join the graphs without any connection between them.
- **AUTO_JOIN_INPUTS**: Automatically detect inputs for both graphs and combines them into one. This only works when both networks have a single input of the same shape.
- **AUTO_CHAIN_NETWORKS**: Automatically detect the output of this model and the input of the other model, and connect them. Only works when this model has a single output, and the other model has a single input, of the same shape.
- **CUSTOM**: Supply a custom dictionary `join_action_info`, which specifies which nodes from this model need to be connected to which of the nodes in the other graph. If keys and values are inputs, the inputs are joined. If keys are outputs, and values are inputs, the networks are chained as described in the dictionary.

join_action_info (dict, optional): Join information to be given when join_action is **NONE**, as explained above.

Example

```
>>> info = {'net1/output_layer1': 'net2/input_layer2',
...         'net1/output_layer2': 'net2/input_layer1'}
>>> runner1.join(runner2, join_action=JoinAction.CUSTOM, join_action_
↳ info=info)
```

`profile(should_use_logical_layers=True, hef_filename=None, runtime_data=None, stream_fps=None)`
DFC API of the Profiler.

Parameters

- `hef_filename(str, optional)` - HEF file path. If given, the HEF file is used. If not given and the HEF from the previous compilation is cached, the cached HEF is used; Otherwise, the automatic mapping tool is used. Use `compile()` to generate and set the HEF. Only in post-placement mode. Defaults to None.
- `should_use_logical_layers(bool, optional)` - Indicates whether the Profiler should combine all physical layers into their original logical layer in the report. Defaults to True.
- `runtime_data(str, optional)` - `runtime_data.json` file path produced by `hailortcli run2 measure-fw-actions`.
- `stream_fps(float, optional)` - FPS used for power and bandwidth calculation.

Returns The first item is a JSON with the profiling result summary. The second item is a CSV table with detailed profiling information about all model layers. The third item is the latency data. Fourth is accuracy data.

Return type tuple

Example

```
>>> runner = ClientRunner(har="my_model.har")
>>> export = runner.profile()
```

`save_autogen_allocation_script(path)`

DFC API for retrieving listed operations of the last allocation in .alls format.

Parameters `path(str)` - Path where the script is saved.

Returns False if an autogenerated script was not created; otherwise it returns True.

Return type bool

`property model_name`

Get the current model (network) name.

`property model_optimization_commands`

`property hw_arch`

`property state`

Get the current model state.

`property hef`

Get the latest HEF compilation.

`property nms_config_file`

`property nms_engine`

`property nms_meta_arch`

`get_params(keys=None)`

Get the native (non-quantized) params the runner uses.

Parameters `keys(list of str, optional)` - List of params to retrieve. If not specified, all params are retrieved.

`get_params_translated(keys=None)`

Get the quantized params the SDK uses.

Parameters `keys(list of str, optional)` - List of params to retrieve. If not specified, all params are retrieved.

`get_params_fp_optimized(keys=None)`

Get the fp optimized params.

Parameters `keys(list of str, optional)` - List of params to retrieve. If not specified, all params are retrieved.

`get_params_statistics(keys=None)`

Get the optimization statistics. During the optimization stage, statistics about the model and the optimization algorithms are gathered. This method returns this information in a ModelParams structure.

Parameters `keys(list of str, optional)` - List of params to retrieve. If not specified, all params are retrieved.

`get_hn_str()`

Get the HN JSON after serialization to a formatted string.

`get_hn_dict()`

Get the HN of the current model as a dictionary.

`get_hn()`

Get the HN of the current model as a dictionary.

`get_hn_model()`

Get the [HailoNN](#) object of the current model.

`get_native_hn_str()`

Get the HN JSON after serialization to a formatted string.

```
get_native_hn_dict()
```

Get the HN of the current model as a dictionary.

```
get_native_hn()
```

Get the HN of the current model as a dictionary.

```
get_native_hn_model()
```

Get the native [HailoNN](#) object of the current model.

```
get_fp_hn_str()
```

Get the full-precision HN JSON after serialization to a formatted string.

```
get_fp_hn_dict()
```

Get the full-precision HN of the current model as a dictionary.

```
get_fp_hn_model()
```

Get the full-precision [HailoNN](#) object of the current model.

```
set_hn(hn)
```

Set the HN of the current model.

Parameters `hn` – Hailo network description (HN), as a file-like object, string, dict or [HailoNN](#).

```
save_hn(path)
```

Save the HN of the current model.

Parameters `path(str)` – Path where the hn file is saved.

```
save_native_hn(path)
```

Save the HN of the current model.

Parameters `path(str)` – Path where the hn file is saved.

```
save_har(har_path, compressed=False, save_original_model=False, compilation_only=False)
```

Save the current model serialized as Hailo Archive file.

Parameters

- `har_path` – Path for the created Hailo archive directory.
- `compressed` – Indicates whether to compress the archive file. Defaults to False.
- `save_original_model` – Indicates whether to save the original model (TF/ONNX) in the archive file. Defaults to False.
- `compilation_only` – Indicates whether to save a reduced size har, containing only compilation related data.

```
load_har(har=None)
```

Set the current model properties using a given Hailo Archive file.

Parameters `har(str or HailoArchive)` – Path to the Hailo Archive file or an initialized [HailoArchive](#) object to restore.

```
model_summary()
```

Prints summary of the model layers.

```
optimize_full_precision(calib_data=None, data_type=None)
```

Apply model optimizations to the model, keeping full-precision:

1. Fusing various layers (e.g., conv and elementwise-add, fold batch_normalization, etc.), including folding of fused layers params.
2. Apply model modification commands from the model script (e.g., resize input, transpose, color conversion, etc.)
3. Run structural optimization algorithms (e.g., dead channels removal, tiling squeeze & excite, etc.)

Parameters

- `calib_data(optional)` - Calibration data for optimization algorithms that require inference on actual input data. The type depends on the `data_type` parameter.
- `data_type(optional, CalibrationDataType)` - `calib_data`'s data type, based on enum values:
 - `auto` - Automatically detected.
 - `np_array` - `numpy.ndarray`, or dictionary with input layer names as keys, and values types of `numpy.ndarray`.
 - `dataset` - `tensorflow.data.Dataset` object with valid signature. signature should be either `((h, w, c), image_info)` or `({'input_layer1': (h1, w1, c1), 'input_layer2': (h2, w2, c2)}, image_info)` `image_info` can be an empty dict for the quantization
 - `npy_file` - path to a npy or npz file
 - `npy_dir` - path to a npy or npz dir. Assumes the same shape for all the items

`analyze_noise(dataset, data_type=CalibrationDataType.auto, data_count: Optional[int] = None, ...)`

Run layer noise analysis on a quantized model:

- Analyze the model accuracy
- Generate analysis data to be visualized in the Hailo Model profiler

Parameters

- `dataset` - data for analysis. The type depends on the `data_type` parameter.
- `data_type(optional, InferenceDataType)` - `dataset`'s data type, based on enum values:
 - `auto` - Automatically detection.
 - `np_array` - `numpy.ndarray`, or dictionary with input layer names as keys, and values types of `numpy.ndarray`.
 - `dataset` - `tensorflow.data.Dataset` object with a valid signature. signature should be either `((h, w, c), image_info)` or `({'input_layer1': (h1, w1, c1), 'input_layer2': (h2, w2, c2)}, image_info)` `image_info` can be an empty dict for inference
 - `npy_file` - path to a npy or npz file.
 - `npy_dir` - path to a npy or npz dir, assumes the same shape to all the items.
- `data_count(optional, int)` - optional argument to limit the number of elements for analysis
- `batch_size(optional, int)` - batch size for analysis
- `analyze_mode(optional, str)` - selects the analyzing mode that will run simple or advanced.

`optimize(calib_data, data_type=CalibrationDataType.auto, *, work_dir=None, checkpoint: ...)`

Apply optimizations to the model:

- Modify the network layers.
- Quantize the model's params, using optional pre-process and post-process algorithms.

Parameters

- `calib_data` - Calibration data for Equalization and quantization process. The type depends on the `data_type` parameter.

- `data_type` (`CalibrationDataType`) - `calib_data`'s data type, based on enum values:
 - `auto` - Automatically detected.
 - `np_array` - `numpy.ndarray`, or dictionary with input layer names as keys, and values types of `numpy.ndarray`.
 - `dataset` - `tensorflow.data.Dataset` object with valid signature. signature should be either `((h, w, c), image_info)` or `({'input_layer1': (h1, w1, c1), 'input_layer2': (h2, w2, c2)}, image_info)` `image_info` can be an empty dict for the quantization
 - `npy_file` - path to a npy or npz file
 - `npy_dir` - path to a npy or npz dir. Assumes the same shape for all the items
- `work_dir` (`optional, str`) - If not `None`, dump quantization debug outputs to this directory.
- `checkpoint` (`optional, SupportedStops`) - The optimization process will stop at the given checkpoint.
- `memento` (`optional, FlowCheckPoint`) - The flow memento of the optimization process. this will be use to resume the optimization process.

Returns The flow memento of the optimization process.

Return type `FlowCheckPoint`

`get_hailo_runtime_model()`

Generate model allowing to run the full ONNX graph using ONNX runtime, including the parts that are offloaded to the Hailo-8 (between the start and end nodes) and the parts that are not.

`save_parsing_report(report_path)`

Save the parsing report to a given path.

Parameters `report_path` (`string`) - Path to save the file.

`get_detected_nms_config(meta_arch, config_path=None)`

Get the detected NMS config file: anchors detected automatically from the model's post-process, and default values corresponding to the meta-architecture specified.

Parameters

- `meta_arch` (`NMSMetaArchitectures`) - Meta architecture of the NMS post process.
- `config_path` (`string, optional`) - Path to save the generated config file. Defaults to `'{meta_arch}_nms_config.json'`.

`init_lora_model(lora_weights_mapping)`

Establish the LoRA model basic state.

Parameters `lora_weights_mapping` (`str`) - A path for a json dictionary that maps between the LORA layer names and the corresponding weight variables to be added later from each new adapter.

`load_lora_weights(lora_weights_path, lora_adapter_name)`

Add LORA weights set (single adapter only) to a quantized Hailo model.

Parameters

- `lora_weights_path` (`str`) - Path to the LORA weights file, in .safetensors format.
- `lora_adapter_name` (`str`) - The name of the adapter representing the LoRA weights.

`property use_service`

`property original_model_meta`

property original_model_path

8.2. hailo_sdk_client.exposed_definitions

This module contains enums used by several SDK APIs.

```
class hailo_sdk_client.exposed_definitions.JoinAction(value)
    Bases: enum.Enum
```

Special actions to perform when joining models.

See Also The `join()` API uses this enum.

```
NONE = 'none'
    join the graphs without any connection between them.
```

```
AUTO_JOIN_INPUTS = 'auto_join_inputs'
    Automatically detects inputs for both graphs and combines them into one. This only works when both
    networks have a single input of the same shape.
```

```
AUTO_CHAIN_NETWORKS = 'auto_chain_networks'
    Automatically detects the output of this model and the input of the other model, and connect them. Only
    works when this model has a single output, and the other model has a single input, of the same shape.
```

```
CUSTOM = 'custom'
    Supply a custom dictionary join_action_info, which specifies which nodes from this model need
    to be connected to which of the nodes in the other graph. If keys and values are inputs, we join the inputs.
    If keys are outputs, and values are inputs, we chain the networks as described in the dictionary.
```

```
class hailo_sdk_client.exposed_definitions.JoinOutputLayersOrder(value)
    Bases: enum.Enum
```

Enum-like class to determine the output order of a model after joining with another model.

```
NEW_OUTPUTS_LAST = 'new_outputs_last'
    First are the outputs of this model who remained outputs, then outputs of the other model. The order in
    each sub-list is equal to the original order.
```

```
NEW_OUTPUTS_FIRST = 'new_outputs_first'
    First are the outputs of the other model, then outputs of this model who remained outputs. The order in
    each sub-list is equal to the original order.
```

```
NEW_OUTPUTS_IN_PLACE = 'new_outputs_in_place'
    If the models are chained, the outputs of the other model are inserted, in their original order, to the
    output list of this model instead of the first output which is no longer an output. If the models are joined
    by inputs, the other model's outputs are added last.
```

```
class hailo_sdk_client.exposed_definitions.NNFramework(value)
    Bases: enum.Enum
```

Enum-like class for different supported neural network frameworks.

```
TENSORFLOW_LITE = 'tflite'
    Tensorflow Lite
```

```
ONNX = 'onnx'
    ONNX
```

```
class hailo_sdk_client.exposed_definitions.States(value)
    Bases: str, enum.Enum
```

Enum-like class with all the `ClientRunner` states.

```
UNINITIALIZED = 'uninitialized'
    Uninitialized state when generating a new ClientRunner
```

```

ORIGINAL_MODEL = 'original_model'
    ClientRunner state after setting the original model path (ONNX/TF model)

HAILO_MODEL = 'hailo_model'
    ClientRunner state after parsing (calling the translate_onnx_model()/translate_tf_model()
    API)

FP_OPTIMIZED_MODEL = 'fp_optimized_model'
    ClientRunner state after calling the optimize_full_precision() API. This state includes
    all the full-precision optimization such as model modification commands.

QUANTIZED_MODEL = 'quantized_model'
    ClientRunner state after calling the optimize() API. This state includes quantized weights.

QUANTIZED_BASE_MODEL = 'quanzited_base_model'
    ClientRunner state after calling, for example, the load_lora_weights() API. This state in-
    cludes layers (e.g. LoRA layers) with non-quantized weights, that were added as a fine-tune to a quantized
    base.

QUANTIZED_SLIM_MODEL = 'quantized_slim_model'
    ClientRunner state after calling the optimize() API and saving in compilation only mode. This
    state includes only the necessary information for compilation (for example quantized weights but not
    full-precision information).

COMPILED_MODEL = 'compiled_model'
    ClientRunner state after compilation (calling the compile() API).

COMPILED_SLIM_MODEL = 'compiled_slim_model'
    ClientRunner state after compilation of a quantized slim model (calling the compile() API). This
    state allows only evaluation (profiling, inference).

class hailo_sdk_client.exposed_definitions.InferenceContext(value)
    Bases: enum.Enum

    Enum-like class with all the possible inference contexts modes

    SDK_NATIVE = 'sdk_native'
        SDK_NATIVE context is for inference of the original model (without any modification).

    SDK_FP_OPTIMIZED = 'sdk_fp_optimized'
        SDK_FP_OPTIMIZED context includes all model modification in floating-point (such as normalization, nms,
        and so on).

    SDK_QUANTIZED = 'sdk_quantized'
        SDK_QUANTIZED context is for inference of the quantized model. Used to measure degradation caused
        by quantization.

    SDK_HAILO_HW = 'sdk_hailo_hw'
        SDK_HAILO_HW inference context to run on the Hailo-HW.

    SDK_BIT_EXACT = 'sdk_bit_exact'
        SDK_BIT_EXACT (preview) bit exact emulation. Currently not all layers and mode are supported

class hailo_sdk_client.exposed_definitions.ContextInfo(*args, **kwargs)
    Bases: Protocol

    This protocol represents a Context Info object that encapsulates the values need for context Infer To create a
    Context Info Object need to run

    with runner.infer_context(*args) as ctx:
        ctx : ContextInfo

    infer_context: hailo_sdk_client.exposed_definitions.InferenceContext
        InferenceContext use for the infer API.

    open: bool
        State of the context.

```

```
graph_export: None
    SdkGraphExport Internal object used by the SDK.

gpu_policy: hailo_model_optimization.acceleras.utils.
acceleras_definitions.DistributionStrategy
    What will be the gpu distributed Policy

lora_adapter_name: Optional[str] = None
    Lora adapter name

__init__(*args, **kwargs)

class hailo_sdk_client.exposed_definitions.Dims(value)
    Bases: str, enum.Enum

    An enumeration.

    BATCH = 'batch'
    STACK = 'stack'
    CHANNELS = 'channels'
    HEIGHT = 'height'
    WIDTH = 'width'
    GROUPS = 'groups'
    HEADS = 'groups'
    DISPARITY = 'groups'
```

8.3. hailo_sdk_client.hailo_archive.hailo_archive

```
class hailo_sdk_client.hailo_archive.hailo_archive.HailoArchive(state, ...)
    Bases: object

    Hailo Archive representation.
```

8.4. hailo_sdk_client.tools.hn_modifications

```
hailo_sdk_client.tools.hn_modifications.translate_rgb_dataset(rgb_dataset, ...)
    Translate a given RGB format images dataset to YUV or BGR format images. This function is useful when the
    model expects YUV or BGR images, while the calibration images used for quantization are in RGB.
```

Parameters

- `rgb_dataset` (`numpy.ndarray`) – Numpy array of RGB format images with shape (`image_count`, `h`, `w`, `3`) to translate.
- `color_type` (`ColorType`) – type of color to translate the data to. Defaults to `yuv`.

9. Common API Reference

9.1. hailo_sdk_common.model_params.model_params

```
class hailo_sdk_common.model_params.model_params.ModelParams(params, ...)
    Bases: object
```

Dict-like class that contains all parameters used by a model such as weights, biases, etc.

9.2. hailo_sdk_common.hailo_nn.hailo_nn

```
class hailo_sdk_common.hailo_nn.hailo_nn.HailoNN(network_name=None, stage=None, ...)
    Bases: networkx.classes.digraph.DiGraph
```

Hailo NN representation. This is the Python class that corresponds to HN files.

```
stable_toposort(key=None)
    Get a generator over the model's layers, topologically sorted.
```

Example

```
>>> example_hn = '''{
...     "name": "Example",
...     "layers": {
...         "in": {"type": "input_layer", "input": [], "output": ["out"], "input_
↪shape": [-1, 10]},
...         "out": {"type": "output_layer", "input": ["in"], "output": [],
↪"input_shape": [-1, 10]}
...     }
... }'''
>>> hailo_nn = HailoNN.from_hn(example_hn)
>>> for layer in hailo_nn.stable_toposort():
...     print('The layer name is "{}"'.format(layer.name))
The layer name is "in"
The layer name is "out"
```

```
to_hn(network_name, npz_path=None, json_dump=True, should_get_default_params=False)
    Export Hailo model to JSON format (HN) and params NPZ file. The NPZ is saved to a file.
```

Parameters

- `network_name` (str) - Name of the network.
- `npz_path` (str, optional) - Path to save the parameters in NPZ format. If it is None, no file is saved. Defaults to None.
- `json_dump` (bool, optional) - Indicates whether to dump the HN to a formatted JSON, or leave it as a dictionary. Defaults to True, which means to dump.
- `should_get_default_params` (bool, optional) - Indicates whether the HN should include fields with default values. Defaults to False, which means they will not be included.

Returns The HN, as a string or a dictionary, depending on the `json_dump` argument.

```
to_hn_npz(network_name, json_dump=True, should_get_default_params=False)
    Export Hailo model to JSON format (HN) and params NPZ file. The NPZ is returned to the caller.
```

Parameters

- `network_name` (str) - Name of the network.
- `json_dump` (bool, optional) - Indicates whether to dump the HN into a formatted JSON, or leave it as a dictionary. Defaults to True, which means to dump.

- `should_get_default_params` (bool, optional) - Indicates whether the HN should include fields with default values. Defaults to False, which means they will not be included.

Returns The first item is the HN, as a string or a dictionary, depending on the `json_dump` argument. The second item contains the model's parameters as a dictionary.

Return type tuple

`set_input_tensors_shapes(inputs_shapes)`
Set the tensor shape (resolution) for each input layer.

Parameters `inputs_shapes` (dict) - Each key is a name of an input layer, and each value is the new shape to assign to it. Currently doesn't support changing number of features.

`static from_fp(fp)`
Get Hailo model from a file.

`static from_hn(hn_json)`
Get Hailo model from HN raw JSON data.

`static from_parsed_hn(hn_json, validate=True)`
Get Hailo model from HN dictionary.

9.3. hailo_sdk_common.hailo_nn.hn_definitions

```
class hailo_sdk_common.hailo_nn.hn_definitions.NMSMetaArchitectures(value)
    Bases: str, enum.Enum
```

Network meta architectures to which on-chip/ on-host post-processing can be added.

`SSD = 'ssd'`
Single Shot Detection meta architecture.

`CENTERNET = 'centernet'`
Centernet meta architecture

`YOLOV5 = 'yolov5'`
Yolov5 meta architecture

`YOLOX = 'yolox'`
Yolox meta architecture

`YOLOV5_SEG = 'yolov5_seg'`
Yolov5 seg meta architecture

`YOLOV6 = 'yolov6'`
Yolov6 meta architecture

`YOLOV8 = 'yolov8'`
Yolov8 meta architecture

`DAMOYOLO = 'damoyolo'`
Damoyolo meta architecture

Bibliography

- [Meller2019] Eldad Meller, Alexander Finkelstein, Uri Almog and Mark Grobman. "Same, same but different: Recovering neural network quantization error through weight factorization." International Conference on Machine Learning, 2019. <http://proceedings.mlr.press/v97/meller19a/meller19a.pdf>
- [Finkelstein2019] Alexander Finkelstein, Uri Almog and Mark Grobman. "Fighting quantization bias with bias." Conference on Computer Vision and Pattern Recognition Workshops, 2019. <https://arxiv.org/pdf/1906.03193.pdf>
- [Finkelstein2022] Alex Finkelstein, Ella Fuchs, Idan Tal, Mark Grobman, Niv Vosco and Eldad Meller. "QFT: Post-training quantization via fast joint finetuning of all degrees of freedom." European Conference on Computer Vision , 2022. <https://arxiv.org/pdf/2212.02634.pdf>
- [Nagel2020] Markus Nagel, Rana Ali Amjad, Mart van Baalen, Christos Louizos and Tijmen Blankevoort. "Up or Down? Adaptive Rounding for Post-Training Quantization." International Conference on Machine Learning, 2020. <https://arxiv.org/pdf/2004.10568.pdf>
- [Vosco2021] Niv Vosco, Alon Shenkler and Mark Grobman. "Tiled Squeeze-and-Excite: Channel Attention With Local Spatial Context." International Conference on Computer Vision Workshops, 2021. https://openaccess.thecvf.com/content/ICCV2021W/NeurArch/papers/Vosco_Tiled_Squeeze-and-Excite_Channel_Attention_With_Local_Spatial_Context_ICCVW_2021_paper.pdf

Python Module Index

h

`hailo_sdk_client.exposed_definitions,`
169

`hailo_sdk_client.hailo_archive.hailo_archive,`
171

`hailo_sdk_client.runner.client_runner,`
159

`hailo_sdk_client.tools.hn_modifications,`
171

`hailo_sdk_common.hailo_nn.hailo_nn,`
172

`hailo_sdk_common.hailo_nn.hn_definitions,`
173

`hailo_sdk_common.model_params.model_params,`
172